

C++ ile Veri Yapıları

ve Algoritmalar Rehberi

Temellerden İleri Tekniklere Kapsamlı Başvuru

Bu belge, veri yapılarını ve algoritmaları modern C++ (C++17) ile, her konuyu **çalışan kod örnekleriyle**, karmaşıklık analizleriyle ve çizimlerle adım adım anlatır. Temel karmaşıklık analizinden dinamik programlamaya kadar bir bilgisayar bilimi müfredatının tamamını kapsar.

Kapsam — Kısım 1 (Temeller): algoritma analizi • asimptotik gösterim (Big-O) • karmaşıklık sınıfları • C++ & STL araçları.

Kapsam — Kısım 2 (Temel Veri Yapıları): diziler & vector • bağlı listeler • yığın (stack) • kuyruk & deque • hash tabloları.

Kapsam — Kısım 3 (Ağaçlar): ikili ağaçlar • BST • AVL & kırmızı-siyah ağaç • heap & öncelik kuyruğu • trie • segment ağacı & Fenwick.

Kapsam — Kısım 4 (Graflar): gösterim • BFS/DFS • en kısa yol • MST • topolojik sıralama • birleşim-bulma (DSU).

Kapsam — Kısım 5 (Algoritma Teknikleri): sıralama • arama • böl-yönet • açgözlü • dinamik programlama (detaylı) • geri izleme • string & bit algoritmaları.

Hazırlanma Tarihi: 12 Temmuz 2026

C++17 • g++ / clang++ • STL

Contents

1	Algoritma Analizi ve Asimptotik Gösterim	2
1.1	Neden Asimptotik Analiz?	2
1.2	Big-O, Big-Omega ve Big-Theta	2
1.3	Sadeleştirme Kuralları	3
1.4	Zaman Ölçme: Teoriye Doğrulama	3
1.5	Amortize (Amortized) Analiz	4
2	C++ ve STL: Algoritma Araç Kutusu	4
2.1	STL Konteynerlerine Genel Bakış	4
2.2	<code>std::vector</code> : En Sık Kullanılan Araç	4
2.3	Yararlı STL Algoritmaları (<code><algorithm></code>)	5
2.4	Şablonlar (Templates): Jenerik Veri Yapıları	5
3	Temeller — Alıştırmalar ve Çözümler	6
3.1	Karmaşıklık Analizi	6
3.2	Big-O Muhakemesi	9
3.3	STL ile Problem Çözme	11
4	Diziler ve Dinamik Diziler	16
4.1	Statik Diziler ve Bellek Düzeni	16
4.2	Dinamik Dizi Nasıl Çalışır?	16
4.3	Dizi İşlemlerinin Karmaşıklığı	17
4.4	Sık Kullanılan Dizi Teknikleri	17
5	Bağlı Listeler	18
5.1	Tek Yönlü Bağlı Liste	18
5.2	Listeyi Ters Çevirme (Klasik Mülakat Sorusu)	19
5.3	Çift Yönlü Bağlı Liste ve Döngü Tespiti	19
6	Yığın (Stack)	20
6.1	Diziyle Yığın Uygulaması	20
6.2	Uygulama: Parantez Dengeleme	20
6.3	Uygulama: Sonraki Büyük Eleman (Monoton Yığın)	20
7	Kuyruk ve Çift Uçlu Kuyruk (Deque)	21
7.1	Dairesel Kuyruk (Circular Buffer)	21
7.2	Deque ve Kayan Pencerede Maksimum	22
8	Hash Tabloları	22
8.1	Nasıl Çalışır: Hash Fonksiyonu ve Çakışma	22
8.2	STL Hash Konteynerleri	22
8.3	Uygulama: İki Sayının Toplamı (Two Sum)	23
9	Temel Veri Yapıları — Alıştırmalar ve Çözümler	23
9.1	Diziler	23
9.2	Bağlı Listeler	26
9.3	Stack (Yığın)	29
9.4	Queue ve Deque	31
9.5	Hash Tabloları	32
9.6	Uygulama: LRU Cache	34

10 Ağaçlar ve İkili Ağaçlar	37
10.1 İkili Ağaç ve Dolaşım (Traversals)	37
10.2 Seviye-Sıralı Dolaşım (BFS) ve Yükseklik	38
11 İkili Arama Ağacı (BST)	38
11.1 Arama ve Ekleme	38
11.2 Silme: Üç Durum	39
12 Dengeli Ağaçlar: AVL ve Kırmızı-Siyah	40
12.1 Rotasyonlar: Dengelemenin Temeli	40
12.2 AVL Ağacına Ekleme	40
12.3 AVL vs Kırmızı-Siyah Ağaç	41
13 Heap ve Öncelik Kuyruğu	41
13.1 Dizile Heap Gösterimi	41
13.2 STL Öncelik Kuyruğu	42
13.3 Uygulama: En Küçük k Eleman (ve Heapsort)	42
14 Trie (Önek Ağacı)	42
15 Segment Ağacı ve Fenwick Ağacı	43
15.1 Segment Ağacı	43
15.2 Fenwick Ağacı (Binary Indexed Tree)	44
16 Ağaçlar — Alıştırmalar ve Çözümler	45
16.1 İkili Ağaç ve Dolaşım	45
16.2 İkili Arama Ağacı (BST)	48
16.3 Dengeli Ağaçlar	50
16.4 Heap ve Öncelik Kuyruğu	51
16.5 Trie	52
16.6 Segment/Fenwick Ağacı	55
16.7 Ağaç Serileştirme	57
17 Graf Gösterimi	59
17.1 Komşuluk Listesi vs Komşuluk Matrisi	59
18 Graf Dolaşımı: BFS ve DFS	59
18.1 Genişlik-Öncelikli Arama (BFS)	60
18.2 Derinlik-Öncelikli Arama (DFS)	60
19 En Kısa Yol Algoritmaları	60
19.1 Dijkstra Algoritması	61
19.2 Bellman-Ford: Negatif Kenarlar	61
19.3 Floyd-Warshall: Tüm Çiftler	62
20 Minimum Kapsayan Ağaç (MST)	62
20.1 Kruskal Algoritması (Union-Find ile)	62
20.2 Prim Algoritması	63
21 Topolojik Sıralama	63
21.1 Kahn Algoritması (BFS Tabanlı)	63

22 Graflar — Alıştırmalar ve Çözümler	64
22.1 BFS/DFS Uygulamaları	64
22.2 Döngü Tespiti ve Topolojik Sıralama	67
22.3 En Kısa Yol	69
22.4 Birleşim-Bulma (DSU)	71
22.5 Güçlü Bağlı Bileşenler (SCC)	74
23 Sıralama Algoritmaları	76
23.1 Basit Sıralamalar: Insertion Sort	76
23.2 Merge Sort: Böl ve Birleştir	76
23.3 Quick Sort: Bölümleme (Partition)	77
23.4 Counting Sort: Karşılaştırmaz Sıralama	77
24 Arama ve İkili Arama	78
24.1 Doğrusal ve İkili Arama	78
24.2 Cevap Üzerinde İkili Arama (Binary Search on Answer)	78
25 Böl ve Yönet (Divide and Conquer)	78
25.1 Uygulama: Hızlı Üs Alma (Binary Exponentiation)	79
26 Açgözlü Algoritmalar (Greedy)	79
26.1 Uygulama: Etkinlik Seçimi (Activity Selection)	79
26.2 Açgözlü Ne Zaman Yanılır?	79
27 Geri İzleme (Backtracking)	79
27.1 Uygulama: Tüm Permütasyonlar	80
27.2 Uygulama: N-Vezir Problemi	80
28 Dinamik Programlama (Ayrıntılı)	80
28.1 DP Ne Zaman Uygulanır? İki Koşul	81
28.2 İki DP Stili: Memoization vs Tabulation	81
28.3 1B DP: Merdiven Çıkma ve Bozuk Para	82
28.4 Sırt Çantası (0/1 Knapsack)	83
28.5 En Uzun Ortak Alt Dizi (LCS)	83
28.6 En Uzun Artan Alt Dizi (LIS)	84
28.7 Düzenleme Uzaklığı (Edit Distance / Levenshtein)	84
28.8 Grid Üzerinde DP: Minimum Yol Toplamı	85
28.9 Bitmask DP: Gezgin Satıcı (TSP)	85
28.10 Ağaç Üzerinde DP	85
28.11 Özetle: DP Problemlerini Tanımak	86
29 String Algoritmaları	86
29.1 KMP: Doğrusal Zamanda Örüntü Arama	86
29.2 Uygulama: Palindrom Kontrolü ve Sayma	87
30 Bit Manipülasyonu	87
31 Algoritma Teknikleri — Alıştırmalar ve Çözümler	88
31.1 Sıralama	88
31.2 Arama	90
31.3 Böl ve Yönet	93
31.4 Açgözlü (Greedy)	94
31.5 Geri İzleme (Backtracking)	97
31.6 Dinamik Programlama	99

31.7 String	102
31.8 Bit Manipülasyonu	104
32 Kapanış: Doğru Aracı Seçmek	106

KISIM 1

Temeller ve Algoritma Analizi

1. Algoritma Analizi ve Asimptotik Gösterim

Bir algoritmayı değerlendirirken iki temel kaynağı ölçeriz: **zaman** (kaç işlem yapılır) ve **bellek** (ne kadar yer tutulur). Bir programı saniye cinsinden ölçmek makineye, derleyiciye ve o anki yüke bağlı olduğundan güvenilir değildir. Bunun yerine, girdi boyutu n büyürken çalışma süresinin *nasıl büyüdüğünü* tarif ederiz. Bu, asimptotik analizdir.

1.1 Neden Asimptotik Analiz?

İki algoritmayı karşılaştırmak istediğimizde, küçük girdilerdeki farklar genelde önemsizdir; asıl mesele girdi büyüdüğünde ne olacağıdır. Örneğin $100n$ işlem yapan bir algoritma, n^2 işlem yapan bir algoritmadan $n > 100$ için daha hızlıdır ve n büyüdükçe fark uçurumlaşır.

Karmaşıklık	$n = 10$	$n = 100$	$n = 10^3$	$n = 10^6$
$\mathcal{O}(1)$	1	1	1	1
$\mathcal{O}(\log n)$	3	7	10	20
$\mathcal{O}(n)$	10	100	1000	10^6
$\mathcal{O}(n \log n)$	33	664	10^4	2×10^7
$\mathcal{O}(n^2)$	100	10^4	10^6	10^{12}
$\mathcal{O}(2^n)$	1024	10^{30}	—	—

Bilgi

Rekabetçi programlamada pratik bir kural: modern bir işlemci saniyede kabaca 10^8 – 10^9 basit işlem yapar. Girdi $n \leq 10^6$ ise $\mathcal{O}(n)$ veya $\mathcal{O}(n \log n)$ hedeflenir; $n \leq 10^3$ ise $\mathcal{O}(n^2)$ kabul edilebilir; $n \leq 20$ ise $\mathcal{O}(2^n)$ üstel çözümler bile geçer.

1.2 Big-O, Big-Omega ve Big-Theta

Üç temel gösterim, bir fonksiyonun büyüme hızını sınırlar:

Gösterim	Ad	Anlamı
$O(f(n))$	Üst sınır	En kötü durumda çalışma süresi <i>en fazla</i> bu kadar büyür.
$\Omega(f(n))$	Alt sınır	Çalışma süresi <i>en az</i> bu kadardır.
$\Theta(f(n))$	Sıkı sınır	Hem üst hem alt sınır; büyüme <i>tam olarak</i> bu mertebede.

Pratikte en çok Big-O kullanılır çünkü genelde “en kötü durumda ne kadar kötü olabilir?” sorusunu sorarız. Resmî tanım: $f(n) = O(g(n))$ demek, öyle pozitif sabitler c ve n_0 vardır ki her $n \geq n_0$ için $f(n) \leq c \cdot g(n)$ olur. Yani yeterince büyük n için f , g ’nin sabit katından fazla büyüzmez.

1.3 Sadeleştirme Kuralları

Big-O ifadelerini sadeleştirirken iki kural yeterlidir:

Kural	Açıklama ve örnek
Sabitleri at	$O(5n) \rightarrow O(n)$, $O(n/2) \rightarrow O(n)$.
Baskın terimi tut	$O(n^2 + n) \rightarrow O(n^2)$, $O(n^2 + n \log n) \rightarrow O(n^2)$.

İç içe döngüleri analiz etme

Aşağıdaki üç örnek, döngü yapısından karmaşıklığın nasıl okunacağını gösterir.

```

1 // (1) Tek döngü -> O(n)
2 for (int i = 0; i < n; ++i)
3     sum += a[i];
4
5 // (2) İç içe döngü -> O(n^2)
6 for (int i = 0; i < n; ++i)
7     for (int j = 0; j < n; ++j)
8         if (a[i] + a[j] == target) /* ... */;
9
10 // (3) Yarıya bölerek ilerleyen döngü -> O(log n)
11 for (int i = 1; i < n; i *= 2)
12     process(i); // i: 1, 2, 4, 8, ... n => log2(n) adım

```

1.4 Zaman Ölçme: Teoriyi Doğrulama

Teorik analizi pratikte doğrulamak için <chrono> kütüphanesiyle gerçek süreyi ölçebiliriz.

```

1 #include <chrono>
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     const int n = 100000;
7     auto start = chrono::high_resolution_clock::now();
8
9     long long sum = 0;
10    for (int i = 0; i < n; ++i)
11        for (int j = 0; j < n; ++j) // O(n^2): 10^10 işlem -> yavaş!
12            sum += i;

```

```

13
14     auto end = chrono::high_resolution_clock::now();
15     auto ms = chrono::duration_cast<chrono::milliseconds>(end - start);
16     cout << "Sure: " << ms.count() << " ms\n";
17     return 0;
18 }

```

1.5 Amortize (Amortized) Analiz

Bazı işlemler tek tek pahalı görünür ama bir dizi işlem üzerinden ortalaması ucuzdur. Klasik örnek `std::vector`'ün `push_back`'idir: dolduğunda kapasitesini iki katına çıkarıp tüm elemanları kopyalar ($\mathcal{O}(n)$), ama bu nadiren olur. n ekleme toplamda $\mathcal{O}(n)$ sürer, dolayısıyla ekleme başına *amortize* maliyet $\mathcal{O}(1)$ 'dir.

Dikkat

$\mathcal{O}()$ en kötü durumu tarif eder ama her zaman gerçekçi maliyeti yansıtmayabilir. Hash tablosunda arama en kötü $\mathcal{O}(n)$ 'dir (tüm anahtarlar çakışırsa) ama *ortalama* $\mathcal{O}(1)$ 'dir. Hangi durumdan bahsettiğini her zaman belirt.

2. C++ ve STL: Algoritma Araç Kutusu

Modern C++, veri yapıları ve algoritmalar için zengin bir standart kütüphane (STL) sunar. Kendi veri yapılarını yazmayı öğrenmek eğitseldir, ancak gerçek kodda çoğu zaman STL'in test edilmiş, optimize edilmiş bileşenlerini kullanırsın. Bu bölüm, rehber boyunca kullanacağımız araçları tanıtır.

2.1 STL Konteynerlerine Genel Bakış

Konteyner	Kullanım	Erişim
<code>vector</code>	Dinamik dizi; en çok kullanılan	$\mathcal{O}(1)$ indeks
<code>array</code>	Sabit boyutlu dizi	$\mathcal{O}(1)$ indeks
<code>deque</code>	Çift uçlu kuyruk	$\mathcal{O}(1)$ uçlar
<code>list</code>	Çift yönlü bağlı liste	$\mathcal{O}(n)$ arama
<code>map / set</code>	Sıralı (kırmızı-siyah ağaç)	$\mathcal{O}(\log n)$
<code>unordered_map / set</code>	Hash tablosu	$\mathcal{O}(1)$ ort.
<code>stack / queue</code>	Yığın / kuyruk (adaptör)	$\mathcal{O}(1)$ uç
<code>priority_queue</code>	İkili heap	$\mathcal{O}(\log n)$

2.2 `std::vector`: En Sık Kullanılan Araç

```

1 #include <vector>
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     vector<int> v = {5, 3, 8, 1}; // liste ilklendirme
7     v.push_back(9);              // sona ekle -> {5,3,8,1,9}

```

```

8     v.pop_back();                // sondan sil -> {5,3,8,1}
9
10    cout << v[0] << " " << v.at(2) << "\n"; // 5 8 (.at sinir kontrolu yapar)
11    cout << "Boyut: " << v.size() << "\n";
12
13    // Aralik tabanlı dongu
14    for (int x : v) cout << x << " ";
15    cout << "\n";
16
17    // 2 boyutlu vector (n x m, sifiryla dolu)
18    int n = 3, m = 4;
19    vector<vector<int>> grid(n, vector<int>(m, 0));
20    grid[1][2] = 7;
21    return 0;
22 }

```

2.3 Yararlı STL Algoritmaları (<algorithm>)

```

1  #include <algorithm>
2  #include <numeric>
3  #include <vector>
4  using namespace std;
5
6  vector<int> v = {5, 3, 8, 1, 3};
7
8  sort(v.begin(), v.end());           // artan: {1,3,3,5,8}
9  sort(v.begin(), v.end(), greater<int>()); // azalan: {8,5,3,3,1}
10 reverse(v.begin(), v.end());        // ters cevir
11
12 int maxVal = *max_element(v.begin(), v.end()); // en buyuk
13 int minVal = *min_element(v.begin(), v.end()); // en kucuk
14 int sum = accumulate(v.begin(), v.end(), 0); // toplam
15
16 // Ikili arama (dizi SIRALI olmalı)
17 bool found = binary_search(v.begin(), v.end(), 5);
18 auto it = lower_bound(v.begin(), v.end(), 3); // >= 3 olan ilk konum
19
20 // Tekrarları kaldır (once sirala): unique + erase idiomu
21 sort(v.begin(), v.end());
22 v.erase(unique(v.begin(), v.end()), v.end());

```

İpucu

Rekabetçi programlama ve hızlı prototipleme için sık kullanılan kısaltmalar: `using ll = long long;` (taşmayı önlemek için), `#define pb push_back,` ve girdi/çıkış hızı için main başında `ios::sync_with_stdio(false); cin.tie(nullptr);`.

2.4 Şablonlar (Templates): Jenerik Veri Yapıları

Kendi veri yapılarımızı herhangi bir tip için çalışacak şekilde yazmak için şablonları kullanırız. Rehber boyunca yapıları çoğunlukla `int` ile göstereceğiz ama gerçekte şu kalıpla jenerikleştirilirler:

```

1  template <typename T>
2  class Stack {
3      vector<T> data;
4  public:
5      void push(const T& x) { data.push_back(x); }

```

```

6     void pop()           { data.pop_back(); }
7     T& top()             { return data.back(); }
8     bool empty() const   { return data.empty(); }
9     size_t size() const   { return data.size(); }
10 };
11
12 // Kullanım: her tip için aynı kod
13 Stack<int> intStack;      intStack.push(5);
14 Stack<string> strStack;   strStack.push("merhaba");

```

Bilgi

Bu rehberdeki tüm kod örnekleri C++17 ile derlenir. Önerilen komut: `g++ -std=c++17 -Wall -Wextra -O2 program.cpp -o program`. Örnekleri kendin derleyip çalıştırarak, değiştirerek denemek öğrenmenin en etkili yoludur.

3. Temeller — Alıştırmalar ve Çözümler

Bu bölüm, KISIM 1’de işlenen algoritma analizi (Big-O), karmaşıklık muhakemesi ve C++ STL konularını pekiştirmek için hazırlanmış çözümlü sorular içerir. Her çözümde ilgili kod parçasının zaman ve gerektiğinde uzay karmaşıklığı açıkça belirtilmiştir.

3.1 Karmaşıklık Analizi

Soru 1

Aşağıdaki kod parçasının zaman karmaşıklığını Big-O cinsinden bulun ve nedenini açıklayın.

```

1 int count = 0;
2 for (int i = 0; i < n; i++)
3     for (int j = 0; j < n; j++)
4         count++;

```

Çözüm. Dıştaki döngü i için tam olarak n kez, içteki döngü j için her i değerinde n kez çalışır. Toplam yineleme sayısı $n \times n = n^2$ olur. `count++` işlemi sabit zamanlı ($\mathcal{O}(1)$) olduğundan toplam maliyet n^2 ile orantılıdır.

```

1 // Toplam yineleme:  $n * n = n^2$ 
2 // Zaman karmaşıklığı:  $\mathcal{O}(n^2)$ 
3 int countPairs(int n) {
4     int count = 0;
5     for (int i = 0; i < n; i++) //  $n$  kez
6         for (int j = 0; j < n; j++) // her  $i$  için  $n$  kez
7             count++; //  $\mathcal{O}(1)$  işlem
8     return count;
9 }

```

Bilgi

İç içe iki bağımsız döngü, her ikisi de n ’e kadar sayıyorsa çarpım kuralıyla $\mathcal{O}(n^2)$ verir. Karmaşıklık n ’e bağlı toplam yineleme sayısı ile belirlenir.

Soru 2

Aşağıdaki döngünün zaman karmaşıklığı nedir? i değişkeninin nasıl değiştiğine dikkat edin.

```

1 for (int i = n; i >= 1; i /= 2)
2     doWork(); // O(1)

```

Çözüm. i her adımda yarıya iniyor: $n, n/2, n/4, \dots, 1$. Kaç adımda 1'e ulaşırız? n 'i kaç kez 2'ye böleriz sorusunun cevabı $\log_2 n$ 'dir. Dolayısıyla döngü yaklaşık $\log_2 n$ kez çalışır ve her adım $\mathcal{O}(1)$ olduğundan toplam karmaşıklık $\mathcal{O}(\log n)$ olur.

```

1 // i: n -> n/2 -> n/4 -> ... -> 1
2 // Adım sayısı: log2(n)
3 // Zaman karmaşıklığı: O(log n)
4 int countSteps(int n) {
5     int steps = 0;
6     for (int i = n; i >= 1; i /= 2)
7         steps++; // O(1)
8     return steps; // yaklaşık floor(log2(n)) + 1
9 }

```

İpucu

Bir döngü değişkeni her adımda sabit bir çarpanla (2, 3, ...) çarpılıyor veya bölünüyorsa, döngü $\mathcal{O}(\log n)$ çalışır. Sabit bir sayı eklenip çıkarılıyorsa $\mathcal{O}(n)$ çalışır.

Soru 3

İki ayrı (iç içe olmayan) döngüden oluşan aşağıdaki kodun karmaşıklığını bulun.

```

1 for (int i = 0; i < n; i++)
2     a(); // O(1)
3 for (int j = 0; j < n * n; j++)
4     b(); // O(1)

```

Çözüm. Birinci döngü n kez, ikinci döngü n^2 kez çalışır. Ardışık (peş peşe) çalışan bloklarda maliyetler toplanır: $n + n^2$. Big-O gösteriminde en baskın terim alınır ve sabit katsayılar ihmal edilir; büyük n için n^2 terimi n terimini domine eder. Sonuç $\mathcal{O}(n^2)$ 'dir.

```

1 // Birinci dongu: n islem
2 // İkinci dongu: n*n islem
3 // Toplam: n + n^2 => baskin terim n^2
4 // Zaman karmaşıklığı: O(n^2)
5 long long total(int n) {
6     long long ops = 0;
7     for (int i = 0; i < n; i++) ops++; // n
8     for (int j = 0; j < (long long)n * n; j++) ops++; // n^2
9     return ops;
10 }

```

Toplam kuralı

Ardışık kod bloklarında karmaşıklıklar *toplanır*; iç içe döngülerde *çarpılır*. Toplamda yalnızca en hızlı büyüyen terim korunur: $\mathcal{O}(n) + \mathcal{O}(n^2) = \mathcal{O}(n^2)$.

Soru 4

Bir dizide bir elemanı aramak için (a) doğrusal arama ve (b) önceden sıralanmış dizide ikili arama kullanılıyor. Her iki yaklaşımın karmaşıklığını karşılaştırın; hangisi ne zaman tercih edilmelidir?

Çözüm. Doğrusal arama en kötü durumda diziyi baştan sona tarar: $\mathcal{O}(n)$. İkili arama her adımda arama uzayını yarıya böldüğü için $\mathcal{O}(\log n)$ 'dir, ancak dizinin *sıralı* olmasını gerektirir. Diziyi bir kez sıralamak $\mathcal{O}(n \log n)$ maliyetlidir. Tek bir arama için doğrusal arama yeterlidir; ancak aynı dizide çok sayıda (q adet) arama yapılacaksa, bir kez sıralayıp her sorguyu $\mathcal{O}(\log n)$ yapmak ($\mathcal{O}(n \log n + q \log n)$) doğrusal aramanın $\mathcal{O}(q \cdot n)$ maliyetinden çok daha iyidir.

```

1  #include <algorithm>
2  #include <vector>
3  using namespace std;
4
5  // Doğrusal arama:  $\mathcal{O}(n)$ , sıralı olma şartı yok
6  int linearSearch(const vector<int>& v, int target) {
7      for (int i = 0; i < (int)v.size(); i++)
8          if (v[i] == target) return i; // en kötü durum:  $n$  kar.
9      return -1;
10 }
11
12 // İkili arama:  $\mathcal{O}(\log n)$ , dizi SIRALI olmalı
13 bool binarySearch(const vector<int>& sorted, int target) {
14     int lo = 0, hi = (int)sorted.size() - 1;
15     while (lo <= hi) {
16         int mid = lo + (hi - lo) / 2; // tasmayı onler
17         if (sorted[mid] == target) return true;
18         if (sorted[mid] < target) lo = mid + 1; // sağ yarı
19         else hi = mid - 1; // sol yarı
20     }
21     return false;
22 }
```

Dikkat

İkili arama yalnızca sıralı dizide doğru çalışır. Sıralama maliyetini de hesaba katın: yalnızca birkaç arama yaparsanız sıralama zahmetine değmez.

Soru 5

`std::vector::push_back` işleminin *amortize* $\mathcal{O}(1)$ olduğu söylenir, oysa bazı çağrılar tüm elemanları yeni belleğe kopyalar. Bu nasıl mümkün olur? n eleman eklemenin toplam maliyetini açıklayın.

Çözüm. `vector` dolduğunda kapasitesini genellikle iki katına çıkarır ve mevcut elemanları yeni belleğe kopyalar. Bu kopyalama tek bir `push_back` için $\mathcal{O}(n)$ olabilir. Ancak yeniden boyutlandırma seyrek gerçekleşir: kapasite $1, 2, 4, 8, \dots, n$ şeklinde büyür. n ekleme boyunca toplam kopyalama sayısı $1 + 2 + 4 + \dots + n < 2n$ olur; bu geometrik seri $\mathcal{O}(n)$ toplam maliyet demektir. Toplam maliyeti işlem sayısına ($\mathcal{O}(n)/n$) bölünce her işlem başına *amortize* $\mathcal{O}(1)$ elde edilir. Bu, "her tek çağrı $\mathcal{O}(1)$ " değil, " n çağrının ortalaması $\mathcal{O}(1)$ " anlamına gelir.

```

1  #include <vector>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      vector<int> v;
7      //  $n$  push_back çağrısının TOPLAM maliyeti  $\mathcal{O}(n)$ 
8      // => çağrı başına amortize  $\mathcal{O}(1)$ 
9      for (int i = 0; i < 16; i++) {
10         v.push_back(i);
11     }
```

```

11 // Kapasite dolunca ikiye katlanır; kopyalama seyrek olur.
12 cout << "size=" << v.size()
13     << " capacity=" << v.capacity() << "\n";
14 }
15 return 0;
16 }

```

Bilgi

Amortize analiz, pahalı ama seyrek işlemlerin maliyetini ucuz sık işlemlere yayar. `reserve(n)` ile kapasiteyi önceden ayırırsanız yeniden tahsisleri tamamen ortadan kaldırıp performansı artırabilirsiniz.

Soru 6

Aşağıdaki iki fonksiyonun *uzay* (bellek) karmaşıklığını karşılaştırın. Çıktı bir dizi döndürülmesi için ayrılan bellek dahil edilmelidir.

```

1 long long sumA(int n) { // A
2     long long s = 0;
3     for (int i = 1; i <= n; i++) s += i;
4     return s;
5 }
6 std::vector<int> sumB(int n) { // B
7     std::vector<int> v;
8     for (int i = 1; i <= n; i++) v.push_back(i);
9     return v;
10 }

```

Çözüm. `sumA` yalnızca sabit sayıda değişken (`s`, `i`) kullanır; girdi büyüklüğünden bağımsız sabit bellek harcar: $\mathcal{O}(1)$ uzay. `sumB` ise n elemanlı bir `vector` oluşturur ve döndürür; kullandığı bellek n ile doğrusal artar: $\mathcal{O}(n)$ uzay. Her ikisinin de zaman karmaşıklığı $\mathcal{O}(n)$ 'dir, ancak uzay maliyetleri farklıdır. Genel kural: sonuç için gereken ek belleği (yardımcı yapılar, döndürülen koleksiyon) sayarız; girdinin kendisini çoğu zaman saymayız.

```

1 // sumA: yardımcı bellek sabit -> uzay  $\mathcal{O}(1)$ , zaman  $\mathcal{O}(n)$ 
2 // sumB:  $n$  elemanlı vektor -> uzay  $\mathcal{O}(n)$ , zaman  $\mathcal{O}(n)$ 

```

İpucu

Uzay karmaşıklığında yalnızca *ek* (auxiliary) bellek sayılır: yeni diziler, rekürsiyon yığını, hash tabloları. Sabit sayıda skaler değişken her zaman $\mathcal{O}(1)$ 'dir.

3.2 Big-O Muhakemesi

Soru 7

Bir yarışma probleminde $n \leq 10^6$ veriliyor ve zaman sınırı 1 saniye. $\mathcal{O}(n^2)$ bir çözüm kabul edilir mi? $\mathcal{O}(n \log n)$ kabul edilir mi? Muhakemenizi açıklayın.

Çözüm. Kaba bir kural olarak modern bir işlemci 1 saniyede yaklaşık 10^8 ile 10^9 temel işlem yapar. $n = 10^6$ için:

- $\mathcal{O}(n^2) = (10^6)^2 = 10^{12}$ işlem. Bu 10^8 – 10^9 bütçesinin çok üstündedir; saniyeler-dakikalar sürer,

Dikkat

kesinlikle zaman aşımına uğrar.

- $\mathcal{O}(n \log n) \approx 10^6 \times 20 = 2 \times 10^7$ işlem. Bu bütçenin rahatça altındadır; *kabul edilir*.

Dolayısıyla $n = 10^6$ ölçeğinde $\mathcal{O}(n^2)$ çözümler pratik değildir; $\mathcal{O}(n \log n)$ veya $\mathcal{O}(n)$ hedeflenmelidir.

```
1 // n = 10^6 için işlem tahminleri:
2 //   O(n)           ~ 1e6           -> çok hızlı
3 //   O(n log n)    ~ 2e7           -> uygun (< ~1 sn)
4 //   O(n^2)        ~ 1e12          -> ÇOK yavaş (zaman aşımı)
5 // Kural: 1 sn'de ~1e8-1e9 basit işlem varsayılır.
```

Ölçeğe göre hedef karmaşıklık

$n \leq 10^6-10^7$ için $\mathcal{O}(n)$ veya $\mathcal{O}(n \log n)$; $n \leq 10^5$ için $\mathcal{O}(n \log n)$; $n \leq 10^3-10^4$ için $\mathcal{O}(n^2)$ genellikle uygundur; $n \leq 20-25$ için $\mathcal{O}(2^n)$ gibi üstel çözümler bile kabul edilebilir.

Soru 8

Aşağıdaki üç problem boyutundan her biri için uygun olabilecek *en yavaş* (ama yeterince hızlı) karmaşıklık sınıfını söyleyin ve neden onu seçtiğinizi gerekçelendirin: (a) $n \leq 20$, (b) $n \leq 10^3$, (c) $n \leq 10^6$.

Sınır Uygun en yavaş karmaşıklık ve gerekçe

$n \leq 20$ $\mathcal{O}(2^n)$ veya $\mathcal{O}(n!)$ kabul edilebilir. $2^{20} \approx 10^6$ olduğundan tüm alt kümeleri/permütasyonları denemek (brute force, bitmask) mümkündür.

Çözüm. $n \leq 10^3$ $\mathcal{O}(n^2)$ uygundur: $(10^3)^2 = 10^6$ işlem. İç içe iki döngü rahatça çalışır; hatta $n^2 \log n$ da sınırdan geçebilir.

$n \leq 10^6$ $\mathcal{O}(n)$ veya $\mathcal{O}(n \log n)$ gerekir: 10^6 veya $\sim 2 \times 10^7$ işlem. $\mathcal{O}(n^2) = 10^{12}$ olur ve zaman aşımına uğrar.

```
1 // n <= 20      : O(2^n) ~ 1e6 (bitmask brute force uygun)
2 // n <= 1000    : O(n^2) ~ 1e6 (ic ice donguler uygun)
3 // n <= 1e6     : O(n) / O(n log n) (n^2 = 1e12 -> yasak)
```

Bilgi

n 'in üst sınırı, hedeflenmesi gereken karmaşıklık için güçlü bir ipucudur. Küçük n (örneğin ≤ 25) çoğu zaman üstel bir çözümün beklendiğine işaret eder.

Soru 9

Bir öğrenci " $\mathcal{O}(n \log n)$ ile $\mathcal{O}(n^2)$ arasındaki fark sadece teorik, pratikte önemsiz" diyor. $n = 10^6$ için bu iki karmaşıklığın işlem sayısını karşılaştırarak bu iddiayı değerlendirin.

Çözüm. $n = 10^6$ için:

- $\mathcal{O}(n \log n) \approx 10^6 \times \log_2(10^6) \approx 10^6 \times 20 = 2 \times 10^7$.
- $\mathcal{O}(n^2) = (10^6)^2 = 10^{12}$.

Oran $10^{12}/(2 \times 10^7) = 5 \times 10^4$; yani $\mathcal{O}(n^2)$ çözüm yaklaşık 50.000 kat daha fazla işlem yapar. $\mathcal{O}(n \log n)$ çözüm milisaniyeler sürerken $\mathcal{O}(n^2)$ çözüm on dakikalar sürebilir. İddia yanlıştır: girdi büyüdükçe iki

sınıf arasındaki fark astronomik boyutlara ulaşır. Fark, sabit katsayıların gizlediği küçük n değerlerinde önemsiz görünse de büyük n 'de belirleyicidir.

```

1  #include <cmath>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      double n = 1e6;
7      double nlogn = n * log2(n);    // ~2e7
8      double nsq   = n * n;          // 1e12
9      cout << "n log n = " << nlogn << "\n";
10     cout << "n^2     = " << nsq   << "\n";
11     cout << "oran     = " << nsq / nlogn << "\n"; // ~5e4
12     return 0;
13 }
```

Dikkat

Big-O sabit katsayıları gizler; bu yüzden küçük n 'de $\mathcal{O}(n^2)$ bir algoritma $\mathcal{O}(n \log n)$ 'den hızlı olabilir. Fark, girdi büyüdükçe kesinleşir; asimptotik davranış büyük ölçekte kazanır.

3.3 STL ile Problem Çözme

Soru 10

Bir struct Student { string name; int score; } vektörünü *puana göre azalan*, eşit puanlarda *isme göre artan* sıralayın. Bunu bir lambda karşılaştırıcı ile yapın ve karmaşıklığı belirtin.

Çözüm. std::sort karşılaştırmalı sıralama yapar ve $\mathcal{O}(n \log n)$ zaman karmaşıklığına sahiptir. Özel sıralama ölçütü için üçüncü argüman olarak bir lambda comparator veririz. Comparator, "birinci eleman ikinciden önce gelmeli mi?" sorusuna true/false döndürmelidir. Azalan puan için `a.score > b.score`, eşitlikte artan isim için `a.name < b.name` kullanırız.

```

1  #include <algorithm>
2  #include <string>
3  #include <vector>
4  #include <iostream>
5  using namespace std;
6
7  struct Student { string name; int score; };
8
9  int main() {
10     vector<Student> students = {
11         {"Ali", 90}, {"Zeynep", 90}, {"Mehmet", 75}
12     };
13     // O(n log n): önce puana göre azalan, eşitlikte isme göre artan
14     sort(students.begin(), students.end(),
15         [](const Student& a, const Student& b) {
16             if (a.score != b.score)
17                 return a.score > b.score; // azalan puan
18             return a.name < b.name;       // artan isim
19         });
20     for (const auto& s : students)
21         cout << s.name << " " << s.score << "\n";
22     return 0;
23 }
```

Bilgi

Comparator "kesin küçüktür" (strict weak ordering) tanımlamalıdır: eşit elemanlar için false döndürmelidir. $\leq$ kullanmak tanımsız davranışa yol açar.

Soru 11

Sıralı bir tam sayı dizisinde, belirli bir $[L, R]$ değer aralığına düşen eleman sayısını `lower_bound` ve `upper_bound` kullanarak $\mathcal{O}(\log n)$ zamanda bulun.

Çözüm. Sıralı bir dizide `lower_bound(L)`, değeri $\geq L$ olan ilk elemana; `upper_bound(R)`, değeri $> R$ olan ilk elemana işaret eder. İki iteratör arasındaki mesafe (`upper - lower`), $[L, R]$ aralığındaki eleman sayısını verir. Her iki fonksiyon da ikili arama yaptığı için $\mathcal{O}(\log n)$ çalışır; dizinin sıralı olması şarttır.

```
1 #include <algorithm>
2 #include <vector>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     vector<int> v = {1, 3, 3, 5, 8, 8, 8, 10}; // SIRALI olmalı
8     int L = 3, R = 8;
9     // ilk >= L ve ilk > R konumları (her biri  $\mathcal{O}(\log n)$ )
10    auto lo = lower_bound(v.begin(), v.end(), L);
11    auto hi = upper_bound(v.begin(), v.end(), R);
12    long long countInRange = hi - lo; // [L, R] içindeki eleman sayısı
13    cout << countInRange << "\n"; // 6 (3,3,5,8,8,8)
14    return 0;
15 }
```

İpucu

`lower_bound` $\geq$ değeri, `upper_bound` $>$ değeri arar. Bir değer dizideki kopya sayısı `upper_bound(x) - lower_bound(x)` ile $\mathcal{O}(\log n)$ bulunur.

Soru 12

Bir metindeki (kelime vektörü) her kelimenin kaç kez geçtiğini sayın ve en çok geçen kelimeyi bulun. `unordered_map` kullanarak ortalama $\mathcal{O}(n)$ bir çözüm yazın.

Çözüm. Frekans sayımı için hash tablosu olan `unordered_map<string,int>` idealdir: ekleme ve erişim ortalama $\mathcal{O}(1)$ 'dir, dolayısıyla n kelimeyi saymak ortalama $\mathcal{O}(n)$ olur. `freq[word]++` ifadesi kelime yoksa 0'dan başlatıp artırır (operator[] varsayılan değer ekler). Sonra map üzerinde tek geçişle ($\mathcal{O}(k)$, k = tekil kelime sayısı) en yüksek frekanslıyı buluruz. Not: `unordered_map` sırasız ve ortalama $\mathcal{O}(1)$; sıralı gezinme gerekiyorsa map ($\mathcal{O}(\log n)$ işlem, kırmızı-siyah ağaç) tercih edilir.

```
1 #include <unordered_map>
2 #include <string>
3 #include <vector>
4 #include <iostream>
5 using namespace std;
6
7 int main() {
8     vector<string> words = {"elma", "armut", "elma", "kiraz", "elma", "armut"};
9     unordered_map<string,int> freq; // ortalama  $\mathcal{O}(1)$  ekleme/erisim
10    for (const string& w : words)
```

```

11     freq[w]++; // yoksa 0'dan baslar -> toplam O(n)
12
13     string best; int bestCount = 0;
14     for (const auto& [word, cnt] : freq) // O(k), k = tekil sayisi
15         if (cnt > bestCount) { bestCount = cnt; best = word; }
16
17     cout << best << " -> " << bestCount << "\n"; // elma -> 3
18     return 0;
19 }

```

Dikkat

`unordered_map` ortalama $\mathcal{O}(1)$, ancak kötü hash çakışmalarında en kötü durum $\mathcal{O}(n)$ olabilir. Deterministik $\mathcal{O}(\log n)$ garanti veya sıralı iterasyon gerekiyorsa `std::map` kullanın.

Soru 13

Bir tam sayı vektöründeki *benzersiz* (tekil) elemanları bulup küçükten büyüğe sıralı yazdırın. Bunu `std::set` ile yapın ve karmaşıklığı belirtin.

Çözüm. `std::set` elemanları hem tekilleştirir hem de sıralı tutar (dengeli ikili arama ağacı). n eleman eklemek, ekleme başına $\mathcal{O}(\log n)$ olduğundan toplam $\mathcal{O}(n \log n)$ maliyetlidir. Set üzerinde gezinmek elemanları otomatik olarak artan sırada verir; ek bir sıralamaya gerek kalmaz. Yalnızca tekilleştirme yeterliyse ve sıra gerekmiyorsa `unordered_set` ortalama $\mathcal{O}(n)$ ile daha hızlıdır.

```

1  #include <set>
2  #include <vector>
3  #include <iostream>
4  using namespace std;
5
6  int main() {
7      vector<int> v = {5, 3, 5, 1, 3, 8, 1};
8      set<int> unique(v.begin(), v.end()); // tekilleştir + sırala: O(n log n)
9      for (int x : unique)                // artan sırada gezinir
10         cout << x << " ";                // 1 3 5 8
11     cout << "\n";
12     return 0;
13 }

```

İpucu

Sıra önemliyse set ($\mathcal{O}(n \log n)$), önemsizse `unordered_set` (ortalama $\mathcal{O}(n)$). Vektörde tekilleştirmenin bir başka yolu: `sort + unique + erase` deyimi (yine $\mathcal{O}(n \log n)$).

Soru 14

Bir sayı akışında (vektör) *en büyük k* elemanı verimli biçimde bulun. Tüm diziyi sıralamak yerine `std::priority_queue` tabanlı $\mathcal{O}(n \log k)$ bir çözüm yazın.

Çözüm. Tüm diziyi sıralamak $\mathcal{O}(n \log n)$ 'dir. Yalnızca en büyük k eleman gerekiyorsa, boyutu k ile sınırlı bir *min-heap* (en küçüğü tepede tutan öncelik kuyruğu) kullanırız. Her elemanı ekleriz; heap boyutu k 'yı aşınca tepedeki en küçüğü çıkarırız. Böylece heapte her zaman o ana kadarki en büyük k eleman kalır. Her işlem $\mathcal{O}(\log k)$, toplam n eleman için $\mathcal{O}(n \log k)$ 'dir; $k \ll n$ olduğunda tam sıralamadan hızlıdır ve daha az bellek kullanır ($\mathcal{O}(k)$).

```

1  #include <queue>

```

```

2  #include <vector>
3  #include <iostream>
4  using namespace std;
5
6  int main() {
7      vector<int> v = {4, 1, 7, 3, 9, 2, 8};
8      int k = 3;
9      // min-heap: en küçük tepede; boyutu k ile sınırlı tutulur
10     priority_queue<int, vector<int>, greater<int>> minHeap;
11     for (int x : v) {
12         minHeap.push(x);           // O(log k)
13         if ((int)minHeap.size() > k)
14             minHeap.pop();         // en küçüğü at -> en büyük k kalır
15     }
16     // Toplam: O(n log k), bellek: O(k)
17     while (!minHeap.empty()) {     // en büyük k eleman (artan sırada)
18         cout << minHeap.top() << " "; // 7 8 9
19         minHeap.pop();
20     }
21     cout << "\n";
22     return 0;
23 }

```

Bilgi

En büyük k için *min-heap*, en küçük k için *max-heap* kullanılır (heaptan hangi ucun atılacağı ters çevrilir). `priority_queue` varsayılan olarak *max-heap*'tir; `greater<int>` ile *min-heap*'e dönüşür.

Soru 15

Bir tam sayı vektörünün elemanlarının toplamını ve en büyük elemanını, döngü yazmadan STL algoritmaları `std::accumulate` ve `std::max_element` ile hesaplayın. Karmaşıklıklarını belirtin.

Çözüm. `std::accumulate` (başlık `<numeric>`) bir aralığın elemanlarını verilen başlangıç değeriyle toplar; tek geçişte çalıştığı için $O(n)$ 'dir. Başlangıç değerinin türü sonuç türünü belirler: taşmayı önlemek için `OLL` (long long) vermek iyi bir alışkanlıktır. `std::max_element` (başlık `<algorithm>`) en büyük elemana bir iteratör döndürür, o da tek geçiş $O(n)$ 'dir; değere `*it` ile erişilir. Her iki algoritma da açık indeksli döngüden hem daha okunaklı hem hataya daha az açıktır.

```

1  #include <numeric>           // accumulate
2  #include <algorithm>        // max_element
3  #include <vector>
4  #include <iostream>
5  using namespace std;
6
7  int main() {
8      vector<int> v = {4, 1, 7, 3, 9, 2};
9      // Toplam: O(n). Başlangıç OLL -> long long, taşmayı önler.
10     long long sum = accumulate(v.begin(), v.end(), OLL);
11     // En büyük elemana iteratör: O(n)
12     auto it = max_element(v.begin(), v.end());
13     cout << "toplam = " << sum << "\n"; // 26
14     cout << "maks   = " << *it << "\n"; // 9
15     return 0;
16 }

```

İpucu

`accumulate` dördüncü argüman olarak özel bir ikili işlem alabilir (örn. çarpım için `multiplies<>()`). `max_element` de bir comparator lambda ile özel ölçütte maksimum bulabilir. İkisi de $\mathcal{O}(n)$ 'dir.

KISIM 2

Temel Veri Yapıları

4. Diziler ve Dinamik Diziler

Dizi (array), aynı tipteki elemanların bellekte *ardışık* olarak tutulduğu en temel veri yapısıdır. Ardışıklık sayesinde herhangi bir elemana indeksle $O(1)$ 'de erişilir: $\text{eleman adresi} = \text{taban} + \text{indeks} \times \text{eleman_boyutu}$. Bu basit formül, dizilerin gücünün ve sınırlarının kaynağıdır.

4.1 Statik Diziler ve Bellek Düzeni

```
1 int a[5] = {10, 20, 30, 40, 50};
2 // Bellek: [10][20][30][40][50]
3 // indeks: 0 1 2 3 4
4 // a[3] = *(a + 3) = *(taban + 3*sizeof(int)) -> O(1) erişim
5
6 std::array<int, 5> b = {1, 2, 3, 4, 5}; // C++ tarzi, boyut tip bilgisinde
7 cout << b.size() << "\n";           // 5
```

4.2 Dinamik Dizi Nasıl Çalışır?

Statik dizinin boyutu derleme zamanında sabittir. `std::vector` ise büyüyeabilen bir dizidir. İçeride bir bloğu ayırır; dolduğunda *daha büyük* (genelde iki kat) yeni bir blok ayırır, eski elemanları kopyalar ve eskiyi serbest bırakır. Bunu kendimiz yazarak mekanizmayı görelim:

```
1 class DynamicArray {
2     int* data;
3     int capacity;
4     int size;
5
6     void grow() { // kapasiteyi iki katına çıkar
7         capacity = (capacity == 0) ? 1 : capacity * 2;
8         int* newData = new int[capacity];
9         for (int i = 0; i < size; ++i)
10             newData[i] = data[i]; // O(n) kopyalama
11         delete[] data;
12         data = newData;
13     }
14 public:
15     DynamicArray() : data(nullptr), capacity(0), size(0) {}
16     ~DynamicArray() { delete[] data; }
17
18     void add(int x) { // amortize O(1)
```

```

19     if (size == capacity) grow();
20     data[size++] = x;
21 }
22 int get(int i) const { return data[i]; } // O(1)
23 int getSize() const { return size; }
24 };

```

Neden iki kat?

Kapasiteyi her seferinde *sabit miktarda* (örneğin +1) artırırsaydık, n ekleme $1 + 2 + \dots + n = O(n^2)$ kopyalamaya mal olurdu. İki katına çıkarınca toplam kopyalama $1 + 2 + 4 + \dots + n < 2n$ olur; yani n ekleme $O(n)$, ekleme başına *amortize* $O(1)$.

4.3 Dizi İşlemlerinin Karmaşıklığı

İşlem	Karmaşıklık	Açıklama
İndeksle erişim	$O(1)$	Adres hesabı doğrudan.
Sona ekleme	$O(1)^*$	Amortize; yer varsa anında.
Baş/ortaya ekleme	$O(n)$	Sonraki elemanlar kaydırılır.
Silme (orta)	$O(n)$	Boşluk kaydırılarak doldurulur.
Arama (sırasız)	$O(n)$	Tek tek bakılır.
Arama (sıralı)	$O(\log n)$	İkili arama.

4.4 Sık Kullanılan Dizi Teknikleri

İki İşaretçi (Two Pointers)

Sıralı bir dizide, iki uçtan ortaya doğru ilerleyen işaretçilerle birçok problem $O(n)$ 'de çözülür. Örnek: sıralı dizide toplamı target olan iki eleman var mı?

```

1 bool hasTwoSum(const vector<int>& a, int target) {
2     int left = 0, right = a.size() - 1;
3     while (left < right) {
4         int sum = a[left] + a[right];
5         if (sum == target) return true;
6         if (sum < target) ++left; // daha büyük toplam gerek
7         else --right; // daha küçük toplam gerek
8     }
9     return false;
10 }

```

Kayan Pencere (Sliding Window)

Ardışık bir alt dizinin özelliğini (toplam, en büyük vb.) ararken, pencereyi kaydırarak her adımda baştan hesaplamak yerine güncelleriz. Örnek: boyutu k olan en büyük pencere toplamı.

```

1 int maxWindowSum(const vector<int>& a, int k) {
2     int window = 0;
3     for (int i = 0; i < k; ++i) window += a[i]; // ilk pencere
4     int maxSum = window;
5     for (int i = k; i < (int)a.size(); ++i) {

```

```

6     window += a[i] - a[i - k];    // sag ekle, sol cikar -> O(1)
7     maxSum = max(maxSum, window);
8 }
9 return maxSum;                    // toplam O(n)
10 }

```

Önek Toplamı (Prefix Sum)

Bir dizide birçok kez “[l, r] aralığının toplamı nedir?” sorusu soruluyorsa, önce önek toplamı tablosu kurarız; sonra her sorgu $O(1)$ ’dir.

```

1 vector<long long> prefix(a.size() + 1, 0);
2 for (int i = 0; i < (int)a.size(); ++i)
3     prefix[i + 1] = prefix[i] + a[i];
4 // [l, r] araligi toplami (dahil):
5 long long rangeSum(int l, int r) { return prefix[r + 1] - prefix[l]; }

```

5. Bağlı Listeler

Bağlı liste (linked list), her elemanı bir *düğüm* (node) olarak tutan ve düğümleri işaretçilerle birbirine bağlayan bir yapıdır. Diziden farkı, elemanların bellekte ardışık *olmamasıdır*; her düğüm bir sonrakinin adresini saklar. Bu, ekleme/silmeyi ucuzlatır ama indeksle erişimi kaybettirir.

Özellik	Dizi	Bağlı Liste
İndeksle erişim	$O(1)$	$O(n)$
Başa ekleme	$O(n)$	$O(1)$
Bellek düzeni	Ardışık	Dağınık + işaretçi maliyeti
Önbellek dostu mu?	Evet	Hayır

5.1 Tek Yönlü Bağlı Liste

Her düğüm bir değer ve *sadece* bir sonraki düğüme işaretçi tutar. Son düğümün next’i nullptr’dır.

```

1 struct Node {
2     int data;
3     Node* next;
4     Node(int v) : data(v), next(nullptr) {}
5 };
6
7 class LinkedList {
8     Node* head = nullptr;
9 public:
10    // Basa ekle -> O(1)
11    void pushFront(int v) {
12        Node* d = new Node(v);
13        d->next = head;
14        head = d;
15    }
16    // Sona ekle -> O(n) (son dugumu bulmak gerekir)
17    void pushBack(int v) {
18        Node* d = new Node(v);

```

```

19     if (!head) { head = d; return; }
20     Node* p = head;
21     while (p->next) p = p->next;
22     p->next = d;
23 }
24 // Bir degeri sil -> O(n)
25 void remove(int v) {
26     Node** p = &head; // isaretciye isaretci: sik idiom
27     while (*p && (*p)->data != v) p = &(*p)->next;
28     if (*p) { Node* toDelete = *p; *p = (*p)->next; delete toDelete; }
29 }
30 void print() const {
31     for (Node* p = head; p; p = p->next) cout << p->data << " -> ";
32     cout << "nullptr\n";
33 }
34 ~LinkedList() {
35     while (head) { Node* s = head; head = head->next; delete s; }
36 }
37 };

```

5.2 Listeyi Ters Çevirme (Klasik Mülakat Sorusu)

Üç işaretçiyle listeyi tek geçişte, $\mathcal{O}(n)$ zaman ve $\mathcal{O}(1)$ ek bellekte ters çeviririz. Her adımda mevcut düğümün başını geriye döndürürüz.

```

1 Node* reverse(Node* head) {
2     Node* prev = nullptr;
3     Node* curr = head;
4     while (curr) {
5         Node* next = curr->next; // ilerisini sakla
6         curr->next = prev;       // bagı geriye cevır
7         prev = curr;            // ikisini de bir adım ilerlet
8         curr = next;
9     }
10    return prev; // yeni bas
11 }

```

5.3 Çift Yönlü Bağlı Liste ve Döngü Tespiti

Çift yönlü listede her düğüm hem önceki hem sonraki düğümü tutar; bu, geriye gitmeyi ve bir düğümü $\mathcal{O}(1)$ 'de silmeyi mümkün kılar. Bir listede döngü olup olmadığını, **Floyd'un tavşan-kaplumbağa** algoritmasıyla (biri iki, biri tek adım atan iki işaretçi) $\mathcal{O}(n)$ zaman, $\mathcal{O}(1)$ bellekte buluruz.

```

1 bool hasCycle(Node* head) {
2     Node* slow = head;
3     Node* fast = head;
4     while (fast && fast->next) {
5         slow = slow->next; // 1 adım
6         fast = fast->next->next; // 2 adım
7         if (slow == fast) return true; // karsilastilar -> dongu var
8     }
9     return false;
10 }

```

Dikkat

Ham işaretçilerle (`new/delete`) bağlı liste yazmak eğitseldir ama gerçek kodda bellek sızıntısı ve çift-serbestleme (`double-free`) hatalarına açıktır. Üretimde `std::list` / `std::forward_list` veya `std::unique_ptr` tabanlı düğümler tercih edilir.

6. Yığın (Stack)

Yığın, **LIFO** (Last In, First Out — son giren ilk çıkar) ilkesiyle çalışan bir yapıdır. Sadece bir uçtan (tepe) ekleme ve çıkarma yapılır; tıpkı üst üste dizilmiş tabaklar gibi. Fonksiyon çağrı yığını, geri-al (`undo`) işlemleri, ifade değerlendirme ve derinlik-öncelikli arama yığınıyla çalışır.

6.1 Diziyle Yığın Uygulaması

```

1  template <typename T>
2  class Stack {
3      vector<T> v;
4  public:
5      void  push(const T& x) { v.push_back(x); } // O(1)
6      void  pop()           { v.pop_back();   } // O(1)
7      T&    top()           { return v.back(); } // O(1)
8      bool  empty() const   { return v.empty(); }
9      size_t size() const   { return v.size(); }
10 };
11
12 // STL karsiligi:
13 #include <stack>
14 stack<int> s;
15 s.push(1); s.push(2);
16 cout << s.top(); // 2
17 s.pop();

```

6.2 Uygulama: Parantez Dengeleme

Bir ifadedeki parantezlerin `(){}[]` doğru eşleşip eşleşmediğini yığınla kontrol ederiz: açılışı yığına at, kapanışta tepedeki eşleşmeli.

```

1  bool isBalanced(const string& s) {
2      stack<char> y;
3      for (char c : s) {
4          if (c == '(' || c == '{' || c == '[') y.push(c);
5          else if (c == ')' || c == '}' || c == ']') {
6              if (y.empty()) return false;
7              char a = y.top(); y.pop();
8              if ((c == ')' && a != '(') ||
9                  (c == '}' && a != '{') ||
10                 (c == ']' && a != '[')) return false;
11          }
12      }
13      return y.empty(); // acik kalan olmamali
14 }

```

6.3 Uygulama: Sonraki Büyük Eleman (Monoton Yığın)

Bir dizide her eleman için “sağındaki ilk daha büyük eleman” sorusunu, azalan bir *monoton yığın* ile toplam $\mathcal{O}(n)$ ’de yanıtlanır.

```

1  vector<int> nextGreater(const vector<int>& a) {
2      int n = a.size();
3      vector<int> result(n, -1);
4      stack<int> y;                // indeks tutar (azalan degerler)
5      for (int i = 0; i < n; ++i) {
6          while (!y.empty() && a[y.top()] < a[i]) {
7              result[y.top()] = a[i];    // a[i], yigindaki daha kucukleri asar
8              y.pop();
9          }
10         y.push(i);
11     }
12     return result;
13 }

```

7. Kuyruk ve Çift Uçlu Kuyruk (Deque)

Kuyruk, **FIFO** (First In, First Out — ilk giren ilk çıkar) ilkesiyle çalışır: bir uçtan (arka) eklenir, diğer uçtan (ön) çıkarılır; market sırası gibi. Genişlik-öncelikli arama, iş zamanlama ve tampon (buffer) yapıları kuyruk kullanır.

7.1 Dairesel Kuyruk (Circular Buffer)

Sabit boyutlu bir diziyi “dairesele” kullanarak, ön ve arka indeksleri modulo ile döndürerek $\mathcal{O}(1)$ ekleme/çıkarma elde ederiz. Böylece eleman kaydırmaya gerek kalmaz.

```

1  class CircularQueue {
2      vector<int> v;
3      int front = 0, rear = 0, count = 0, cap;
4  public:
5      CircularQueue(int k) : v(k), cap(k) {}
6      bool isFull() const { return count == cap; }
7      bool isEmpty() const { return count == 0; }
8
9      bool enqueue(int x) {                // arkaya ekle -> O(1)
10         if (isFull()) return false;
11         v[rear] = x;
12         rear = (rear + 1) % cap;          // dairesele ilerleme
13         ++count;
14         return true;
15     }
16     bool dequeue(int& x) {                // onden cikar -> O(1)
17         if (isEmpty()) return false;
18         x = v[front];
19         front = (front + 1) % cap;
20         --count;
21         return true;
22     }
23 };
24
25 // STL karsiligi:
26 #include <queue>
27 queue<int> q;
28 q.push(1); q.push(2);

```

```

29 cout << q.front();    // 1
30 q.pop();

```

7.2 Deque ve Kayan Pencerede Maksimum

`std::deque` (double-ended queue) her iki uçtan da $\mathcal{O}(1)$ ekleme/çıkarma sunar. Güçlü bir uygulaması, boyutu k olan her pencerenin maksimumunu toplam $\mathcal{O}(n)$ 'de bulan **monoton deque** tekniğidir.

```

1  #include <deque>
2  vector<int> windowMax(const vector<int>& a, int k) {
3      deque<int> dq;                                // azalan degerli indeksler
4      vector<int> result;
5      for (int i = 0; i < (int)a.size(); ++i) {
6          if (!dq.empty() && dq.front() == i - k) dq.pop_front(); // pencereden cikan
7          while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
8          dq.push_back(i);
9          if (i >= k - 1) result.push_back(a[dq.front()]); // on = maksimum
10     }
11     return result;
12 }

```

8. Hash Tabloları

Hash tablosu, anahtar-değer çiftlerini *ortalama* $\mathcal{O}(1)$ zamanda saklayan ve arayan bir yapıdır. Fikir: bir *hash fonksiyonu* anahtarı bir dizi indeksine çevirir; böylece anahtarı “hesaplayarak” doğru-
dan yerine gideriz. C++’ta `std::unordered_map` ve `std::unordered_set` bunu sağlar.

8.1 Nasıl Çalışır: Hash Fonksiyonu ve Çakışma

İki farklı anahtar aynı indekse düşerse buna *çakışma* (collision) denir. En yaygın çözüm **zincirleme** (chaining): her kova (bucket) bir bağlı liste tutar, çakışanlar aynı listeye eklenir. İyi bir hash fonksiyonuyla çakışma az olur ve işlemler $\mathcal{O}(1)$ kalır.

```

1  class HashSet {
2      static const int BUCKETS = 1009;    // asal sayi: dagilim daha iyi
3      vector<list<int>> table;
4      int hashKey(int key) const { return ((key % BUCKETS) + BUCKETS) % BUCKETS; }
5  public:
6      HashSet() : table(BUCKETS) {}
7      void add(int x) {
8          auto& bucket = table[hashKey(x)];
9          for (int v : bucket) if (v == x) return; // zaten var
10         bucket.push_back(x);
11     }
12     bool contains(int x) const {
13         const auto& bucket = table[hashKey(x)];
14         for (int v : bucket) if (v == x) return true;
15         return false;
16     }
17     void remove(int x) { table[hashKey(x)].remove(x); }
18 };

```

8.2 STL Hash Konteynerleri

```

1  #include <unordered_map>

```

```

2 #include <unordered_set>
3
4 unordered_map<string, int> counter;
5 counter["elma"]++;           // yoksa 0'dan baslar, sonra 1
6 counter["elma"]++;           // 2
7 cout << counter["elma"];     // 2
8
9 unordered_set<int> seen;
10 seen.insert(5);
11 if (seen.count(5)) cout << "var\n";

```

8.3 Uygulama: İki Sayının Toplamı (Two Sum)

Sırasız bir dizide toplamı target olan iki elemanın indekslerini, hash haritasıyla tek geçişte $\mathcal{O}(n)$ 'de buluruz.

```

1 vector<int> twoSum(const vector<int>& a, int target) {
2     unordered_map<int, int> seen;           // deger -> indeks
3     for (int i = 0; i < (int)a.size(); ++i) {
4         int needed = target - a[i];
5         auto it = seen.find(needed);
6         if (it != seen.end()) return {it->second, i};
7         seen[a[i]] = i;
8     }
9     return {};
10 }

```

Bilgi

map vs unordered_map: map kırmızı-siyah ağaçtır, anahtarları *sıralı* tutar ve tüm işlemler $\mathcal{O}(\log n)$ 'dir. unordered_map hash tablosudur, sırasızdır ve *ortalama* $\mathcal{O}(1)$ 'dir ama en kötü durumda (kötü hash / saldırı) $\mathcal{O}(n)$ 'e düşebilir. Sıraya ihtiyacın yoksa ve hız önemliyse unordered_map seç.

9. Temel Veri Yapıları — Alıştırmalar ve Çözümler

Bilgi

Bu bölümde dizilerden hash tablolarına kadar temel veri yapılarını, klasik algoritmik desenler (two-pointer, sliding window, prefix sum, monoton yığın/deque) üzerinden çalışacağız. Her sorunun altında Türkçe açıklama, tam çalışan C++ çözümü ve zaman/bellek karmaşıklığı verilmiştir. Kod içinde tanımlar İngilizce, yorumlar Türkçedir.

9.1 Diziler

Soru 16

İki Toplam — Sıralı Dizi (Two-Pointer). Artan sırada verilmiş bir nums dizisi ve bir target değeri verildiğinde, toplamları hedefe eşit olan iki elemanın (1-tabanlı) indekslerini bulun. Tam olarak bir çözüm olduğunu varsayın.

Çözüm.

Dizi sıralı olduğu için tüm ikilileri denemeye gerek yoktur. Bir işaretçiyi başa (left), diğerini sona (right) koyarız. Toplam hedeften küçükse left sağa kayar (toplamı büyütmek için), büyükse right

sola kayar. Bu sayede her elemana en fazla bir kez bakarız.

```

1  #include <vector>
2  using namespace std;
3
4  vector<int> twoSumSorted(const vector<int>& nums, int target) {
5      int left = 0, right = (int)nums.size() - 1;
6      while (left < right) {
7          int sum = nums[left] + nums[right];
8          if (sum == target)
9              return {left + 1, right + 1}; // 1-tabanlı indeksler
10         else if (sum < target)
11             ++left; // toplama buyutmek için soldan ilerle
12         else
13             --right; // toplama kucultmek için sagdan geri gel
14     }
15     return {}; // bulunamadi (varsayima gore olmaz)
16 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$. Sıralı olmayan dizide bu yöntem çalışmaz; o durumda hash tablosu (bkz. Two Sum) kullanılır.

Soru 17

Sıralı Dizinin Karelerini Sıralı Üret. Artan sırada (negatifler dahil) verilmiş nums dizisinin her elemanının karesini içeren, yine artan sıralı bir dizi döndürün. $\mathcal{O}(n)$ olmalı.

Çözüm.

Negatif sayıların karesi büyük olabileceğinden en büyük kareler dizinin iki ucundadır. İki işaretçiyi uçlara koyup mutlak değeri büyük olanın karesini sonuç dizisinin sonundan başlayarak yerleştiririz.

```

1  #include <vector>
2  using namespace std;
3
4  vector<int> sortedSquares(const vector<int>& nums) {
5      int n = (int)nums.size();
6      vector<int> result(n);
7      int left = 0, right = n - 1;
8      for (int pos = n - 1; pos >= 0; --pos) {
9          int leftSq = nums[left] * nums[left];
10         int rightSq = nums[right] * nums[right];
11         if (leftSq > rightSq) { // sol uc daha büyük kareye sahip
12             result[pos] = leftSq;
13             ++left;
14         } else {
15             result[pos] = rightSq;
16             --right;
17         }
18     }
19     return result;
20 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(n)$ (sonuç dizisi). Önce kareleyip sonra sort çağırmak $\mathcal{O}(n \log n)$ olurdu; two-pointer bunu doğrusala indirir.

Soru 18

Boyutu k Olan Pencerede Maksimum Toplam (Sliding Window). nums dizisinde, ardışık k elemanlık pencereler arasındaki en büyük toplamı bulun.

Çözüm.

İlk k elemanın toplamını hesaplarız. Sonra pencereyi bir sağa kaydırırken çıkan elemanı çıkarıp giren elemanı ekleriz; her pencereyi baştan toplamayız. Bu sabit maliyetli kaydırma sayesinde toplam iş doğrusaldır.

```

1  #include <vector>
2  #include <algorithm>
3  #include <climits>
4  using namespace std;
5
6  int maxSumWindow(const vector<int>& nums, int k) {
7      int n = (int)nums.size();
8      if (n < k) return 0; // gecersiz durum
9      int windowSum = 0;
10     for (int i = 0; i < k; ++i) windowSum += nums[i]; // ilk pencere
11     int best = windowSum;
12     for (int i = k; i < n; ++i) {
13         windowSum += nums[i] - nums[i - k]; // gireni ekle, cikani cikar
14         best = max(best, windowSum);
15     }
16     return best;
17 }
```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$.

Soru 19

En Uzun Tekrarsız Alt Dizi/Alt Karakter Dizisi (Değişken Pencere). Bir string içinde tekrar eden karakter içermeyen en uzun bitişik alt dizinin uzunluğunu bulun.

Çözüm.

Sağ kenarı (right) ilerletirken bir karakter ikinci kez görülürse, sol kenarı (left) o karakterin önceki konumundan sonraya çekeriz. Her karakterin son görüldüğü indeksi bir tabloda tutarız. Pencere her zaman tekrarsız kalır.

```

1  #include <string>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int longestUniqueSubstring(const string& s) {
7      vector<int> lastSeen(256, -1); // her karakterin son indeksi
8      int left = 0, best = 0;
9      for (int right = 0; right < (int)s.size(); ++right) {
10         unsigned char c = s[right];
11         if (lastSeen[c] >= left)
12             left = lastSeen[c] + 1; // tekrari disarida birak
13         lastSeen[c] = right;
14         best = max(best, right - left + 1);
15     }
16     return best;
17 }
```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$ (sabit alfabe). Her kenar en fazla n kez ilerlediğinden iç içe döngü yoktur.

Soru 20

Prefix Sum ile Aralık Toplamı Sorguları ve Hedefe Eşit Alt Dizi Sayısı. (a) Çok sayıda $[i, j]$ aralık toplamı sorgusunu her biri $\mathcal{O}(1)$ olacak şekilde yanıtlayın. (b) Toplamı tam target olan bitişik alt dizilerin sayısını bulun.

Çözüm.

Önek toplamı $\text{prefix}[i]$, ilk i elemanın toplamıdır. Bir aralığın toplamı $\text{prefix}[j+1] - \text{prefix}[i]$ ile sabit zamanda bulunur. Alt dizi sayımı için ise şu gözlemi kullanırız: bir alt dizinin toplamı target ise, o anki önek toplamından target çıkardığımız değer daha önce kaç kez görülmüşse o kadar geçerli alt dizi biter. Görülen önek toplamlarını bir hash tablosunda sayarız.

```

1  #include <vector>
2  #include <unordered_map>
3  using namespace std;
4
5  // (a) Ön işleme: aralık toplamı sorguları
6  struct RangeSum {
7      vector<long long> prefix;
8      RangeSum(const vector<int>& nums) {
9          prefix.assign(nums.size() + 1, 0);
10         for (size_t i = 0; i < nums.size(); ++i)
11             prefix[i + 1] = prefix[i] + nums[i];
12     }
13     long long query(int i, int j) const { // kapsayıcı [i, j]
14         return prefix[j + 1] - prefix[i];
15     }
16 };
17
18 // (b) Toplamı target olan alt dizi sayısı
19 int countSubarraysWithSum(const vector<int>& nums, int target) {
20     unordered_map<long long, int> seen;
21     seen[0] = 1; // bos onek: toplam 0 bir kez gorulmus sayilir
22     long long running = 0;
23     int count = 0;
24     for (int x : nums) {
25         running += x;
26         auto it = seen.find(running - target);
27         if (it != seen.end()) count += it->second;
28         ++seen[running];
29     }
30     return count;
31 }

```

Karmaşıklık: (a) Ön işleme $\mathcal{O}(n)$, her sorgu $\mathcal{O}(1)$. (b) Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(n)$.

9.2 Bağlı Listeler**Bilgi**

Aşağıdaki çözümlerde tekil bağlı liste düğümü için standart bir tanım kullanılır:

```

1  struct ListNode {
2      int val;
3      ListNode* next;
4      ListNode(int v) : val(v), next(nullptr) {}
5  };

```

Soru 21

Ortadaki Düğümü Bul (Slow/Fast). Bir bağlı listenin ortasındaki düğümü tek geçişte bulun. Çift uzunlukta iki ortadan ikincisini döndürün.

Çözüm.

Klasik "tavşan ve kaplumbağa" tekniği: slow birer, fast ikişer adım ilerler. fast sona ulaştığında slow tam ortadadır. Böylece uzunluğu önceden saymadan tek geçişte ortayı buluruz.

```

1 ListNode* middleNode(ListNode* head) {
2     ListNode* slow = head;
3     ListNode* fast = head;
4     while (fast && fast->next) {
5         slow = slow->next;           // birer adım
6         fast = fast->next->next;      // ikiser adım
7     }
8     return slow; // çift uzunlukta ikinci ortayı verir
9 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$.

Soru 22

Bağlı Listeyi Ters Çevir. Bir tekil bağlı listeyi yerinde (in-place) ters çevirip yeni başı döndürün.

Çözüm.

Üç işaretçi ile ilerleriz: prev, curr, ve geçici nextNode. Her adımda mevcut düğümün next bağı bir önceki düğüme çeviririz. Döngü bittiğinde prev yeni baştır.

```

1 ListNode* reverseList(ListNode* head) {
2     ListNode* prev = nullptr;
3     ListNode* curr = head;
4     while (curr) {
5         ListNode* nextNode = curr->next; // bağlantıyı kaybetmemek için sakla
6         curr->next = prev;                // yönü ters çevir
7         prev = curr;                     // prev'i ilerlet
8         curr = nextNode;                  // curr'i ilerlet
9     }
10    return prev; // yeni baş
11 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$.

İpucu

Ters çevirme deseni birçok liste sorusunun yapı taşıdır: palindrom kontrolü, k'lı grup ters çevirme ve listenin ikinci yarısını çevirip birleştirme gibi problemlerde tekrar tekrar karşımıza çıkar.

Soru 23

İki Sıralı Listeyi Birleştir. Artan sıralı iki bağlı listeyi tek bir sıralı listede birleştirin.

Çözüm.

Bir "kukla" (dummy) baş düğümü kullanmak, sonuç listesinin başını özel durum olarak ele almak-tan kurtarır. tail işaretçisi ile her adımda iki listenin küçük başlı olanını sonuca ekleriz.

```

1 ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
2     ListNode dummy(0); // kukla baş: kod sadelesir
3     ListNode* tail = &dummy;
4     while (a && b) {

```

```

5     if (a->val <= b->val) { tail->next = a; a = a->next; }
6     else { tail->next = b; b = b->next; }
7     tail = tail->next;
8 }
9 tail->next = a ? a : b; // kalan kuyruğu ekle
10 return dummy.next;
11 }

```

Karmaşıklık: Zaman $\mathcal{O}(n + m)$, bellek $\mathcal{O}(1)$ (yeni düğüm oluşturulmaz).

Soru 24

Döngü Tespiti (Floyd Cycle Detection). Bir bağlı listede döngü (cycle) olup olmadığını sabit bellekte belirleyin.

Çözüm.

Yine slow/fast işaretçileri kullanılır. Döngü varsa hızlı işaretçi eninde sonunda yavaş işaretçiyi yakalar (aralarındaki mesafe her adımda bir azalır). Döngü yoksa fast listenin sonuna (nullptr) ulaşır.

```

1 bool hasCycle(ListNode* head) {
2     ListNode* slow = head;
3     ListNode* fast = head;
4     while (fast && fast->next) {
5         slow = slow->next;
6         fast = fast->next->next;
7         if (slow == fast) return true; // isaretciler carpisti: dongu var
8     }
9     return false; // fast sona ulasti: dongu yok
10 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$.

Soru 25

Liste Palindrom mu? Bir bağlı listenin değerlerinin palindrom olup olmadığını $\mathcal{O}(1)$ ek bellekle kontrol edin.

Çözüm.

Üç adım: (1) slow/fast ile ortayı bul, (2) ikinci yarıyı ters çevir, (3) iki yarıyı baştan karşılaştır. Böylece diziyi kopyalamadan sabit bellekle çalışırız.

```

1 bool isPalindrome(ListNode* head) {
2     if (!head || !head->next) return true;
3     // 1) ortayı bul
4     ListNode* slow = head; ListNode* fast = head;
5     while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
6     // 2) ikinci yarıyı ters çevir
7     ListNode* prev = nullptr;
8     while (slow) {
9         ListNode* nextNode = slow->next;
10        slow->next = prev;
11        prev = slow;
12        slow = nextNode;
13    }
14    // 3) iki yarıyı karşılaştır
15    ListNode* firstHalf = head; ListNode* secondHalf = prev;
16    while (secondHalf) {
17        if (firstHalf->val != secondHalf->val) return false;
18        firstHalf = firstHalf->next;

```

```

19     secondHalf = secondHalf->next;
20 }
21 return true;
22 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$.

Dikkat

Bu çözüm listenin ikinci yarısını kalıcı olarak değiştirir. Çağırın taraf listenin bozulmamasını bekliyorsa, karşılaştırmadan sonra ikinci yarıyı tekrar ters çevirerek eski haline döndürmelisiniz.

9.3 Stack (Yığın)

Soru 26

Min Stack — $\mathcal{O}(1)$ getMin. push, pop, top ve o anki minimumu döndüren getMin işlemlerini hepsi $\mathcal{O}(1)$ olacak şekilde destekleyen bir yığın tasarlayın.

Çözüm.

Ana yığının yanında ikinci bir "minimum yığını" tutarız. Her yeni elemanla birlikte, o ana kadarki minimumu minStack tepesinde saklarız. Böylece minimum sorgusu sadece tepeye bakmaktır.

```

1  #include <stack>
2  #include <algorithm>
3  using namespace std;
4
5  class MinStack {
6      stack<int> data;
7      stack<int> minStack; // her seviyedeki calisan minimum
8  public:
9      void push(int x) {
10         data.push(x);
11         if (minStack.empty()) minStack.push(x);
12         else minStack.push(min(x, minStack.top()));
13     }
14     void pop() {
15         data.pop();
16         minStack.pop();
17     }
18     int top() const { return data.top(); }
19     int getMin() const { return minStack.top(); }
20 };

```

Karmaşıklık: Tüm işlemler $\mathcal{O}(1)$, bellek $\mathcal{O}(n)$.

Soru 27

Geçerli Parantezler. Yalnızca () [] {} karakterlerinden oluşan bir dizginin doğru şekilde kapanıp kapanmadığını kontrol edin.

Çözüm.

Açan parantezleri bir yığına iteriz. Kapanan bir parantez geldiğinde, yığının tepesindeki açanla eşleşmesi gerekir; eşleşmezse veya yığın boşsa dizgi geçersizdir. Sonda yığın boş olmalıdır.

```

1  #include <string>
2  #include <stack>
3  using namespace std;
4

```

```

5 bool isValidParentheses(const string& s) {
6     stack<char> st;
7     for (char c : s) {
8         if (c == '(' || c == '[' || c == '{') {
9             st.push(c); // acani sakla
10        } else {
11            if (st.empty()) return false; // eslesecek acan yok
12            char open = st.top(); st.pop();
13            if ((c == ')' && open != '(') ||
14                (c == ']' && open != '[') ||
15                (c == '}' && open != '{'))
16                return false; // yanlis tur eslesme
17        }
18    }
19    return st.empty(); // kapanmamis acan kalmamali
20 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(n)$.

Soru 28

Postfix (Ters Lehçe) İfade Değerlendirme. Token'lardan oluşan bir postfix ifadeyi (örneğin ["2", "1", "+", "3", "*"]) değerlendirin.

Çözüm.

Postfix ifadeler yığın için biçilmiş kaftandır: sayı görünce yığına iteriz, operatör görünce tepeden iki operandı çekip işlemi yapar ve sonucu geri iteriz. İkinci çekilen operand, işlemin sol tarafıdır (çıkarma ve bölmede sıra önemlidir).

```

1 #include <vector>
2 #include <string>
3 #include <stack>
4 using namespace std;
5
6 int evalRPN(const vector<string>& tokens) {
7     stack<long long> st;
8     for (const string& tok : tokens) {
9         if (tok == "+" || tok == "-" || tok == "*" || tok == "/") {
10             long long b = st.top(); st.pop(); // sag operand
11             long long a = st.top(); st.pop(); // sol operand
12             if (tok == "+") st.push(a + b);
13             else if (tok == "-") st.push(a - b);
14             else if (tok == "*") st.push(a * b);
15             else st.push(a / b); // tam sayi bolmesi
16        } else {
17            st.push(stoll(tok)); // sayiyi yigina it
18        }
19    }
20    return (int)st.top();
21 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(n)$.

Soru 29

Günlük Sıcaklıklar (Monoton Yığın / Next Greater). Her gün için, daha sıcak bir günün gelmesine kaç gün kaldığını bulun. Yoksa 0 yazın.

Çözüm.

İndeksleri, sıcaklıkları azalan bir yığında tutarız (monoton azalan yığın). Yeni bir gün, yığının tepesindeki günden daha sıcaksa, o tepe gün için cevabı bulmuş oluruz; onu çıkarıp gün farkını yazarız. Her indeks yığına en fazla bir kez girip çıktığından toplam iş doğrusaldır.

```

1  #include <vector>
2  #include <stack>
3  using namespace std;
4
5  vector<int> dailyTemperatures(const vector<int>& temps) {
6      int n = (int)temps.size();
7      vector<int> answer(n, 0);
8      stack<int> st; // cozulmemis gunlerin indeksleri (azalan sicaklik)
9      for (int i = 0; i < n; ++i) {
10         while (!st.empty() && temps[i] > temps[st.top()]) {
11             int prev = st.top(); st.pop();
12             answer[prev] = i - prev; // kac gun sonra daha sicak
13         }
14         st.push(i);
15     }
16     return answer;
17 }
```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(n)$. Monoton yığın, "next greater/smaller element" ailesindeki tüm problemlerin ortak kalıbıdır.

9.4 Queue ve Deque

Soru 30

İki Stack ile Kuyruk (Queue) Uygula. FIFO davranışı gösteren push, pop ve peek işlemlerini yalnızca iki yığın kullanarak gerçekleştirin.

Çözüm.

Bir inStack gelen elemanları alır, bir outStack çıkışları verir. outStack boşaldığında inStack'ın tamamını ona aktarırız; bu aktarma sırayı ters çevirdiği için en eski eleman outStack'ın tepesine gelir. Amortize edilmiş maliyet sabittir: her eleman ömrü boyunca en fazla bir kez aktarılır.

```

1  #include <stack>
2  using namespace std;
3
4  class MyQueue {
5      stack<int> inStack; // gelenler
6      stack<int> outStack; // cikacaklar (ters sirada)
7      void transferIfNeeded() {
8          if (outStack.empty()) {
9              while (!inStack.empty()) {
10                 outStack.push(inStack.top()); // sirayi ters cevirerek aktar
11                 inStack.pop();
12             }
13         }
14     }
15 public:
16     void push(int x) { inStack.push(x); }
17     int pop() {
18         transferIfNeeded();
19         int front = outStack.top(); outStack.pop();
20         return front;
21     }
22     int peek() {
```

```

23     transferIfNeeded();
24     return outStack.top();
25 }
26 bool empty() const { return inStack.empty() && outStack.empty(); }
27 };

```

Karmaşıklık: push $\mathcal{O}(1)$; pop ve peek amortize $\mathcal{O}(1)$ (en kötü tek çağrı $\mathcal{O}(n)$). Bellek $\mathcal{O}(n)$.

Soru 31

Kayan Pencere Maksimumu (Monoton Deque). nums dizisinde k boyutlu her pencerenin maksimumunu içeren diziyi $\mathcal{O}(n)$ zamanda üretin.

Çözüm.

İndeksleri, değerleri azalan bir çift uçlu kuyrukta (deque) tutarız. Yeni eleman gelince arkadan ondan küçük veya eşit tüm indeksleri atarız (onlar artık asla maksimum olamaz). Önden ise pencereden çıkmış indeksleri atarız. Böylece deque'in başı her zaman mevcut pencerenin maksimumudur.

```

1  #include <vector>
2  #include <deque>
3  using namespace std;
4
5  vector<int> maxSlidingWindow(const vector<int>& nums, int k) {
6      vector<int> result;
7      deque<int> dq; // indeksler, degerleri azalan sirada
8      for (int i = 0; i < (int)nums.size(); ++i) {
9          // arkadan: yeni elemandan kucuk/esit olanlar gereksiz
10         while (!dq.empty() && nums[dq.back()] <= nums[i])
11             dq.pop_back();
12         dq.push_back(i);
13         // onden: pencere disina cikan indeksi at
14         if (dq.front() <= i - k)
15             dq.pop_front();
16         // pencere tamamlaninca maksimumu yaz
17         if (i >= k - 1)
18             result.push_back(nums[dq.front()]);
19     }
20     return result;
21 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$ (her indeks en fazla bir kez girip çıkar), bellek $\mathcal{O}(k)$.

9.5 Hash Tabloları

Soru 32

İki Toplam (Two Sum — İndeksler). Sırasız bir nums dizisinde, toplamaları target olan iki elemanın indekslerini döndürün.

Çözüm.

Diziyi tararken her eleman için "aradığım tamamlayıcı" olan $\text{target} - \text{nums}[i]$ değerini daha önce görüp görmediğimizi hash tablosunda kontrol ederiz. Görülen değerleri indeksleriyle birlikte tabloya ekleriz. Tek geçişte çözülür.

```

1  #include <vector>
2  #include <unordered_map>
3  using namespace std;
4
5  vector<int> twoSum(const vector<int>& nums, int target) {
6      unordered_map<int, int> seen; // deger -> indeks

```

```

7   for (int i = 0; i < (int)nums.size(); ++i) {
8       int need = target - nums[i]; // aranan tamamlayıcı
9       auto it = seen.find(need);
10      if (it != seen.end())
11          return {it->second, i};
12      seen[nums[i]] = i;
13  }
14  return {};
15  }

```

Karmaşıklık: Zaman ortalama $\mathcal{O}(n)$, bellek $\mathcal{O}(n)$.

Soru 33

Anagramları Grupa. Bir kelime listesini, harfleri aynı olan (anagram) kelimeler aynı grupta olacak şekilde gruplayın.

Çözüm.

Her kelimenin harflerini sıraladığımızda, aynı anagram grubundaki tüm kelimeler aynı "imzayı" verir. Bu sıralı imzayı hash tablosunda anahtar olarak kullanıp kelimeleri gruplarız.

```

1  #include <vector>
2  #include <string>
3  #include <unordered_map>
4  #include <algorithm>
5  using namespace std;
6
7  vector<vector<string>> groupAnagrams(const vector<string>& words) {
8      unordered_map<string, vector<string>> groups;
9      for (const string& w : words) {
10         string key = w;
11         sort(key.begin(), key.end()); // anagram imzasi
12         groups[key].push_back(w);
13     }
14     vector<vector<string>> result;
15     result.reserve(groups.size());
16     for (auto& kv : groups)
17         result.push_back(move(kv.second));
18     return result;
19 }

```

Karmaşıklık: n kelime ve en fazla L uzunluk için zaman $\mathcal{O}(n \cdot L \log L)$, bellek $\mathcal{O}(n \cdot L)$. Alternatif olarak 26 harf sayımıyla oluşturulan imza sıralamayı ortadan kaldırır.

Soru 34

En Sık k Eleman. nums dizisinde en sık geçen k elemanı döndürün.

Çözüm.

Önce hash tablosuyla her elemanın frekansını sayarız. Sonra en sık k elemanı seçmek için bir min-heap (öncelik kuyruğu) tutarız; boyutu k 'yı aşınca en az frekanslıyı atarız. Böylece heap işlemleri $\mathcal{O}(\log k)$ ile sınırlı kalır.

```

1  #include <vector>
2  #include <unordered_map>
3  #include <queue>
4  using namespace std;
5
6  vector<int> topKFrequent(const vector<int>& nums, int k) {
7      unordered_map<int, int> freq;

```

```

8   for (int x : nums) ++freq[x]; // frekansları say
9   // min-heap: (frekans, deger); en az sık tepede
10  priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
11  for (auto& kv : freq) {
12      pq.push({kv.second, kv.first});
13      if ((int)pq.size() > k) pq.pop(); // en az sikligi cikar
14  }
15  vector<int> result;
16  while (!pq.empty()) { result.push_back(pq.top().second); pq.pop(); }
17  return result;
18 }

```

Karmaşıklık: Zaman $\mathcal{O}(n \log k)$, bellek $\mathcal{O}(n)$. Frekansa göre bucket sort ile $\mathcal{O}(n)$ de yapılabilir.

Soru 35

İlk Tekrarsız Karakter. Bir dizgide yalnızca bir kez geçen ilk karakterin indeksini döndürün; yoksa -1 döndürün.

Çözüm.

İki geçiş: birinci geçişte her karakterin sayısını hash tablosunda (veya sabit boyutlu dizide) tutarız; ikinci geçişte soldan ilk kez sayısı 1 olan karakteri buluruz. Dizgi sırasını koruduğumuz için ikinci geçiş "ilk" olanı garanti eder.

```

1  #include <string>
2  #include <array>
3  using namespace std;
4
5  int firstUniqueChar(const string& s) {
6      array<int, 256> count{}; // sıfırla başlatılır
7      for (unsigned char c : s) ++count[c]; // 1. geçiş: say
8      for (int i = 0; i < (int)s.size(); ++i)
9          if (count[(unsigned char)s[i]] == 1) // 2. geçiş: ilk tekil
10             return i;
11     return -1;
12 }

```

Karmaşıklık: Zaman $\mathcal{O}(n)$, bellek $\mathcal{O}(1)$ (sabit alfabe).

9.6 Uygulama: LRU Cache

LRU Cache Nedir?

LRU (Least Recently Used — En Az Kısa Süre Önce Kullanılan) önbellek, sabit kapasiteli bir yapıdır: kapasite dolduğunda, en uzun süredir dokunulmamış (en eski erişilen) elemanı atarak yer açar. Amaç hem get hem put işlemini $\mathcal{O}(1)$ yapmaktır. Bunun için iki veri yapısını birlikte kullanırız: bir unordered_map (anahtardan düğüme $\mathcal{O}(1)$ erişim için) ve bir çift yönlü bağlı liste (std::list, kullanım sırasını korumak ve herhangi bir elemanı $\mathcal{O}(1)$ taşımak için). Liste başı = en son kullanılan; liste sonu = atılacak aday.

Soru 36

LRU Cache Tam Implementasyonu. capacity kapasiteli bir LRUCache sınıfı yazın. get(key) anahtarı varsa değerini döndürsün ve onu "en son kullanılan" yapsın (yoksa -1). put(key, value) ekleme/güncelleme yapsın; kapasite aşılsa en az kullanılanı atsın. Her iki işlem de $\mathcal{O}(1)$ olsun.

Çözüm.

std::list<pair<int,int> kullanımı sırasını (baş = en yeni) tutar; her düğüm (key, value) çiftidir. unordered_map<int, list iterator> her anahtarı listedeki düğüme bağlar. list::splice

ile bir düğümü kopyalamadan, iteratörlerini geçersiz kılmadan listenin başına $\mathcal{O}(1)$ taşıyabiliriz — bu, tasarımın kilit noktasıdır.

```

1  #include <list>
2  #include <unordered_map>
3  #include <utility>
4  using namespace std;
5
6  class LRUCache {
7      int capacity;
8      // liste: on = en son kullanılan, arka = en az kullanılan
9      list<pair<int,int>> items;           // (key, value)
10     unordered_map<int, list<pair<int,int>>::iterator> index; // key -> dugum
11 public:
12     explicit LRUCache(int capacity) : capacity(capacity) {}
13
14     int get(int key) {
15         auto it = index.find(key);
16         if (it == index.end()) return -1; // bulunamadi
17         // erisildi: bu dugumu listenin basina tasi (en son kullanılan)
18         items.splice(items.begin(), items, it->second);
19         return it->second->second; // value
20     }
21
22     void put(int key, int value) {
23         auto it = index.find(key);
24         if (it != index.end()) {
25             // var olan anahtar: degeri guncelle ve basa tasi
26             it->second->second = value;
27             items.splice(items.begin(), items, it->second);
28             return;
29         }
30         // kapasite doluysa en az kullanilani (listenin arkasi) at
31         if ((int)items.size() == capacity) {
32             int oldKey = items.back().first;
33             index.erase(oldKey);
34             items.pop_back();
35         }
36         // yeni elemani basa ekle ve indeksle
37         items.emplace_front(key, value);
38         index[key] = items.begin();
39     }
40 };

```

Karmaşıklık: get ve put her ikisi de $\mathcal{O}(1)$ (splice, emplace, erase hepsi sabit zaman). Bellek $\mathcal{O}(\text{capacity})$.

Dikkat

splice kullanmanın kritik nedeni, taşınan düğümün iteratörünü geçersiz kılmamasıdır. Bu yüzden index tablosundaki iteratörler taşımadan sonra da geçerli kalır; ayrıca yeniden ekleme yapmaya gerek kalmaz. `items.erase` + `push_front` ile yapsaydınız iteratörü güncellemeyi unutmak yaygın bir hata olurdu.

Soru 37

LRU Cache Kullanımı. Kapasitesi 2 olan bir LRU Cache üzerinde şu işlemler sırayla uygulanıyor: `put(1,1)`, `put(2,2)`, `get(1)`, `put(3,3)`, `get(2)`, `put(4,4)`, `get(1)`, `get(3)`, `get(4)`. Her `get` çağrısının çıktısını ve `put(3,3)` ile `put(4,4)` sırasında hangi anahtarın atıldığını izleyin.

Çözüm.

Adım adım (liste solu = en son kullanılan):

İşlem	Durum ve Sonuç
<code>put(1,1)</code>	liste: [1]
<code>put(2,2)</code>	liste: [2, 1]
<code>get(1)</code>	1 döner; 1 başa taşınır $\rightarrow$ [1, 2]
<code>put(3,3)</code>	kapasite dolu; en eski olan 2 atılır $\rightarrow$ [3, 1]
<code>get(2)</code>	2 atıldığı için -1 döner
<code>put(4,4)</code>	kapasite dolu; en eski olan 1 atılır $\rightarrow$ [4, 3]
<code>get(1)</code>	1 atıldığı için -1 döner
<code>get(3)</code>	3 döner; 3 başa taşınır $\rightarrow$ [3, 4]
<code>get(4)</code>	4 döner; 4 başa taşınır $\rightarrow$ [4, 3]

Doğrulama kodu:

```

1 #include <iostream>
2
3 int main() {
4     LRUCache cache(2);
5     cache.put(1, 1);
6     cache.put(2, 2);
7     std::cout << cache.get(1) << "\n"; // 1
8     cache.put(3, 3); // 2 atilir
9     std::cout << cache.get(2) << "\n"; // -1
10    cache.put(4, 4); // 1 atilir
11    std::cout << cache.get(1) << "\n"; // -1
12    std::cout << cache.get(3) << "\n"; // 3
13    std::cout << cache.get(4) << "\n"; // 4
14    return 0;
15 }
```

Beklenen çıktı satırları: 1, -1, -1, 3, 4. Bu örnek, LRU mantığında bir elemana `get` veya `put` ile "dokunmanın" onu atılmaktan koruduğunu, dokunulmayan elemanın zamanla atıldığını gösterir.

Bilgi

Karşılaştırmalı özet — işlem başına tipik karmaşıklıklar: two-pointer / sliding window / prefix sum $\rightarrow \mathcal{O}(n)$; bağlı liste temel işlemleri $\rightarrow \mathcal{O}(n)$ zaman, $\mathcal{O}(1)$ bellek; yığın/kuyruk işlemleri $\rightarrow \mathcal{O}(1)$ (kuyruk için amortize); hash tablosu arama/ekleme $\rightarrow$ ortalama $\mathcal{O}(1)$; LRU Cache `get/put` $\rightarrow \mathcal{O}(1)$. Bu desenleri tanımak, birçok karmaşık problemi tanıdık yapı taşlarına indirgemenizi sağlar.

KISIM 3

Ağaçlar

10. Ağaçlar ve İkili Ağaçlar

Ağaç, düğümlerin hiyerarşik olarak bağlandığı, döngüsüz bir yapıdır. En üstteki düğüm *kök* (root), altındakiler *çocuk* (child), çocuğu olmayanlar *yaprak* (leaf) düğümlerdir. Dosya sistemleri, ifade ağaçları, arama yapıları ve daha fazlası ağaçlarla modellenir.

Terim	Anlamı
Kök (root)	En üstteki, ebeveyni olmayan düğüm.
Yaprak (leaf)	Çocuğu olmayan düğüm.
Derinlik (depth)	Kökten o düğüme olan kenar sayısı.
Yükseklik (height)	Düğümünden en uzak yaprağa olan kenar sayısı.
Alt ağaç (subtree)	Bir düğüm ve altındaki tüm düğümler.

10.1 İkili Ağaç ve Dolaşımalar (Traversals)

İkili ağaçta her düğümün en fazla iki çocuğu (sol, sağ) vardır. Bir ağacı “dolaşmak”, tüm düğümleri belirli bir sırada ziyaret etmektir. Üç temel derinlik-öncelikli dolaşım vardır:

Dolaşım	Sıra	Kullanım
Pre-order	Kök, Sol, Sağ	Ağacı kopyalama, önek ifade.
In-order	Sol, Kök, Sağ	BST’de <i>sıralı</i> çıktı verir.
Post-order	Sol, Sağ, Kök	Ağacı silme, sonек ifade.

```

1 struct TreeNode {
2     int data;
3     TreeNode* left = nullptr;
4     TreeNode* right = nullptr;
5     TreeNode(int v) : data(v) {}
6 };
7

```

```

8 void inorder(TreeNode* k) {           // Sol -> Kok -> Sag
9     if (!k) return;
10    inorder(k->left);
11    cout << k->data << " ";
12    inorder(k->right);
13 }
14
15 void preorder(TreeNode* k) {         // Kok -> Sol -> Sag
16     if (!k) return;
17     cout << k->data << " ";
18     preorder(k->left);
19     preorder(k->right);
20 }
21
22 void postorder(TreeNode* k) {        // Sol -> Sag -> Kok
23     if (!k) return;
24     postorder(k->left);
25     postorder(k->right);
26     cout << k->data << " ";
27 }

```

10.2 Seviye-Sıralı Dolaşım (BFS) ve Yükseklik

Seviye sıralı dolaşım, ağacı katman katman gezer ve bir kuyruk kullanır. Yükseklik ise özyinelemeyle bulunur.

```

1 void levelOrder(TreeNode* root) {
2     if (!root) return;
3     queue<TreeNode*> q;
4     q.push(root);
5     while (!q.empty()) {
6         TreeNode* d = q.front(); q.pop();
7         cout << d->data << " ";
8         if (d->left) q.push(d->left);
9         if (d->right) q.push(d->right);
10    }
11 }
12
13 int height(TreeNode* k) {
14     if (!k) return -1;           // bos agac: -1 (kenar sayisi)
15     return 1 + max(height(k->left), height(k->right));
16 }

```

11. İkili Arama Ağacı (BST)

İkili arama ağacı (Binary Search Tree), *sıralı* bir ikili ağaçtır: her düğüm için **sol alt ağacındaki tüm değerler ondan küçük, sağ alt ağacındakiler büyüktür**. Bu değişmez (invariant) sayesinde arama, ekleme ve silme, ağacın yüksekliği kadar sürede — dengeli bir ağaçta $\mathcal{O}(\log n)$ 'de — yapılır.

11.1 Arama ve Ekleme

Aranan değeri kökle karşılaştırırız: küçükse sola, büyükse sağa gideriz. Her adımda arama uzayını yarıya indiririz.

```

1 struct Node {
2     int data;

```

```

3   Node* left = nullptr;
4   Node* right = nullptr;
5   Node(int v) : data(v) {}
6 };
7
8 Node* search(Node* k, int x) {           // O(h)
9     while (k && k->data != x)
10        k = (x < k->data) ? k->left : k->right;
11    return k;                           // bulunamazsa nullptr
12 }
13
14 Node* insert(Node* k, int x) {           // O(h)
15     if (!k) return new Node(x);
16     if (x < k->data) k->left = insert(k->left, x);
17     else if (x > k->data) k->right = insert(k->right, x);
18     // esitse: eklemiyoruz (tekrarsiz BST)
19     return k;
20 }

```

11.2 Silme: Üç Durum

Silme en incelikli işlemdir çünkü üç ayrı durum vardır. Anahtar fikir: iki çocuklu bir düğümü silerken, onu *sağ alt ağacının en küçük değeri* (in-order ardılı) ile değiştiririz.

Durum	Yapılan
Yaprak (çocuk yok)	Doğrudan sil.
Tek çocuk	Düğümü çocuğuyla değiştir.
İki çocuk	In-order ardılı ile değiştir, sonra ardılı sil.

```

1 Node* findMin(Node* k) { while (k->left) k = k->left; return k; }
2
3 Node* remove(Node* k, int x) {
4     if (!k) return nullptr;
5     if (x < k->data) k->left = remove(k->left, x);
6     else if (x > k->data) k->right = remove(k->right, x);
7     else {                                     // dugum bulundu
8         if (!k->left) { Node* s = k->right; delete k; return s; }
9         if (!k->right) { Node* s = k->left; delete k; return s; }
10        Node* successor = findMin(k->right); // iki cocuklu durum
11        k->data = successor->data;           // ardili yukari tasi
12        k->right = remove(k->right, successor->data);
13    }
14    return k;
15 }

```

Dikkat

BST'nin $\mathcal{O}(\log n)$ garantisi *dengeli* olmasına bağlıdır. Sıralı veriyi (1, 2, 3, 4, ...) sırayla eklersen ağaç dev bir bağlı listeye dönüşür ve işlemler $\mathcal{O}(n)$ 'e çıkar. Bu problemi bir sonraki bölümdeki kendi-kendini dengeleyen ağaçlar çözer.

12. Dengeli Ağaçlar: AVL ve Kırmızı-Siyah

Dengeli arama ağaçları, her eklemekten/silmeden sonra yapıyı yeniden düzenleyerek yüksekliği $\mathcal{O}(\log n)$ 'de tutar; böylece BST'nin en kötü durum sorununu ortadan kaldırır. Dengeleme, *rotasyon* (rotation) adı verilen $\mathcal{O}(1)$ 'lik yerel işlemlerle yapılır.

12.1 Rotasyonlar: Dengelemenin Temeli

Rotasyon, BST sırasını bozmadan düğümleri yeniden bağlayarak alt ağaç yüksekliklerini değiştirir. Sol ve sağ rotasyon birbirinin simetriğidir.

```

1 struct AVLNode {
2     int data, height = 1;
3     AVLNode* left = nullptr;
4     AVLNode* right = nullptr;
5     AVLNode(int v) : data(v) {}
6 };
7
8 int getHeight(AVLNode* d) { return d ? d->height : 0; }
9 int balance(AVLNode* d) { return d ? getHeight(d->left) - getHeight(d->right) : 0; }
10 void update(AVLNode* d) { d->height = 1 + max(getHeight(d->left), getHeight(d->right)); }
11
12 AVLNode* rotateRight(AVLNode* y) { // sol ağır düğümü dengele
13     AVLNode* x = y->left;
14     AVLNode* T = x->right;
15     x->right = y; y->left = T; // rotasyon
16     update(y); update(x);
17     return x; // yeni kök
18 }
19
20 AVLNode* rotateLeft(AVLNode* x) { // sağ ağır düğümü dengele
21     AVLNode* y = x->right;
22     AVLNode* T = y->left;
23     y->left = x; x->right = T;
24     update(x); update(y);
25     return y;
26 }

```

12.2 AVL Ağacına Ekleme

AVL ağacında her düğüm için *denge faktörü* (sol yükseklik – sağ yükseklik) $\{-1, 0, +1\}$ aralığında tutulur. Ekleme sonrası bir düğüm bu aralığı aşarsa, dört durumdan (LL, RR, LR, RL) uygun rotasyon(lar) uygulanır.

```

1 AVLNode* insert(AVLNode* d, int x) {
2     if (!d) return new AVLNode(x);
3     if (x < d->data) d->left = insert(d->left, x);
4     else if (x > d->data) d->right = insert(d->right, x);
5     else return d; // tekrar yok
6     update(d);
7
8     int b = balance(d);
9     if (b > 1 && x < d->left->data) return rotateRight(d); // LL
10    if (b < -1 && x > d->right->data) return rotateLeft(d); // RR
11    if (b > 1 && x > d->left->data) { // LR
12        d->left = rotateLeft(d->left); return rotateRight(d);
13    }
14    if (b < -1 && x < d->right->data) { // RL

```

```

15     d->right = rotateRight(d->right); return rotateLeft(d);
16 }
17 return d;
18 }

```

12.3 AVL vs Kırmızı-Siyah Ağaç

Özellik	AVL	Kırmızı-Siyah
Denge	Katı (daha kısa)	Gevşek
Arama	Daha hızlı	Biraz yavaş
Ekleme/silme	Daha çok rotasyon	Daha az rotasyon
Kullanım	Okuma ağırlıklı	std::map, std::set

Bilgi

C++'ın std::map ve std::set yapıları içeride kırmızı-siyah ağaç kullanır. Yani pratikte kendi dengeli ağacını yazman nadiren gerekir; ama nasıl çalıştığını bilmek, $\mathcal{O}(\log n)$ garantisinin nereden geldiğini anlamayı sağlar.

13. Heap ve Öncelik Kuyruğu

Heap (yığıt/öbek), **tam ikili ağaç** (complete binary tree) biçiminde bir yapıdır ve bir *öncelik* değişmezini korur: **max-heap**'te her ebeveyn çocuklarından büyük veya eşittir (min-heap'te küçük veya eşit). Bu sayede en büyük (veya en küçük) elemana $\mathcal{O}(1)$ 'de erişilir; ekleme ve çıkarma $\mathcal{O}(\log n)$ 'dir. Öncelik kuyruğu (priority queue) heap ile uygulanır.

13.1 Diziyle Heap Gösterimi

Tam ikili ağaç olduğundan heap'i bir dizide, işaretçi olmadan tutabiliriz. i indeksli düğüm için: sol çocuk $2i + 1$, sağ çocuk $2i + 2$, ebeveyn $(i - 1)/2$ 'dir.

```

1 class MaxHeap {
2     vector<int> h;
3     void siftUp(int i) { // ekleme sonrası:  $\mathcal{O}(\log n)$ 
4         while (i > 0 && h[(i - 1) / 2] < h[i]) {
5             swap(h[i], h[(i - 1) / 2]);
6             i = (i - 1) / 2;
7         }
8     }
9     void siftDown(int i) { // cikarma sonrası:  $\mathcal{O}(\log n)$ 
10        int n = h.size();
11        while (true) {
12            int largest = i, left = 2*i + 1, right = 2*i + 2;
13            if (left < n && h[left] > h[largest]) largest = left;
14            if (right < n && h[right] > h[largest]) largest = right;
15            if (largest == i) break;
16            swap(h[i], h[largest]);
17            i = largest;
18        }
19    }

```

```

20 public:
21     void push(int x) { h.push_back(x); siftUp(h.size() - 1); }
22     int top() const { return h[0]; } // O(1)
23     void pop() { // en buyugu cikar
24         h[0] = h.back(); h.pop_back();
25         if (!h.empty()) siftDown(0);
26     }
27     bool empty() const { return h.empty(); }
28 };

```

13.2 STL Öncelik Kuyruğu

```

1 #include <queue>
2 // Varsayılan: MAX-heap (en büyük tepede)
3 priority_queue<int> pq;
4 pq.push(3); pq.push(1); pq.push(4);
5 cout << pq.top(); // 4
6 pq.pop();
7
8 // MIN-heap için:
9 priority_queue<int, vector<int>, greater<int>> minpq;
10 minpq.push(3); minpq.push(1); minpq.push(4);
11 cout << minpq.top(); // 1

```

13.3 Uygulama: En Küçük k Eleman (ve Heapsort)

Bir akıştaki en küçük k elemanı, boyutu k olan bir *max-heap* ile $\mathcal{O}(n \log k)$ 'de buluruz: heap doluyorsa ve yeni eleman tepeden küçükse tepeyi at, yenisini ekle.

```

1 vector<int> kSmallest(const vector<int>& a, int k) {
2     priority_queue<int> maxHeap; // boyutu en fazla k
3     for (int x : a) {
4         maxHeap.push(x);
5         if ((int)maxHeap.size() > k) maxHeap.pop(); // en buyugu at
6     }
7     vector<int> result;
8     while (!maxHeap.empty()) { result.push_back(maxHeap.top()); maxHeap.pop(); }
9     return result;
10 }

```

Heapsort

Tüm elemanları bir heap'e atıp tek tek çıkarırsak sıralı elde ederiz: bu $\mathcal{O}(n \log n)$ 'lik *heapsort* algoritmasıdır. STL'de `std::make_heap`, `std::push_heap`, `std::pop_heap` ile aynı mekanizmaya doğrudan erişilir.

14. Trie (Önek Ağacı)

Trie (önek ağacı / prefix tree), string'leri karakter karakter saklayan bir ağaçtır. Kökten bir düğüme giden yol, bir öneki temsil eder. Bir kelimenin aranması, eklenmesi veya bir önekle başlayan kelimelerin bulunması, kelime uzunluğu L olmak üzere $\mathcal{O}(L)$ 'de yapılır — sözlük boyutundan bağımsız! Otomatik tamamlama, yazım denetimi ve IP yönlendirme trie kullanır.

```

1 struct TrieNode {
2     TrieNode* children[26] = {nullptr}; // 'a'-'z'
3     bool isEndOfWord = false;
4 };
5
6 class Trie {
7     TrieNode* root = new TrieNode();
8 public:
9     void insert(const string& s) { // O(L)
10         TrieNode* d = root;
11         for (char c : s) {
12             int i = c - 'a';
13             if (!d->children[i]) d->children[i] = new TrieNode();
14             d = d->children[i];
15         }
16         d->isEndOfWord = true;
17     }
18     bool search(const string& s) const { // tam kelime var mi?
19         TrieNode* d = root;
20         for (char c : s) {
21             int i = c - 'a';
22             if (!d->children[i]) return false;
23             d = d->children[i];
24         }
25         return d->isEndOfWord;
26     }
27     bool startsWith(const string& prefix) const { // bu onekle baslayan var mi?
28         TrieNode* d = root;
29         for (char c : prefix) {
30             int i = c - 'a';
31             if (!d->children[i]) return false;
32             d = d->children[i];
33         }
34         return true; // isEndOfWord'e bakmıyoruz
35     }
36 };

```

İpucu

Trie, bellek karşılığında hız satın alır: her düğüm 26 işaretçi tutabilir. Bellek kritikse çocukları `unordered_map<char, TrieNode*>` ile seyrek tutmak veya sıkıştırılmış trie (radix tree) kullanmak daha verimlidir.

15. Segment Ağacı ve Fenwick Ağacı

Bir dizide hem *aralık sorguları* (örneğin “[*l*, *r*] toplamı / minimumu”) hem de *güncellemeler* sık yapıyorsa, düz dizi yetersizdir: sorgu $\mathcal{O}(n)$ sürer. Segment ağacı ve Fenwick ağacı, her ikisini de $\mathcal{O}(\log n)$ ’de yapar.

15.1 Segment Ağacı

Segment ağacı, diziyi aralıklara bölen bir ikili ağaçtır: her düğüm bir aralığın özetini (toplam, min, max...) tutar. Kök tüm diziyi, yapraklar tek elemanları temsil eder.

```

1 class SegmentTree {
2     int n;

```

```

3   vector<long long> tree;           // 1-tabanlı, 4n boyut
4   void build(const vector<int>& a, int node, int l, int r) {
5       if (l == r) { tree[node] = a[l]; return; }
6       int mid = (l + r) / 2;
7       build(a, 2*node, l, mid);
8       build(a, 2*node+1, mid+1, r);
9       tree[node] = tree[2*node] + tree[2*node+1]; // toplam birleştir
10  }
11  long long query(int node, int l, int r, int ql, int qr) {
12      if (qr < l || r < ql) return 0; // hic kesismiyor
13      if (ql <= l && r <= qr) return tree[node]; // tamamen iceride
14      int mid = (l + r) / 2;
15      return query(2*node, l, mid, ql, qr)
16             + query(2*node+1, mid+1, r, ql, qr);
17  }
18  void update(int node, int l, int r, int i, int value) {
19      if (l == r) { tree[node] = value; return; }
20      int mid = (l + r) / 2;
21      if (i <= mid) update(2*node, l, mid, i, value);
22      else update(2*node+1, mid+1, r, i, value);
23      tree[node] = tree[2*node] + tree[2*node+1];
24  }
25  public:
26      SegmentTree(const vector<int>& a) : n(a.size()), tree(4*n) {
27          build(a, 1, 0, n-1);
28      }
29      long long query(int l, int r) { return query(1, 0, n-1, l, r); }
30      void update(int i, int value) { update(1, 0, n-1, i, value); }
31  };

```

15.2 Fenwick Ağacı (Binary Indexed Tree)

Fenwick ağacı, yalnızca *önek toplamı* tipi sorgular için segment ağacına göre çok daha kısa ve az bellekli bir alternatiftir. Sihri, bir indeksin son kurulu bitini ($i \& -i$) kullanarak ağaçta zıplamaktır.

```

1   class Fenwick {
2       int n;
3       vector<long long> tree;           // 1-tabanlı
4   public:
5       Fenwick(int n) : n(n), tree(n + 1, 0) {}
6
7       void add(int i, long long value) { // a[i] += value, O(log n)
8           for (++i; i <= n; i += i & -i) // son biti ekleyerek ilerle
9               tree[i] += value;
10      }
11      long long prefixSum(int i) { // a[0..i] toplamı, O(log n)
12          long long s = 0;
13          for (++i; i > 0; i -= i & -i) // son biti cikararak geri git
14              s += tree[i];
15          return s;
16      }
17      long long rangeSum(int l, int r) {
18          return prefixSum(r) - (l > 0 ? prefixSum(l - 1) : 0);
19      }
20  };

```

Kıstas	Segment Ağacı	Fenwick
Esneklik	Min/max/gcd/... her şey	Genelde sadece toplam
Kod uzunluğu	Uzun	Çok kısa
Bellek	$\sim 4n$	n
Aralık güncelleme	Lazy propagation ile	Sınırlı

16. Ağaçlar — Alıştırmalar ve Çözümler

Bu bölümde ağaç veri yapılarının temel ve ileri düzey problemlerini çözeceğiz. İkili ağaçlardan başlayıp arama ağaçları, dengeli ağaçlar, heap yapıları, Trie ve aralık sorgu ağaçlarına (Segment/Fenwick) kadar geniş bir yelpazeyi ele alacağız. Her çözümde zaman ve alan karmaşıklığını açıkça belirteceğiz.

Bilgi

Bölüm boyunca ikili ağaç düğümü için şu standart yapıyı kullanacağız:

```
1 struct TreeNode {
2     int val;
3     TreeNode *left, *right;
4     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
5 };
```

16.1 İkili Ağaç ve Dolaşım

Soru 38

Bir ikili ağacın yüksekliğini (height) hesaplayan bir fonksiyon yazın. Yükseklik, kökten en derindeki yaprağa giden yoldaki kenar sayısıdır. Boş ağacın yüksekliği -1 , tek düğümlü ağacın yüksekliği 0 'dır.

Çözüm.

Yükseklik, özyinelemeli olarak tanımlanır: bir düğümün yüksekliği, sol ve sağ alt ağaçlarının yüksekliklerinin maksimumundan bir fazladır. Boş ağaç için taban durumu -1 döndürürüz; böylece yaprak düğümü otomatik olarak $\max(-1, -1) + 1 = 0$ değerini alır.

```
1 int height(TreeNode* root) {
2     if (root == nullptr) return -1;           // taban durumu: boş ağaç
3     int leftH = height(root->left);           // sol alt ağacın yüksekliği
4     int rightH = height(root->right);          // sağ alt ağacın yüksekliği
5     return 1 + std::max(leftH, rightH);        // en derin taraf + kenar
6 }
```

Karmaşıklık

Her düğüm bir kez ziyaret edildiği için zaman $\mathcal{O}(n)$, özyineleme yığını en kötü durumda (de-jenere/çarpık ağaç) $\mathcal{O}(n)$, dengeli ağaçta $\mathcal{O}(\log n)$ alan kullanır.

İpucu

Yükseklik (height) ve derinlik (depth) sık karıştırılır. Derinlik kökten *aşağı* sayılır (kökün derinliği 0), yükseklik yaprakтан *yukarı* sayılır (yaprığın yüksekliği 0). Bir düğümün derinliğini bulmak için kökten aşağı inerken sayaç artırılır.

Soru 39

Bir ikili ağacın çapını (diameter) bulun. Çap, ağaçtaki herhangi iki düğüm arasındaki en uzun yolun kenar sayısıdır. Bu yolun kökten geçmesi gerekmez.

Çözüm.

Herhangi bir düğümden geçen en uzun yol, o düğümün sol alt ağacının yüksekliği ile sağ alt ağacının yüksekliğinin toplamıdır. Çap, tüm düğümler üzerindeki bu toplamın maksimumudur. Yüksekliği hesaplarırken aynı gezinme sırasında çapı da güncelleyerek $\mathcal{O}(n)$ çözüm elde ederiz; her düğümden ayrı ayrı yükseklik hesaplayan naif yaklaşım $\mathcal{O}(n^2)$ olurdu.

```

1 int diameterHelper(TreeNode* node, int& best) {
2     if (node == nullptr) return -1;           // boş ağaç yüksekliği -1
3     int leftH = diameterHelper(node->left, best);
4     int rightH = diameterHelper(node->right, best);
5     // bu düğümden geçen yolun kenar sayısı: (leftH+1) + (rightH+1)
6     best = std::max(best, leftH + rightH + 2);
7     return 1 + std::max(leftH, rightH);       // yüksekliği yukarı taşı
8 }
9
10 int diameter(TreeNode* root) {
11     int best = 0;
12     diameterHelper(root, best);
13     return best;
14 }
```

Karmaşıklık

Tek geçişte hem yükseklik hem çap hesaplandığı için zaman $\mathcal{O}(n)$, alan $\mathcal{O}(h)$ (ağaç yüksekliği kadar yığın).

Soru 40

Yığın (stack) kullanarak özyinelemesiz (iteratif) inorder dolaşımı yazın ve düğüm değerlerini bir vektörde döndürün.

Çözüm.

Inorder dolaşımda sıra: sol alt ağaç, kök, sağ alt ağaç. İteratif çözümde bir yığın kullanırız: mevcut düğümden başlayarak sürekli sola inip yol boyunca düğümleri yığma koyarız. Sola gidilemediğinde yığından bir düğüm çıkarıp değerini işleriz ve o düğümün sağ çocuğuna geçeriz. Bu, özyinelemenin yaptığı işi elle simüle eder.

```

1 std::vector<int> inorderIterative(TreeNode* root) {
2     std::vector<int> result;
3     std::stack<TreeNode*> st;
4     TreeNode* curr = root;
5     while (curr != nullptr || !st.empty()) {
6         while (curr != nullptr) { // olabildiğince sola in
7             st.push(curr);
8             curr = curr->left;
9         }
10        result.push_back(st.top()->val);
11        st.pop();
12        curr = st.top()->right;
13    }
14 }
```

```

10     curr = st.top(); st.pop(); // en soldaki işlenmemiş düğüm
11     result.push_back(curr->val); // kökü ziyaret et
12     curr = curr->right; // sağ alt ağaca geç
13 }
14 return result;
15 }

```

Karmaşıklık

Her düğüm yığına bir kez girip bir kez çıktığı için zaman $\mathcal{O}(n)$, yığın en kötü durumda $\mathcal{O}(h)$ alan (çarpık ağaçta $\mathcal{O}(n)$).

Soru 41

Bir ikili ağacın seviye sıralı (level-order / BFS) dolaşımını, her seviyeyi ayrı bir alt vektör olacak şekilde döndürün.

Çözüm.

Seviye sıralı dolaşım için bir kuyruk (queue) kullanırız. Her iterasyonda kuyruğun o anki boyutu, işlenecek seviyedeki düğüm sayısını verir. Bu sayı kadar düğümü çıkarıp değerlerini o seviyenin vektörüne ekler, çocuklarını kuyruğa koyarız. Böylece seviyeleri birbirinden ayırmış oluruz.

```

1  std::vector<std::vector<int>> levelOrder(TreeNode* root) {
2      std::vector<std::vector<int>> levels;
3      if (root == nullptr) return levels;
4      std::queue<TreeNode*> q;
5      q.push(root);
6      while (!q.empty()) {
7          int sz = q.size(); // bu seviyedeki düğüm sayısı
8          std::vector<int> level;
9          for (int i = 0; i < sz; ++i) {
10             TreeNode* node = q.front(); q.pop();
11             level.push_back(node->val);
12             if (node->left) q.push(node->left);
13             if (node->right) q.push(node->right);
14         }
15         levels.push_back(level);
16     }
17     return levels;
18 }

```

Karmaşıklık

Her düğüm bir kez kuyruğa girer/çıkartılır: zaman $\mathcal{O}(n)$. Kuyruk en geniş seviye kadar düğüm tutabilir, en kötü durumda $\mathcal{O}(n)$ alan.

Soru 42

İki ikili ağacın yapısal olarak aynı ve düğüm değerlerinin özdeş olup olmadığını kontrol eden bir fonksiyon yazın.

Çözüm.

İki ağaç aynıysa: ya ikisi de boş, ya da ikisinin de kökleri aynı değere sahiptir ve sol alt ağaçları birbiriyle, sağ alt ağaçları birbiriyle aynıdır. Bu tanımları doğrudan özyinelemeye çeviririz.

```

1  bool isSameTree(TreeNode* p, TreeNode* q) {
2      if (p == nullptr && q == nullptr) return true; // ikisi de boş

```

```

3   if (p == nullptr || q == nullptr) return false; // biri boş, diğeri değil
4   if (p->val != q->val) return false;           // değerler farklı
5   return isSameTree(p->left, q->left)           // sol alt ağaçlar
6       && isSameTree(p->right, q->right);       // sağ alt ağaçlar
7 }

```

Karmaşıklık

En fazla $\min(n_1, n_2)$ düğüm çifti karşılaştırılır: zaman $\mathcal{O}(n)$, özyineleme yığını $\mathcal{O}(h)$ alan.

Soru 43

Bir ikili ağacın ayna görüntüsünü alın (invert): her düğümün sol ve sağ çocuklarını yer değiştirin.

Çözüm.

Ağacı tersine çevirmek için her düğümde sol ve sağ çocukları takas eder, sonra alt ağaçlar için özyinelemeli olarak aynı işlemi yaparız. Sıra önemli değildir; önce takas edip sonra inebilir ya da önce inip sonra takas edebiliriz.

```

1  TreeNode* invertTree(TreeNode* root) {
2      if (root == nullptr) return nullptr;
3      std::swap(root->left, root->right); // çocukları yer değiştir
4      invertTree(root->left);           // yeni sol (eski sağ) alt ağacı çevir
5      invertTree(root->right);          // yeni sağ (eski sol) alt ağacı çevir
6      return root;
7  }

```

Karmaşıklık

Her düğüm bir kez ziyaret edilir: zaman $\mathcal{O}(n)$, alan $\mathcal{O}(h)$.

16.2 İkili Arama Ağacı (BST)

Bilgi

İkili arama ağacında (BST) her düğüm için: sol alt ağaçtaki tüm değerler düğümün değerinden *küçük*, sağ alt ağaçtaki tüm değerler düğümün değerinden *büyük*tür. Bu özellik alt ağaçların *tamamı* için geçerli olmalıdır, sadece doğrudan çocuklar için değil.

Soru 44

Verilen bir ikili ağacın geçerli bir BST olup olmadığını doğrulayın.

Çözüm.

Yaygın bir hata, yalnızca her düğümü doğrudan çocuklarıyla karşılaştırmaktır; bu yeterli değildir çünkü BST kuralı tüm alt ağaç için geçerli olmalıdır. Doğru yöntem, her düğüme izin verilen bir (alt sınır, üst sınır) aralığı taşımaktır. Sola inerken üst sınır düğümün değeri olur, sağa inerken alt sınır düğümün değeri olur.

```

1  bool validate(TreeNode* node, long lo, long hi) {
2      if (node == nullptr) return true;
3      // düğüm değeri açık aralık (lo, hi) içinde olmalı
4      if (node->val <= lo || node->val >= hi) return false;
5      return validate(node->left, lo, node->val) // üst sınır daralır
6          && validate(node->right, node->val, hi); // alt sınır daralır
7  }

```

```

8
9 bool isValidBST(TreeNode* root) {
10     return validate(root, LONG_MIN, LONG_MAX);
11 }

```

Dikkat

Sınırlar için long kullandık çünkü düğüm değeri INT_MIN veya INT_MAX olabilir; int sınırlarla başlarsak taşma ve yanlış karşılaştırma olur.

Karmaşıklık

Her düğüm bir kez ziyaret edilir: zaman $\mathcal{O}(n)$, alan $\mathcal{O}(h)$. Alternatif olarak inorder dolaşımın kesin artan olup olmadığı da kontrol edilebilir.

Soru 45

Bir BST'de k . en küçük elemanı bulun ($1 \leq k \leq n$).

Çözüm.

BST'nin inorder dolaşımı değerleri artan sırada verir. Dolayısıyla inorder sırasında k . ziyaret edilen düğüm cevaptır. Tüm dolaşımı bitirmeden, k . düğüme ulaştığımızda durabiliriz. İteratif inorder ile bunu temiz biçimde yaparız.

```

1 int kthSmallest(TreeNode* root, int k) {
2     std::stack<TreeNode*> st;
3     TreeNode* curr = root;
4     while (curr != nullptr || !st.empty()) {
5         while (curr != nullptr) {           // sola in
6             st.push(curr);
7             curr = curr->left;
8         }
9         curr = st.top(); st.pop();
10        if (--k == 0) return curr->val; // k. ziyaret edilen düğüm
11        curr = curr->right;
12    }
13    return -1; // k geçersizse (buraya normalde ulaşılmaz)
14 }

```

Karmaşıklık

En fazla k düğüm işlenerek durulur: zaman $\mathcal{O}(h + k)$, alan $\mathcal{O}(h)$. Sık sorgu gerekiyorsa her düğümde alt ağaç boyutu tutularak $\mathcal{O}(h)$ sorgu elde edilebilir.

Soru 46

Bir BST'de iki düğümün en küçük ortak atasını (Lowest Common Ancestor, LCA) bulun.

Çözüm.

BST özelliği sayesinde LCA'yı kökten inerek buluruz. Kökten başlarız: eğer her iki değer de kökün değerinden büyükse LCA sağ alt ağaçtadır; her ikisi de küçükse sol alt ağaçtadır. Aksi halde (değerler kökün iki yanına düşüyor ya da biri köke eşitse) mevcut düğüm ayrışmanın (split) olduğu ilk noktadır, yani LCA'dır.

```

1 TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
2     TreeNode* curr = root;
3     while (curr != nullptr) {

```

```

4     if (p->val < curr->val && q->val < curr->val)
5         curr = curr->left;           // her ikisi de solda
6     else if (p->val > curr->val && q->val > curr->val)
7         curr = curr->right;          // her ikisi de sağda
8     else
9         return curr;                 // ayrışma noktası = LCA
10    }
11    return nullptr;
12 }

```

Karmaşıklık

Kökten aşağı tek bir yol izlenir: zaman $\mathcal{O}(h)$, ekstra alan $\mathcal{O}(1)$.

16.3 Dengeli Ağaçlar

Bilgi

Bir ikili ağaç yükseklik-dengeli (height-balanced) ise, *her* düğümde sol ve sağ alt ağaçların yükseklik farkı en fazla 1'dir. AVL ağaçları bu değişmezi ekleme/silme sırasında dönüşlerle (rotation) korur ve $\mathcal{O}(\log n)$ işlem garantisi verir.

Soru 47

Bir ikili ağacın yükseklik-dengeli olup olmadığını $\mathcal{O}(n)$ zamanda kontrol edin.

Çözüm.

Naif yaklaşım her düğümde yükseklik hesaplayıp fark kontrolü yapar ve $\mathcal{O}(n^2)$ olur. Bunun yerine *aşağıdan yukarı* (bottom-up) tek geçiş kullanırız: her düğüm için hem yüksekliği hesaplarız hem de alt ağacın dengeli olup olmadığını yukarı taşıyoruz. Bir alt ağaç dengesizse özel bir işaret (-2) döndürerek tüm hesabı erkenden dengesiz olarak sonlandırırız.

```

1 // dengeliyse yüksekliği, değilse -2 döndürür
2 int checkHeight(TreeNode* node) {
3     if (node == nullptr) return -1;           // boş ağaç yüksekliği -1
4     int lh = checkHeight(node->left);
5     if (lh == -2) return -2;                 // sol alt ağaç dengesiz
6     int rh = checkHeight(node->right);
7     if (rh == -2) return -2;                 // sağ alt ağaç dengesiz
8     if (std::abs(lh - rh) > 1) return -2;     // bu düğümde denge bozuk
9     return 1 + std::max(lh, rh);             // yüksekliği yukarı taşı
10 }
11
12 bool isBalanced(TreeNode* root) {
13     return checkHeight(root) != -2;
14 }

```

İpucu

Erken çıkış (-2 işareti) sayesinde bir dengesizlik bulunduğunda ağacın kalanı taranmaz; yine de en kötü durum karmaşıklığı $\mathcal{O}(n)$ 'dir.

Karmaşıklık

Her düğüm en fazla bir kez ziyaret edilir: zaman $\mathcal{O}(n)$, özyineleme yığını $\mathcal{O}(h)$ alan.

16.4 Heap ve Öncelik Kuyruğu

Bilgi

Öncelik kuyruğu (priority queue) tipik olarak bir ikili heap ile gerçekleştirilir. `std::priority_queue` varsayılan olarak *maks-heap*'tir (en büyük eleman tepede). Min-heap için `std::priority_queue<int, std::vector<int>, std::greater<int>>` kullanılır. Ekleme ve çıkarma $\mathcal{O}(\log n)$, tepeye bakma $\mathcal{O}(1)$ 'dir.

Soru 48

Bir tam sayı akışında (stream) k . en büyük elemanı sürekli olarak takip eden bir yapı tasarlayın. Her yeni eleman eklendiğinde güncel k . en büyük değeri döndürün.

Çözüm.

Boyutu en fazla k olan bir *min-heap* tutarız. Heap dolu değilse elemanı ekleriz. Heap doluysa ve yeni eleman heap'in tepesinden (şu ana kadarki en küçük "ilk k ") büyükse, tepeyi çıkarıp yeni elemanı ekleriz. Böylece heap her zaman en büyük k elemanı tutar ve tepesi tam olarak k . en büyük elemandır.

```

1 class KthLargest {
2     int k;
3     std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
4 public:
5     KthLargest(int k, std::vector<int>& nums) : k(k) {
6         for (int x : nums) add(x);
7     }
8     int add(int val) {
9         minHeap.push(val);
10        if ((int)minHeap.size() > k)
11            minHeap.pop();           // en küçüğü at, ilk k büyüğü tut
12        return minHeap.top();       // tepe = k. en büyük
13    }
14 };

```

Karmaşıklık

Her add çağrısı $\mathcal{O}(\log k)$, alan $\mathcal{O}(k)$. n elemanlık akış için toplam $\mathcal{O}(n \log k)$.

Soru 49

Bir tam sayı akışının medyanını her ekleme sonrası $\mathcal{O}(\log n)$ zamanda döndüren MedianFinder sınıfını iki heap ile tasarlayın.

Çözüm.

İki heap kullanırız: lo bir maks-heap (küçük yarının en büyüğü tepede), hi bir min-heap (büyük yarının en küçüğü tepede). Değişmezi koruruz: lo 'nun boyutu hi 'ye eşit ya da bir fazladır ve lo 'daki her eleman hi 'deki her elemandan küçük veya eşittir. Eleman sayısı tekse medyan lo 'nun tepesidir; çiftse iki tepenin ortalamasıdır.

```

1 class MedianFinder {
2     std::priority_queue<int> lo;           // maks-heap
3     std::priority_queue<int, std::vector<int>, std::greater<int>> hi; // min-heap
4 public:
5     void addNum(int num) {
6         lo.push(num);                     // önce küçük yarıya ekle
7         hi.push(lo.top());                // lo'nun en büyüğünü hi'ye taşı
8         lo.pop();                         // (böylece sıralama korunur)

```

```

9         if (hi.size() > lo.size()) { // boyut dengesini koru
10             lo.push(hi.top());
11             hi.pop();
12         }
13     }
14     double findMedian() {
15         if (lo.size() > hi.size()) return lo.top(); // tek sayıda eleman
16         return (lo.top() + hi.top()) / 2.0; // çift sayıda eleman
17     }
18 };

```

Karmaşıklık

addNum $\mathcal{O}(\log n)$, findMedian $\mathcal{O}(1)$, alan $\mathcal{O}(n)$.

Soru 50

k adet artan sıralı bağlı listeyi tek bir sıralı bağlı listede birleştirin. ListNode yapısını kullanın.

Çözüm.

Her listenin başındaki en küçük düğümleri bir min-heap'te tutarız. Heap'ten en küçük düğümü çıkarıp sonuç listesine ekler, o düğümün listedeki bir sonrakini heap'e koyarız. Heap boşalana kadar devam ederiz. Bu, her adımda tüm aktif listelerin başları arasından en küçüğü verimli seçmemizi sağlar.

```

1 struct ListNode {
2     int val;
3     ListNode* next;
4     ListNode(int x) : val(x), next(nullptr) {}
5 };
6
7 ListNode* mergeKLists(std::vector<ListNode*>& lists) {
8     // heap: (değer, düğüm) çiftlerini en küçük değere göre sıralar
9     auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
10    std::priority_queue<ListNode*, std::vector<ListNode*>, decltype(cmp)> pq(cmp);
11    for (ListNode* node : lists)
12        if (node) pq.push(node); // her listenin başını ekle
13
14    ListNode dummy(0); // yapay baş düğüm
15    ListNode* tail = &dummy;
16    while (!pq.empty()) {
17        ListNode* smallest = pq.top(); pq.pop();
18        tail->next = smallest; // en küçüğü sonuca ekle
19        tail = tail->next;
20        if (smallest->next) // o listeden bir sonrakini ekle
21            pq.push(smallest->next);
22    }
23    return dummy.next;
24 }

```

Karmaşıklık

Toplam N düğüm varsa her düğüm heap'e bir kez girip çıkar; heap boyutu en fazla k : zaman $\mathcal{O}(N \log k)$, alan $\mathcal{O}(k)$.

16.5 Trie

Bilgi

Trie (önek ağacı), her düğümün bir karakteri temsil ettiği ve kökten bir düğüme giden yolun bir öneki (prefix) kodladığı bir ağaçtır. Kelime ekleme, arama ve önek sorgusu, kelime uzunluğu L olmak üzere $\mathcal{O}(L)$ zamanda yapılır — alfabe boyutundan bağımsızdır.

Soru 51

Küçük harflerden oluşan kelimeler için `insert`, `search` ve `startsWith` işlemlerini destekleyen bir Trie yapısı gerçekleştirin.

Çözüm.

Her Trie düğümü, 26 küçük harf için çocuk işaretçileri ve o düğümde biten bir kelime olup olmadığını belirten bir `isEnd` bayrağı tutar. Ekleme sırasında karakter karakter ilerler, yoksa yeni düğüm oluştururuz. `search` kelimenin tamamının var olduğunu *ve* son düğümde `isEnd` olduğunu ister; `startsWith` ise yalnızca yolun var olmasını yeterli görür.

```

1 class Trie {
2     struct Node {
3         Node* children[26] = {nullptr};
4         bool isEnd = false;
5     };
6     Node* root;
7
8     Node* findNode(const std::string& s) {
9         Node* curr = root;
10        for (char c : s) {
11            int idx = c - 'a';
12            if (!curr->children[idx]) return nullptr; // yol yok
13            curr = curr->children[idx];
14        }
15        return curr;
16    }
17 public:
18     Trie() : root(new Node()) {}
19
20     void insert(const std::string& word) {
21         Node* curr = root;
22         for (char c : word) {
23             int idx = c - 'a';
24             if (!curr->children[idx])
25                 curr->children[idx] = new Node(); // eksik düğümü oluştur
26             curr = curr->children[idx];
27         }
28         curr->isEnd = true; // kelime sonunu işaretle
29     }
30     bool search(const std::string& word) {
31         Node* node = findNode(word);
32         return node != nullptr && node->isEnd; // tam kelime olmalı
33     }
34     bool startsWith(const std::string& prefix) {
35         return findNode(prefix) != nullptr; // önek yeterli
36     }
37 };

```

Karmaşıklık

Her işlem, kelime/önek uzunluğu L için $\mathcal{O}(L)$. Alan, toplam karakter sayısı ile orantılıdır: en kötü durumda $\mathcal{O}(\text{toplam karakter} \times 26)$.

Soru 52

addWord ve search destekleyen bir WordDictionary tasarlayın. Aramada . karakteri herhangi bir tek harfle eşleşebilsin (joker).

Çözüm.

Yapı yine bir Trie'dir; fark aramadır. Normal karakterde tek çocuğa ineriz. . jokerinde ise mevcut düğümün tüm çocukları üzerinden özyinelemeli olarak dener, herhangi biri kalan deseni eşliyorsa başarılı sayarız. Bu, jokerli aramayı bir geri izleme (backtracking) problemine dönüştürür.

```

1 class WordDictionary {
2     struct Node {
3         Node* children[26] = {nullptr};
4         bool isEnd = false;
5     };
6     Node* root;
7
8     bool dfs(const std::string& word, int i, Node* node) {
9         if (node == nullptr) return false;
10        if (i == (int)word.size()) return node->isEnd; // desen bitti
11        char c = word[i];
12        if (c == '.' ) { // joker: tüm çocukları dene
13            for (int j = 0; j < 26; ++j)
14                if (dfs(word, i + 1, node->children[j])) return true;
15            return false;
16        }
17        return dfs(word, i + 1, node->children[c - 'a']); // normal karakter
18    }
19 public:
20     WordDictionary() : root(new Node()) {}
21
22     void addWord(const std::string& word) {
23         Node* curr = root;
24         for (char c : word) {
25             int idx = c - 'a';
26             if (!curr->children[idx]) curr->children[idx] = new Node();
27             curr = curr->children[idx];
28         }
29         curr->isEnd = true;
30     }
31     bool search(const std::string& word) {
32         return dfs(word, 0, root);
33     }
34 };

```

Karmaşıklık

Ekleme $\mathcal{O}(L)$. Arama, jokersiz durumda $\mathcal{O}(L)$; her joker en kötü durumda 26 dala ayrılabilirdiği için jokerli durumda üst sınır $\mathcal{O}(26^L)$ olsa da pratikte çok daha hızlıdır.

Soru 53

Bir kelime dizisinin en uzun ortak önekini (longest common prefix) bulun. Ortak önek yoksa boş string döndürün.

Çözüm.

Bu problem için tam Trie kurmak şart değildir; dikey tarama (vertical scanning) daha basittir. İlk kelimenin karakterlerini birer birer alır, aynı konumdaki karakterin *tüm* kelimelerde aynı olup olmadığını kontrol ederiz. İlk uyuşmazlıkta veya bir kelime bittiğinde o ana kadar biriken önek cevaptır. (Kavramsal olarak bu, Trie’de dallanma olmayan tek zinciri izlemeye denktir.)

```
1 std::string longestCommonPrefix(std::vector<std::string>& strs) {
2     if (strs.empty()) return "";
3     for (int i = 0; i < (int)strs[0].size(); ++i) {
4         char c = strs[0][i];           // referans karakter
5         for (int j = 1; j < (int)strs.size(); ++j) {
6             // kelime bittiyse ya da karakter uyuşmuyorsa önek burada biter
7             if (i >= (int)strs[j].size() || strs[j][i] != c)
8                 return strs[0].substr(0, i);
9         }
10    }
11    return strs[0]; // ilk kelimenin tamamı ortak önek
12 }
```

Karmaşıklık

Tüm karakterlerin toplam sayısı S olmak üzere en kötü durumda zaman $\mathcal{O}(S)$, ekstra alan $\mathcal{O}(1)$.

16.6 Segment/Fenwick Ağacı**Bilgi**

Fenwick ağacı (Binary Indexed Tree, BIT) ve segment ağacı, bir dizi üzerinde *nokta güncelleme* ve *aralık sorgusunu* her ikisini de $\mathcal{O}(\log n)$ zamanda yapan yapılardır. Fenwick daha az kod ve bellekle önek toplamı gibi tersinir işlemler için idealdir; segment ağacı minimum, maksimum, gcd gibi daha genel birleştirmeleri destekler.

Soru 54

Fenwick ağacı (BIT) kullanarak nokta güncelleme ve önek toplamı (dolayısıyla aralık toplamı) sorgusunu destekleyen tam bir yapı yazın. Diziyi 1-tabanlı indeksleyin.

Çözüm.

Fenwick ağacında her indeks, en düşük anlamlı bitine ($i \& -i$) bağlı bir aralığın toplamını tutar. update işleminde indeksi $i \ += i \& -i$ ile büyütürük etkilenen tüm bloklara değeri ekleriz. prefixSum işleminde $i \ -= i \& -i$ ile küçülürük $[1, i]$ aralığının toplamını parçalardan toplarız. Aralık toplamı $[l, r]$, $\text{prefixSum}(r) - \text{prefixSum}(l-1)$ ile bulunur.

```
1 class Fenwick {
2     std::vector<long> tree; // 1-tabanlı; tree[0] kullanılmaz
3     int n;
4 public:
5     Fenwick(int size) : tree(size + 1, 0), n(size) {}
6
7     // i konumundaki değere delta ekle
8     void update(int i, long delta) {
9         for (; i <= n; i += i & (-i)) // etkilenen blokları gez
```

```

10         tree[i] += delta;
11     }
12     // [1, i] önek toplamı
13     long prefixSum(int i) {
14         long sum = 0;
15         for (; i > 0; i -= i & (-i))    // parça toplamalarını topla
16             sum += tree[i];
17         return sum;
18     }
19     // [l, r] aralık toplamı
20     long rangeSum(int l, int r) {
21         return prefixSum(r) - prefixSum(l - 1);
22     }
23 };

```

Dikkat

Fenwick ağacı 1-tabanlıdır; $i \& -i$ ifadesi $i = 0$ 'da sonsuz döngüye yol açacağından indeksler mutlaka 1'den başlamalıdır. Dış diziniz 0-tabanlıysa çağrılarda indekse 1 ekleyin.

Karmaşıklık

update ve prefixSum her ikisi de $\mathcal{O}(\log n)$, alan $\mathcal{O}(n)$. İlk kurulum n güncelleme ile $\mathcal{O}(n \log n)$.

Soru 55

Bir segment ağacı ile aralık minimum sorgusu (Range Minimum Query) ve nokta güncellemeyi destekleyen bir yapı yazın.

Çözüm.

Segment ağacını bir dizi üzerinde kurarız: her düğüm bir aralığın minimumunu tutar, çocukları o aralığın iki yarısını temsil eder. build özinelemeli olarak yaprakları dizideki değerlerle doldurur, iç düğümleri çocuklarının minimumu yapar. query sorgu aralığıyla düğüm aralığının kesişimine göre üç durumu ele alır: tamamen dışında (etkisiz eleman $+\infty$), tamamen içinde (düğüm değeri), kısmen örtüşme (iki çocuğu birleştirir). update ilgili yaprağı değiştirip kök yolundaki düğümleri günceller.

```

1  class SegmentTree {
2      std::vector<int> tree;    // 4*n boyutunda
3      int n;
4
5      void build(const std::vector<int>& a, int node, int lo, int hi) {
6          if (lo == hi) { tree[node] = a[lo]; return; } // yaprak
7          int mid = (lo + hi) / 2;
8          build(a, 2*node, lo, mid);
9          build(a, 2*node+1, mid+1, hi);
10         tree[node] = std::min(tree[2*node], tree[2*node+1]);
11     }
12     int query(int node, int lo, int hi, int l, int r) {
13         if (r < lo || hi < l) return INT_MAX; // tamamen dışında
14         if (l <= lo && hi <= r) return tree[node]; // tamamen içinde
15         int mid = (lo + hi) / 2; // kısmi örtüşme
16         return std::min(query(2*node, lo, mid, l, r),
17                         query(2*node+1, mid+1, hi, l, r));
18     }
19     void update(int node, int lo, int hi, int pos, int val) {
20         if (lo == hi) { tree[node] = val; return; } // yaprağı değiştir
21         int mid = (lo + hi) / 2;

```

```

22     if (pos <= mid) update(2*node, lo, mid, pos, val);
23     else update(2*node+1, mid+1, hi, pos, val);
24     tree[node] = std::min(tree[2*node], tree[2*node+1]); // yukarı güncelle
25 }
26 public:
27     SegmentTree(const std::vector<int>& a) : tree(4 * a.size()), n(a.size()) {
28         build(a, 1, 0, n - 1);
29     }
30     int queryMin(int l, int r) { return query(1, 0, n - 1, l, r); }
31     void pointUpdate(int pos, int val) { update(1, 0, n - 1, pos, val); }
32 };

```

Karmaşıklık

Kurulum $\mathcal{O}(n)$, her sorgu ve her güncelleme $\mathcal{O}(\log n)$, alan $\mathcal{O}(n)$ ($4n$ düğüm ayrılır).

16.7 Ağaç Serileştirme

Bilgi

Serileştirme (serialization), bir ağaç yapısını diske yazılabilecek ya da ağ üzerinden gönderilebilecek düz bir string'e çevirmektir. Deserileştirme, bu string'den özgün ağacı birebir geri kurar. İkili ağaçlar için önsıralı (preorder) dolaşıma boş çocuk işaretçilerini de eklemek en yaygın ve sağlam yöntemdir.

Soru 56

Bir ikili ağacı bir string'e çeviren serialize ve bu string'den ağacı geri kuran deserialize fonksiyonlarını yazın. Değerler arası ilişkiyi koruyun; boş ağaç da desteklensin.

Çözüm.

Önsıralı (kök, sol, sağ) dolaşım kullanırız ve boş çocukları özel bir işaretle (#) kodlarız. Boş işaretçilerin kaydedilmesi, ağacın yapısını tek başına preorder'ın belirleyebilmesini sağlar — bu yüzden ayrıca inorder'a gerek kalmaz. Değerleri boşlukla ayırırız. Geri kurarken token'ları aynı önsıralı düzende okuruz: bir # görürsek nullptr döndürür, aksi halde düğümü oluşturup önce sol sonra sağ alt ağacı özyinelemeli okuruz. Okuma sırası yazma sırasıyla birebir aynı olduğundan ağaç eksiksiz geri kurulur.

```

1 class Codec {
2     void serializeHelper(TreeNode* node, std::ostream& out) {
3         if (node == nullptr) { out << "# "; return; } // boş işaretçi
4         out << node->val << ' '; // kök
5         serializeHelper(node->left, out); // sol
6         serializeHelper(node->right, out); // sağ
7     }
8     TreeNode* deserializeHelper(std::istream& in) {
9         std::string token;
10        in >> token;
11        if (token == "#") return nullptr; // boş alt ağaç
12        TreeNode* node = new TreeNode(std::stoi(token));
13        node->left = deserializeHelper(in); // sol (aynı sıra)
14        node->right = deserializeHelper(in); // sağ
15        return node;
16    }
17 public:
18     std::string serialize(TreeNode* root) {
19         std::ostream out;

```

```
20     serializeHelper(root, out);
21     return out.str();
22 }
23 TreeNode* deserialize(const std::string& data) {
24     std::istringstream in(data);
25     return deserializeHelper(in);
26 }
27 };
```

İpucu

Preorder + boş işaretçi kombinasyonu ağacı tek başına benzersiz biçimde belirler. Buna karşılık boş işaretçi olmadan tek bir dolaşım (yalnızca preorder veya yalnızca inorder) ağacı belirlemeye yetmez; bu yüzden # işaretleri kritiktir.

Karmaşıklık

Hem `serialize` hem `deserialize` her düğümü (ve boş işaretçileri) bir kez işler: zaman $\mathcal{O}(n)$, üretilen string ve özyineleme yığını $\mathcal{O}(n)$ alan kullanır.

KISIM 4

Graflar

17. Graf Gösterimi

Graf, *düğüm*lerin (vertex) *kenar*larla (edge) bağlandığı bir yapıdır. Ağacın aksine graf döngü içerebilir ve bir düğümden diğerine birden çok yol olabilir. Sosyal ağlar, haritalar, bağımlılık çizgeleri ve daha fazlası graflarla modellenir. Kenarlar *yönlü* (directed) veya *yönsüz* (undirected), *ağırlıklı* (weighted) veya ağırlıksız olabilir.

17.1 Komşuluk Listesi vs Komşuluk Matrisi

Bir grafi iki temel biçimde saklarız. Seçim, grafın *yoğunluğuna* bağlıdır:

Gösterim	Bellek	Kenar var mı?	Ne zaman?
Komşuluk listesi	$\mathcal{O}(V + E)$	$\mathcal{O}(\text{derece})$	Seyrek graf (çoğu durum)
Komşuluk matrisi	$\mathcal{O}(V^2)$	$\mathcal{O}(1)$	Yoğun graf, hızlı kenar sorgusu

```

1 int V = 5; // dugum sayisi
2 // Komşuluk listesi (ağırlıksız)
3 vector<vector<int>> adj(V);
4 adj[0].push_back(1); adj[1].push_back(0); // yonsuz kenar 0-1
5
6 // Komşuluk listesi (ağırlıklı): {komsu, ağırlık}
7 vector<vector<pair<int,int>>> g(V);
8 g[0].push_back({1, 4}); g[1].push_back({0, 4}); // 0-1 arasi ağırlık 4
9
10 // Komşuluk matrisi
11 vector<vector<int>> mat(V, vector<int>(V, 0));
12 mat[0][1] = mat[1][0] = 1;

```

18. Graf Dolaşımı: BFS ve DFS

Bir grafi “dolaşmak”, bir başlangıç düğümünden erişilebilen tüm düğümleri ziyaret etmektir. İki temel strateji vardır ve pek çok graf algoritmasının temelini oluştururlar.

18.1 Genişlik-Öncelikli Arama (BFS)

BFS, başlangıçtan başlayarak düğümleri *katman katman* (mesafeye göre) gezer; bir kuyruk kullanır. Ağırlıksız graflarda **en kısa yolu** (en az kenarla) bulur.

```

1  vector<int> bfs(const vector<vector<int>>& adj, int start) {
2      int V = adj.size();
3      vector<int> dist(V, -1);           // -1 = ziyaret edilmedi
4      queue<int> q;
5      dist[start] = 0;
6      q.push(start);
7      while (!q.empty()) {
8          int u = q.front(); q.pop();
9          for (int v : adj[u]) {
10             if (dist[v] == -1) {        // ilk kez goruluyor
11                 dist[v] = dist[u] + 1;  // bir katman uzakta
12                 q.push(v);
13             }
14         }
15     }
16     return dist;                       // O(V + E)
17 }

```

18.2 Derinlik-Öncelikli Arama (DFS)

DFS, bir yolu *sonuna kadar* takip eder, tıkanınca geri döner (backtrack). Özyineleme (veya açık bir yığın) ile yapılır. Döngü tespiti, bağlı bileşenler ve topolojik sıralama DFS üzerine kuruludur.

```

1  void dfs(const vector<vector<int>>& adj, int u, vector<bool>& visited) {
2      visited[u] = true;
3      cout << u << " ";
4      for (int v : adj[u])
5          if (!visited[v])
6              dfs(adj, v, visited);      // O(V + E)
7  }
8
9  // Bagli bileşen sayısı (yonsuz graf)
10 int countComponents(const vector<vector<int>>& adj) {
11     int V = adj.size(), count = 0;
12     vector<bool> visited(V, false);
13     for (int i = 0; i < V; ++i)
14         if (!visited[i]) { dfs(adj, i, visited); ++count; }
15     return count;
16 }

```

Dikkat

Derin graflarda özyinelemeli DFS *yığın taşmasına* (stack overflow) yol açabilir ($V \sim 10^5$ ve zincir biçimli graf). Böyle durumlarda DFS'i açık bir `std::stack` ile yinelemeli (iterative) yaz.

19. En Kısa Yol Algoritmaları

Ağırlıklı graflarda “bir düğümden diğerine en düşük toplam ağırlıklı yol nedir?” sorusu temeldir. Farklı kısıtlar için farklı algoritmalar kullanılır.

Algoritma	Karmaşıklık	Negatif kenar?	Kapsam
BFS	$O(V + E)$	— (ağırlıksız)	Tek kaynak
Dijkstra	$O((V + E) \log V)$	Hayır	Tek kaynak
Bellman-Ford	$O(VE)$	Evet	Tek kaynak
Floyd-Warshall	$O(V^3)$	Evet	Tüm çiftler

19.1 Dijkstra Algoritması

Dijkstra, *negatif olmayan* ağırlıklı graflarda tek bir kaynaktan tüm düğümlere en kısa yolu bulur. Ağgözlü çalışır: her adımda henüz kesinleşmemiş en yakın düğümü seçer. Bir *min-heap* (öncelik kuyruğu) ile verimlidir.

```

1  vector<long long> dijkstra(const vector<vector<pair<int,int>>>& g, int source) {
2      int V = g.size();
3      const long long INF = 1e18;
4      vector<long long> dist(V, INF);
5      priority_queue<pair<long long,int>,
6                      vector<pair<long long,int>>,
7                      greater<>> pq;          // min-heap: {mesafe, dugum}
8      dist[source] = 0;
9      pq.push({0, source});
10     while (!pq.empty()) {
11         auto [d, u] = pq.top(); pq.pop();
12         if (d > dist[u]) continue;          // eski (bayat) kayıt, atla
13         for (auto [v, w] : g[u]) {
14             if (dist[u] + w < dist[v]) {      // gevsetme (relaxation)
15                 dist[v] = dist[u] + w;
16                 pq.push({dist[v], v});
17             }
18         }
19     }
20     return dist;
21 }
```

19.2 Bellman-Ford: Negatif Kenarlar

Bellman-Ford negatif kenarlarla çalışır ve **negatif döngü** olup olmadığını da tespit eder. Fikir: tüm kenarları $V - 1$ kez gevset; hâlâ bir gevsetme mümkünse negatif döngü vardır.

```

1  struct Edge { int u, v, w; };
2
3  bool bellmanFord(int V, const vector<Edge>& edges, int source,
4                  vector<long long>& dist) {
5      const long long INF = 1e18;
6      dist.assign(V, INF);
7      dist[source] = 0;
8      for (int i = 0; i < V - 1; ++i)        // V-1 tur
9          for (auto& e : edges)
10             if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v])
11                 dist[e.v] = dist[e.u] + e.w;
12     // V. tur: hala gevseme varsa negatif dongu
13     for (auto& e : edges)
14         if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v])
15             return false;                  // negatif dongu var
```

```

16     return true;
17 }

```

19.3 Floyd-Warshall: Tüm Çiftler

Floyd-Warshall, her düğüm çifti arasındaki en kısa yolu tek seferde bulur. Üç iç içe döngüyle her düğümü “ara nokta” olarak dener; kısacık ama güçlü bir dinamik programlama uygulamasıdır.

```

1 void floydWarshall(vector<vector<long long>>& d) {    // d: baslangic matris
2     int V = d.size();
3     for (int k = 0; k < V; ++k)                    // ara nokta
4         for (int i = 0; i < V; ++i)
5             for (int j = 0; j < V; ++j)
6                 if (d[i][k] + d[k][j] < d[i][j])
7                     d[i][j] = d[i][k] + d[k][j];
8 }

```

20. Minimum Kapsayan Ağaç (MST)

Ağırlıklı, bağlı ve yönsüz bir grapta, tüm düğümleri en düşük toplam ağırlıkla birbirine bağlayan (döngüsüz) kenar alt kümesine *minimum kapsayan ağaç* denir. Ağ tasarımı, kümeleme ve yaklaşık algoritmalar MST kullanır. İki klasik algoritma vardır; her ikisi de açgözlüdür.

20.1 Kruskal Algoritması (Union-Find ile)

Kruskal, kenarları ağırlığa göre sıralar ve döngü oluşturmeyen en ucuz kenarları sırayla ekler. Döngü kontrolü için *Birleşim-Bulma* (Disjoint Set Union) yapısını kullanır.

```

1 struct DSU {
2     vector<int> parent, rank_;
3     DSU(int n) : parent(n), rank_(n, 0) {
4         for (int i = 0; i < n; ++i) parent[i] = i;
5     }
6     int find(int x) {                    // yol sikistirma
7         return parent[x] == x ? x : parent[x] = find(parent[x]);
8     }
9     bool unite(int a, int b) {          // rank ile birlestirme
10        a = find(a); b = find(b);
11        if (a == b) return false;       // zaten ayni kume -> dongu
12        if (rank_[a] < rank_[b]) swap(a, b);
13        parent[b] = a;
14        if (rank_[a] == rank_[b]) ++rank_[a];
15        return true;
16    }
17 };
18
19 long long kruskal(int V, vector<tuple<int,int,int>> edges) {
20     sort(edges.begin(), edges.end());  // agirliga gore (ilk eleman)
21     DSU dsu(V);
22     long long sum = 0;
23     for (auto [w, u, v] : edges)
24         if (dsu.unite(u, v)) sum += w; // dongu olusturmuyorsa ekle
25     return sum;                        // O(E log E)
26 }

```

20.2 Prim Algoritması

Prim, tek bir düğümden başlar ve ağacı, her adımda ağaca bağlanan en ucuz kenarı ekleyerek büyütür. Dijkstra'ya çok benzer; bir min-heap kullanır.

```

1 long long prim(const vector<vector<pair<int,int>>>& g) {
2     int V = g.size();
3     vector<bool> inTree(V, false);
4     priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
5     pq.push({0, 0}); // {agirlik, dugum}
6     long long sum = 0;
7     int added = 0;
8     while (!pq.empty() && added < V) {
9         auto [w, u] = pq.top(); pq.pop();
10        if (inTree[u]) continue; // zaten agacta
11        inTree[u] = true;
12        sum += w;
13        ++added;
14        for (auto [v, weight] : g[u])
15            if (!inTree[v]) pq.push({weight, v});
16    }
17    return sum; // O(E log V)
18 }
```

21. Topolojik Sıralama

Yönlü ve döngüsüz bir grafta (DAG), her kenar $u \rightarrow v$ için u 'nın v 'den önce geldiği bir düğüm sıralamasına *topolojik sıralama* denir. Görev zamanlama, derleme bağımlılıkları ve ders ön-koşulları bunu kullanır. İki yöntem vardır.

21.1 Kahn Algoritması (BFS Tabanlı)

Giriş derecesi (in-degree) 0 olan düğümlerden başlarız; her birini sıraya koyup çıkardıkça komşularının derecesini azaltırız. Bu, aynı zamanda döngü tespiti de yapar: tüm düğümler işlenemezse graf döngülüdür.

```

1 vector<int> topoSort(const vector<vector<int>>& adj) {
2     int V = adj.size();
3     vector<int> inDegree(V, 0), result;
4     for (int u = 0; u < V; ++u)
5         for (int v : adj[u]) ++inDegree[v];
6
7     queue<int> q;
8     for (int u = 0; u < V; ++u)
9         if (inDegree[u] == 0) q.push(u);
10
11    while (!q.empty()) {
12        int u = q.front(); q.pop();
13        result.push_back(u);
14        for (int v : adj[u])
15            if (--inDegree[v] == 0) q.push(v);
16    }
17    if ((int)result.size() != V) return {}; // dongu var -> gecersiz
18    return result; // O(V + E)
19 }
```

Bilgi

Topolojik sıralama *yalnızca* yönlü döngüsüz graflar (DAG) için tanımlıdır. Sonuç genelde tek değildir: birden fazla geçerli sıralama olabilir. Kahn algoritmasının döndürdüğü boş vektör, grafta bir döngü olduğunu — yani sıralamanın imkânsız olduğunu — gösterir.

22. Graflar — Alıştırmalar ve Çözümler

22.1 BFS/DFS Uygulamaları

Bu bölümde graf gezinmesinin (traversal) klasik uygulamalarını inceliyoruz. Hem `grid` (ızgara) üzerinde hem de komşuluk listesi (adjacency list) ile temsil edilen graflar üzerinde çalışacağız. Grid'ler örtük (implicit) graflardır: her hücre bir düğüm, her hücrenin 4 (bazen 8) komşusu ise kenardır.

Soru 57

Bir `grid` verildiğinde ('1' kara, '0' deniz), yatay ve dikey olarak birbirine bağlı kara hücrelerinin oluşturduğu ada sayısını bulun. (Number of Islands)

Çözüm.

Her kara hücreğini bir düğüm gibi düşünürüz. Henüz ziyaret edilmemiş bir kara hücresi bulunduğumuzda yeni bir ada başlatır ve DFS/BFS ile o adaya ait tüm bağlı kara hücrelerini gezip işaretleriz. Böylece her ada tam bir kez sayılır. Ziyaret işaretini ayrı bir matris tutmak yerine, gezdiğimiz hücreyi '0' yaparak yerinde (in-place) yapıyoruz; bu bellek tasarrufu sağlar.

Bilgi

Bağlı bileşen (connected component) sayma mantığı burada gizlidir: her DFS çağrısı bir bileşeni tamamen tüketir, dıştaki döngü kaç kez yeni DFS başlattıysa o kadar bileşen (ada) vardır.

```

1  class Solution {
2  public:
3      int numIslands(vector<vector<char>>& grid) {
4          int rows = grid.size();
5          if (rows == 0) return 0;
6          int cols = grid[0].size();
7          int count = 0;
8
9          for (int r = 0; r < rows; ++r) {
10             for (int c = 0; c < cols; ++c) {
11                 if (grid[r][c] == '1') {
12                     ++count;           // yeni ada bulundu
13                     dfs(grid, r, c);   // bu adaya ait her şeyi bat
14                 }
15             }
16         }
17         return count;
18     }
19
20 private:
21     void dfs(vector<vector<char>>& grid, int r, int c) {
22         int rows = grid.size(), cols = grid[0].size();
23         // sınır dışı veya deniz ise dur
24         if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != '1')
25             return;
26     }

```

```

27     grid[r][c] = '0';           // ziyaret edildi olarak isaretle (batir)
28     dfs(grid, r + 1, c);       // asagi
29     dfs(grid, r - 1, c);       // yukari
30     dfs(grid, r, c + 1);       // saga
31     dfs(grid, r, c - 1);       // sola
32 }
33 };

```

Karmaşıklık: her hücre en fazla bir kez ziyaret edilir, dolayısıyla zaman $\mathcal{O}(R \cdot C)$, bellek en kötü durumda (tüm grid kara) özyineleme yığını için $\mathcal{O}(R \cdot C)$.

Dikkat

Çok büyük ızgaralarda özyinelemeli DFS yığın taşmasına (stack overflow) yol açabilir. Bu durumda açık bir stack veya queue ile iteratif (döngüsel) BFS/DFS tercih edin.

Soru 58

Bir grafin kenarları verilmiş; n düğüm ve edges listesi var. Grafin kaç bağlı bileşeni (connected component) olduğunu iteratif BFS ile bulun.

Çözüm.

Komşuluk listesini kurar, tüm düğümleri dolaşırız. Ziyaret edilmemiş her düğüm yeni bir bileşendir; o düğümden başlayan BFS o bileşenin tamamını ziyaret eder. Bir önceki soruyla aynı fikir, yalnızca temsil ızgara değil komşuluk listesidir.

```

1  int countComponents(int n, vector<vector<int>>& edges) {
2      vector<vector<int>> adj(n);
3      for (auto& e : edges) {
4          adj[e[0]].push_back(e[1]);
5          adj[e[1]].push_back(e[0]); // yonsuz graf: cift yonlu ekle
6      }
7
8      vector<bool> visited(n, false);
9      int components = 0;
10
11     for (int start = 0; start < n; ++start) {
12         if (visited[start]) continue;
13         ++components; // yeni bileşen
14         queue<int> q;
15         q.push(start);
16         visited[start] = true;
17         while (!q.empty()) {
18             int node = q.front(); q.pop();
19             for (int next : adj[node]) {
20                 if (!visited[next]) {
21                     visited[next] = true;
22                     q.push(next);
23                 }
24             }
25         }
26     }
27     return components;
28 }

```

Karmaşıklık: her düğüm ve kenar bir kez işlenir, zaman $\mathcal{O}(V + E)$, bellek $\mathcal{O}(V + E)$ (komşuluk listesi + kuyruk + ziyaret dizisi).

Soru 59

Yönsüz bir grafin iki parçalı (bipartite) olup olmadığını belirleyin: düğümler iki renge boyanabilir mi, öyle ki her kenar farklı renkli iki düğümü bağlasın?

Çözüm.

BFS ile renklendirme yaparız. Başlangıç düğümüne 0 rengini veririz; her komşuya karşıt rengi ($1 - \text{color}$) atarız. Eğer bir komşu zaten boyanmışsa ve rengi mevcut düğümle aynıysa, iki parçalı değildir (aynı renkte bir kenar bulundu). Graf bağlantısız olabileceğinden her bileşen için ayrı başlatırız.

Sezgi

İki parçalılık, grafin tek uzunlukta (odd-length) döngü içermemesiyle eşdeğerdir. Renklendirme ilişkisi tam olarak böyle bir tek döngüyü yakalar.

```

1 bool isBipartite(vector<vector<int>>& adj) {
2     int n = adj.size();
3     vector<int> color(n, -1);    // -1: henuz boyanmadi
4
5     for (int start = 0; start < n; ++start) {
6         if (color[start] != -1) continue;
7         color[start] = 0;
8         queue<int> q;
9         q.push(start);
10        while (!q.empty()) {
11            int node = q.front(); q.pop();
12            for (int next : adj[node]) {
13                if (color[next] == -1) {
14                    color[next] = 1 - color[node];    // karsit renk
15                    q.push(next);
16                } else if (color[next] == color[node]) {
17                    return false;    // ayni renkli kenar: iki parcali degil
18                }
19            }
20        }
21    }
22    return true;
23 }
```

Karmaşıklık: zaman $\mathcal{O}(V + E)$, bellek $\mathcal{O}(V)$.

Soru 60

Bir grid'de 0 geçilebilir, 1 engel. Sol üst köşeden (0,0) sağ alt köşeye 4 yönlü hareketle en kısa yol uzunluğunu (kaç hücre) bulun; yoksa -1 döndürün.

Çözüm.

Kenarların ağırlığı eşit (her adım maliyet 1) olduğundan en kısa yol için BFS yeterlidir. BFS düğümleri uzaklık sırasına göre keşfeder; hedefe ilk ulaştığımız anda bulduğumuz uzaklık en kısadır. dist matrisi hem ziyaret işareti hem de mesafe tutar.

İpucu

Ağırlıksız (veya tüm ağırlıkları eşit) graflarda en kısa yol her zaman BFS ile bulunur; Dijkstra'ya gerek yoktur ve BFS daha hızlıdır.

```

1 int shortestPath(vector<vector<int>>& grid) {
2     int n = grid.size();
3     if (n == 0 || grid[0][0] == 1 || grid[n-1][n-1] == 1) return -1;
```

```

4
5     vector<vector<int>> dist(n, vector<int>(n, -1));
6     queue<pair<int,int>> q;
7     q.push({0, 0});
8     dist[0][0] = 1;    // baslangic hucresi de sayilir
9
10    int dr[] = {1, -1, 0, 0};
11    int dc[] = {0, 0, 1, -1};
12
13    while (!q.empty()) {
14        auto [r, c] = q.front(); q.pop();
15        if (r == n - 1 && c == n - 1) return dist[r][c];
16        for (int k = 0; k < 4; ++k) {
17            int nr = r + dr[k], nc = c + dc[k];
18            if (nr >= 0 && nr < n && nc >= 0 && nc < n
19                && grid[nr][nc] == 0 && dist[nr][nc] == -1) {
20                dist[nr][nc] = dist[r][c] + 1;
21                q.push({nr, nc});
22            }
23        }
24    }
25    return -1;    // hedefe ulasilamadi
26 }

```

Karmaşıklık: zaman $\mathcal{O}(R \cdot C)$, bellek $\mathcal{O}(R \cdot C)$.

22.2 Döngü Tespiti ve Topolojik Sıralama

Yönlü asiklik graflar (DAG — Directed Acyclic Graph) bağımlılık modellemenin temelidir: dersler, derleme sırası, iş zamanlaması. Topolojik sıralama, her kenar $u \rightarrow v$ için u 'nın v 'den önce geldiği bir düğüm dizilimidir. Böyle bir sıralama ancak ve ancak grafta yönlü döngü yoksa vardır.

Soru 61

Yönlü bir grafta döngü olup olmadığını DFS ile üç-renk yöntemiyle tespit edin.

Çözüm.

Her düğümü üç durumdan biriyle işaretleriz: 0 beyaz (hiç ziyaret edilmedi), 1 gri (işleniyor, özyineleme yığımında), 2 siyah (tamamen bitti). DFS sırasında gri bir düğüme geri kenar (back edge) bulursak döngü vardır. Yalnızca siyah bir düğüme rastlamak döngü değildir; o dal zaten güvenle tamamlanmıştır.

Dikkat

Yönlü graflarda döngü tespiti için tek bir visited bayrağı yetmez; düğümün hâlâ işlenmekte olduğunu (gri) ayrıca ayırt etmelisiniz. Yönsüz graflarda ise ebeveyni (parent) hariç tutmak gerekir.

```

1 class CycleDetector {
2 public:
3     bool hasCycle(int n, vector<vector<int>>& adj) {
4         vector<int> state(n, 0);    // 0 beyaz, 1 gri, 2 siyah
5         for (int i = 0; i < n; ++i)
6             if (state[i] == 0 && dfs(i, adj, state))
7                 return true;
8         return false;
9     }
10
11 private:
12     bool dfs(int node, vector<vector<int>>& adj, vector<int>& state) {

```

```

13     state[node] = 1;    // gri: yigina girdi
14     for (int next : adj[node]) {
15         if (state[next] == 1) return true;    // geri kenar: dongu
16         if (state[next] == 0 && dfs(next, adj, state)) return true;
17     }
18     state[node] = 2;    // siyah: tamamlandi
19     return false;
20 }
21 };

```

Karmaşıklık: zaman $\mathcal{O}(V + E)$, bellek $\mathcal{O}(V)$ (durum dizisi + özyineleme yığını).

Soru 62

numCourses ders ve prerequisites önkoşul çiftleri ([a, b]: a için önce b alınmalı) verildi. Tüm dersleri tamamlamak mümkün müdür? (Course Schedule)

Çözüm.

Bu, ders bağımlılık grafinde yönlü döngü olup olmadığı sorusudur: döngü varsa (A, B'yi B de A'yı gerektirir gibi) tamamlamak imkânsızdır. Kahn algoritmasını (BFS tabanlı topolojik sıralama) kullanıyoruz: giriş derecesi (in-degree) sıfır olan düğümlerden başlarız, her işlediğimizde komşuların giriş derecesini azaltırız. Sonunda işlenen düğüm sayısı toplamdan azsa graf bir döngü içerir.

Bilgi

Kahn algoritmasının güzel yanı, aynı anda hem döngü tespiti yapması hem de döngü yoksa geçerli bir topolojik sıra üretmesidir.

```

1 bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
2     vector<vector<int>> adj(numCourses);
3     vector<int> indegree(numCourses, 0);
4
5     for (auto& p : prerequisites) {
6         // b -> a: once b, sonra a
7         adj[p[1]].push_back(p[0]);
8         ++indegree[p[0]];
9     }
10
11     queue<int> q;
12     for (int i = 0; i < numCourses; ++i)
13         if (indegree[i] == 0) q.push(i);    // onkosulsuz dersler
14
15     int processed = 0;
16     while (!q.empty()) {
17         int node = q.front(); q.pop();
18         ++processed;
19         for (int next : adj[node]) {
20             if (--indegree[next] == 0)    // son onkosul da karsilandi
21                 q.push(next);
22         }
23     }
24     return processed == numCourses;    // hepsi islendiye dongu yok
25 }

```

Karmaşıklık: zaman $\mathcal{O}(V + E)$, bellek $\mathcal{O}(V + E)$.

Soru 63

Bir DAG için geçerli bir topolojik sıralama üretin; graf döngü içeriyorsa boş bir liste döndürün.

Çözüm.

Yine Kahn algoritmasını kullanırız, ancak bu kez işlenen düğümleri sırayla bir sonuç vektörüne ekleriz. Kuyruktan çıkardığımız sıra, geçerli bir topolojik dizilim verir. Sonunda listenin boyutu düğüm sayısından küçükse döngü vardır ve boş liste döndürürüz.

```

1  vector<int> topologicalSort(int n, vector<vector<int>>& adj) {
2      vector<int> indegree(n, 0);
3      for (int u = 0; u < n; ++u)
4          for (int v : adj[u])
5              ++indegree[v];
6
7      queue<int> q;
8      for (int i = 0; i < n; ++i)
9          if (indegree[i] == 0) q.push(i);
10
11     vector<int> order;
12     while (!q.empty()) {
13         int node = q.front(); q.pop();
14         order.push_back(node);          // sonuca ekle
15         for (int next : adj[node])
16             if (--indegree[next] == 0)
17                 q.push(next);
18     }
19
20     if ((int)order.size() != n)
21         return {};    // dongu var: gecerli siralama yok
22     return order;
23 }
```

Karmaşıklık: zaman $\mathcal{O}(V + E)$, bellek $\mathcal{O}(V)$.

Alternatif

Topolojik sıra DFS ile de üretilebilir: her düğümün DFS'i bittiğinde onu bir yığına itersiniz; sonunda yığını ters çevirmek topolojik sırayı verir. Kahn yöntemi ise döngüyü daha doğrudan raporlar.

22.3 En Kısa Yol

Ağırlıklı graflarda en kısa yol için BFS yetmez; kenar maliyetleri farklı olduğunda Dijkstra algoritması gerekir. Ağırlıklar negatif olmadığı sürece Dijkstra en kısa yolları doğru bulur.

Soru 64

Negatif olmayan ağırlıklı yönlü bir grafta, source düğümünden diğer tüm düğümlere en kısa mesafeleri Dijkstra ile bulun.

Çözüm.

Dijkstra, açgözlü (greedy) bir yaklaşımla her adımda henüz kesinleşmemiş en küçük mesafeli düğümü seçer ve komşularını gevşetir (relax). Bu en küçük seçimi verimli yapmak için minimum öbeği (priority_queue) kullanırız. Bir düğümü ilk kez kuyruktan minimum mesafeyle çıkardığımızda o mesafe kesinleşmiştir. Eski (bayat) girişleri, saklanan mesafe güncel dist değerinden büyükse atlayarak eleriz.

Bilgi

priority_queue varsayılan olarak maksimum öbektir. Minimum öbek için greater<> karşılaştırıcısı kullanılır: priority_queue<T, vector<T>, greater<T>>.

```

1  vector<long long> dijkstra(int n, vector<vector<pair<int,int>>>& adj, int source) {
2      const long long INF = LLONG_MAX;
3      vector<long long> dist(n, INF);
4      dist[source] = 0;
5
6      // (mesafe, dugum) çiftleri; en küçük mesafe tepede
7      using State = pair<long long, int>;
8      priority_queue<State, vector<State>, greater<State>> pq;
9      pq.push({0, source});
10
11     while (!pq.empty()) {
12         auto [d, u] = pq.top(); pq.pop();
13         if (d > dist[u]) continue; // bayat giris: atla
14
15         for (auto& [v, w] : adj[u]) {
16             long long nd = d + w;
17             if (nd < dist[v]) { // gevsetme (relaxation)
18                 dist[v] = nd;
19                 pq.push({nd, v});
20             }
21         }
22     }
23     return dist;
24 }

```

Karmaşıklık: öbek tabanlı Dijkstra ile zaman $\mathcal{O}((V + E) \log V)$, bellek $\mathcal{O}(V + E)$. Negatif kenar varsa Dijkstra yanlış sonuç verir; o durumda Bellman-Ford ($\mathcal{O}(V \cdot E)$) gerekir.

Soru 65

beginWord, endWord ve bir wordList verildi. Her adımda tek bir harf değiştirerek (ve her ara kelime listede olmalı) beginWord'den endWord'e ulaşan en kısa dönüşüm dizisinin uzunluğunu bulun. (Word Ladder)

Çözüm.

Kelimeleri düğüm, tek harf farkla ayrıran kelime çiftlerini kenar kabul eden örtük bir graf kurarız. Tüm kenarların maliyeti 1 olduğundan en kısa yol BFS ile bulunur. Her kelimenin komşularını bulmak için her konumdaki harfi 'a'-'z' ile değiştirir, oluşan kelime listede (set) varsa komşu sayarız. Ziyaret edilen kelimeleri kümeden silerek tekrar gezmeyi önleriz.

```

1  int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
2      unordered_set<string> dict(wordList.begin(), wordList.end());
3      if (dict.find(endWord) == dict.end()) return 0;
4
5      queue<string> q;
6      q.push(beginWord);
7      int steps = 1; // baslangic kelimesi de sayilir
8
9      while (!q.empty()) {
10         int levelSize = q.size();
11         for (int i = 0; i < levelSize; ++i) { // bir seviye (uzaklik) bat
12             string word = q.front(); q.pop();
13             if (word == endWord) return steps;
14
15             for (int pos = 0; pos < (int)word.size(); ++pos) {
16                 char original = word[pos];
17                 for (char c = 'a'; c <= 'z'; ++c) {
18                     word[pos] = c;

```

```

19         if (dict.find(word) != dict.end()) {
20             dict.erase(word);    // ziyaret edildi: tekrar bakma
21             q.push(word);
22         }
23     }
24     word[pos] = original;    // kelimeyi geri yukle
25 }
26 }
27 ++steps;
28 }
29 return 0;    // ulasilamadi
30 }

```

Karmaşıklık: N kelime ve L kelime uzunluğu için zaman $\mathcal{O}(N \cdot L^2)$ (her kelime için L konum $\times 26$ deneme, her deneme L maliyetli karma), bellek $\mathcal{O}(N \cdot L)$.

Soru 66

Kenar ağırlıkları yalnızca 0 veya 1 olan bir grafta source'tan en kısa yolu, priority_queue yerine deque ile $\mathcal{O}(V + E)$ zamanda bulun. (0-1 BFS) Yaklaşımı açıklayın.

Çözüm.

Tüm ağırlıklar 0 veya 1 olduğunda Dijkstra'nın öbeğine gerek kalmaz. Çift uçlu kuyruk (deque) kullanırız: bir kenarı gevşetirken ağırlık 0 ise düğümü kuyruğun *önüne*, 1 ise *arkasına* ekleriz. Böylece kuyruk her zaman mesafeye göre (en fazla iki farklı değer) sıralı kalır ve öbeğin $\mathcal{O}(\log V)$ çarpanından kurtuluruz.

İpucu

0-1 BFS mantığı, ağırlıkların 0..k aralığında olduğu durumlara "dial's algorithm" ile genelleşir. Ancak temel fikir aynıdır: küçük ağırlığı öne, büyüğü arkaya koy.

```

1  vector<int> zeroOneBFS(int n, vector<vector<pair<int,int>>>& adj, int source) {
2      const int INF = INT_MAX;
3      vector<int> dist(n, INF);
4      dist[source] = 0;
5
6      deque<int> dq;
7      dq.push_front(source);
8
9      while (!dq.empty()) {
10         int u = dq.front(); dq.pop_front();
11         for (auto& [v, w] : adj[u]) {    // w yalnızca 0 veya 1
12             if (dist[u] + w < dist[v]) {
13                 dist[v] = dist[u] + w;
14                 if (w == 0) dq.push_front(v);    // 0 kenar: one
15                 else       dq.push_back(v);     // 1 kenar: arkaya
16             }
17         }
18     }
19     return dist;
20 }

```

Karmaşıklık: zaman $\mathcal{O}(V + E)$, bellek $\mathcal{O}(V)$. Klasik Dijkstra'ya göre $\log$ çarpanı yoktur.

22.4 Birleşim-Bulma (DSU)

Birleşim-Bulma (Union-Find / Disjoint Set Union — DSU) veri yapısı, ayrık kümeler üzerinde iki işlemi neredeyse sabit zamanda yapar: find (bir elemanın hangi kümede olduğunu bul) ve union (iki

kümeyle birleştirir). İki optimizasyon kritiktir: yol sıkıştırma (path compression) ve rütbe/boyuta göre birleştirme (union by rank/size). İkisi birlikte işlem başına ters-Ackermann $\mathcal{O}(\alpha(n))$ verir; pratikte sabit sayılır.

Temel Yapı

Aşağıdaki üç problemde de aynı DSU sınıfını kullanacağız. Önce sınıfı bir kez tanımlayıp sonra uygulamalarına bakalım.

```

1 class DSU {
2 public:
3     DSU(int n) : parent(n), rank_(n, 0), count(n) {
4         for (int i = 0; i < n; ++i) parent[i] = i;    // her eleman kendi kokunu gösterir
5     }
6
7     int find(int x) {
8         if (parent[x] != x)
9             parent[x] = find(parent[x]);    // yol sıkıştırma
10        return parent[x];
11    }
12
13    bool unite(int a, int b) {
14        int ra = find(a), rb = find(b);
15        if (ra == rb) return false;    // zaten aynı kumede
16        if (rank_[ra] < rank_[rb]) swap(ra, rb);
17        parent[rb] = ra;    // rütbeyle göre birleştir
18        if (rank_[ra] == rank_[rb]) ++rank_[ra];
19        --count;    // kume sayısı bir azaldı
20        return true;
21    }
22
23    int componentCount() const { return count; }
24
25 private:
26     vector<int> parent;
27     vector<int> rank_;
28     int count;    // ayrık kume sayısı
29 };

```

Soru 67

n düğüm ve edges verildi. Bağlı bileşen sayısını DSU ile bulun.

Çözüm.

Başlangıçta her düğüm kendi kümesindedir, yani n bileşen vardır. Her kenar için iki uç düğümü unite ederiz; farklı kümelerdeyseler birleşir ve sayaç azalır. Tüm kenarlar işlendiğinde componentCount bağlı bileşen sayısını verir.

```

1 int countComponentsDSU(int n, vector<vector<int>>& edges) {
2     DSU dsu(n);
3     for (auto& e : edges)
4         dsu.unite(e[0], e[1]);
5     return dsu.componentCount();
6 }

```

Karmaşıklık: E kenar için zaman $\mathcal{O}(E \cdot \alpha(n))$, bellek $\mathcal{O}(n)$.

Soru 68

Yönsüz bir grafta döngü olup olmadığını DSU ile tespit edin.

Çözüm.

Kenarları tek tek işleriz. Bir kenarın iki ucu unite edilmeden önce zaten aynı kümedeyse (find kökleri eşitse), bu kenar zaten bağlı iki düğümü yeniden birleştiriyor demektir; yani bir döngü oluşur. unite bu durumda false döndürdüğü için tespiti tek satırda yaparız.

Dikkat

Bu yöntem yönsüz graflar içindir. Yönlü graflarda döngü tespiti için DSU uygun değildir; oradaki üç-renk DFS veya Kahn yöntemini kullanın.

```
1 bool hasCycleUndirected(int n, vector<vector<int>>& edges) {
2     DSU dsu(n);
3     for (auto& e : edges) {
4         if (!dsu.unite(e[0], e[1]))
5             return true;    // uclar zaten bagli: dongu
6     }
7     return false;
8 }
```

Karmaşıklık: zaman $\mathcal{O}(E \cdot \alpha(n))$, bellek $\mathcal{O}(n)$.

Soru 69

accounts listesi verildi; her eleman [isim, email1, email2, ...] biçimindedir. Ortak en az bir e-postayı paylaşan hesaplar aynı kişiye aittir (isimler aynıdır). Hesapları birleştirip her kişinin sıralı e-posta listesini üretin. (Accounts Merge)

Çözüm.

Her hesabı bir düğüm sayarız. Aynı e-postayı içeren hesapları DSU ile birleştiririz: her e-postayı ilk gördüğümüz hesaba bağlarız (emailToAccount eşlemesi); aynı e-posta başka bir hesapta tekrar görünürse iki hesabı unite ederiz. Sonra her e-postayı ait olduğu hesabın kökü altında toplar, her kök için e-postaları sıralayıp isimle birlikte çıkttı veririz.

Bilgi

DSU'nun gücü tam burada görülür: "geçişli birleşme" (A ile B ortak posta, B ile C ortak posta $\Rightarrow$ A, B, C aynı kişi) doğal olarak ele alınır; DSU geçişliliği (transitivity) ücretsiz sağlar.

```
1 vector<vector<string>> accountsMerge(vector<vector<string>>& accounts) {
2     int n = accounts.size();
3     DSU dsu(n);
4     unordered_map<string, int> emailToAccount;    // email -> ilk hesap indeksi
5
6     for (int i = 0; i < n; ++i) {
7         for (int j = 1; j < (int)accounts[i].size(); ++j) {
8             const string& email = accounts[i][j];
9             auto it = emailToAccount.find(email);
10            if (it == emailToAccount.end())
11                emailToAccount[email] = i;
12            else
13                dsu.unite(i, it->second);    // ayni email: hesaplari birlestir
14        }
15    }
16
17    // her kok icin e-postalari topla (siralilik icin set kullan)
```

```

18 unordered_map<int, set<string>> merged;
19 for (auto& [email, idx] : emailToAccount)
20     merged[dsu.find(idx)].insert(email);
21
22 vector<vector<string>> result;
23 for (auto& [root, emails] : merged) {
24     vector<string> entry;
25     entry.push_back(accounts[root][0]); // isim
26     entry.insert(entry.end(), emails.begin(), emails.end());
27     result.push_back(entry);
28 }
29 return result;
30 }

```

Karmaşıklık: toplam e-posta sayısı K için zaman $\mathcal{O}(K \log K)$ (sıralama baskın), bellek $\mathcal{O}(K)$.

22.5 Güçlü Bağlı Bileşenler (SCC)

Yönlü bir grafta güçlü bağlı bileşen (Strongly Connected Component — SCC), içindeki her düğüm çiftinin karşılıklı olarak birbirine ulaşabildiği maksimal düğüm kümesidir. SCC'leri tek bir düğüme büzersek, geriye her zaman bir DAG (yönlü asiklik graf) kalır; bu "yoğunlaştırma" (condensation) birçok problemi basitleştirir.

Kosaraju algoritması iki DFS geçişiyle SCC'leri bulur:

Adım 1 Orijinal grafta DFS yap; her düğümü *bitiş sırasına* göre bir yığına it.

Adım 2 Grafın transpozunu (tüm kenarları ters çevir) oluştur.

Adım 3 Yığından düğümleri çıkararak transpoz grafta DFS yap; her yeni DFS bir SCC verir.

Neden Çalışır?

İlk DFS'in *bitiş* sıraları, SCC'leri yoğunlaştırma DAG'ında topolojik sıraya dizer. Kenarları ters çevirdiğimizde, en geç biten SCC'den başlayan DFS o SCC'nin dışına çıkamaz — çünkü ters grafta o SCC'ye giden kenarlar artık başka SCC'lere gitmez. Böylece her DFS tam bir SCC toplar.

Soru 70

Yönlü bir graf verildiğinde tüm güçlü bağlı bileşenleri Kosaraju algoritmasıyla bulun; her SCC'yi ayrı bir düğüm listesi olarak döndürün.

Çözüm.

Üç adımı doğrudan uygularız. İlk DFS *bitiş* sırasına göre bir *order* yığını doldurur. Transpoz grafi *adjT* olarak kurarız. Son olarak *order*'ı tersten (en geç biten önce) gezerek transpoz grafta DFS yaparız; her başlatılan DFS bir SCC toplar.

```

1 class Kosaraju {
2 public:
3     vector<vector<int>> findSCCs(int n, vector<vector<int>>& adj) {
4         vector<bool> visited(n, false);
5         vector<int> order; // bitis sirasi yigini
6
7         // 1. adim: orijinal grafta DFS, bitis sirasini biriktir
8         for (int i = 0; i < n; ++i)
9             if (!visited[i])
10                 dfs1(i, adj, visited, order);
11
12         // 2. adim: transpoz grafi kur
13         vector<vector<int>> adjT(n);
14         for (int u = 0; u < n; ++u)

```

```

15     for (int v : adj[u])
16         adjT[v].push_back(u);    // kenarı ters çevir
17
18     // 3. adım: bitis sirasının tersinden transpozda DFS
19     fill(visited.begin(), visited.end(), false);
20     vector<vector<int>> sccs;
21     for (int i = (int)order.size() - 1; i >= 0; --i) {
22         int node = order[i];
23         if (!visited[node]) {
24             vector<int> component;
25             dfs2(node, adjT, visited, component);
26             sccs.push_back(component);
27         }
28     }
29     return sccs;
30 }
31
32 private:
33 void dfs1(int u, vector<vector<int>>& adj,
34           vector<bool>& visited, vector<int>& order) {
35     visited[u] = true;
36     for (int v : adj[u])
37         if (!visited[v])
38             dfs1(v, adj, visited, order);
39     order.push_back(u);    // dugum bitti: yigina it
40 }
41
42 void dfs2(int u, vector<vector<int>>& adjT,
43           vector<bool>& visited, vector<int>& component) {
44     visited[u] = true;
45     component.push_back(u);
46     for (int v : adjT[u])
47         if (!visited[v])
48             dfs2(v, adjT, visited, component);
49 }
50 };

```

Karmaşıklık: iki DFS ve bir transpoz kurulumu, her biri $\mathcal{O}(V + E)$; toplam zaman $\mathcal{O}(V + E)$, bellek $\mathcal{O}(V + E)$ (transpoz graf + yığın).

Bilgi

Kosaraju'ya alternatif Tarjan algoritmasıdır; o tek DFS geçişiyle (low-link değerleri ile) SCC bulur ve transpoz gerektirmez. Kosaraju kavramsal olarak daha anlaşılır, Tarjan pratikte biraz daha hızlıdır. İkisi de $\mathcal{O}(V + E)$ 'dir.

KISIM 5

Algoritma Teknikleri

23. Sıralama Algoritmaları

Sıralama, verilerin belirli bir düzene sokulmasıdır ve pek çok başka algoritmanın (ikili arama, birçok ağgözlü yöntem) ön koşuludur. Farklı sıralama algoritmaları farklı ödünleşmeler sunar. Anahtar kavramlar: *kararlılık* (stable — eşit elemanların görelî sırası korunur mu?) ve *yerinde* (in-place — ek bellek gerekir mi?).

Algoritma	Ort.	En kötü	Bellek	Kararlı?
Bubble	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Evet
Selection	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Hayır
Insertion	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Evet
Merge	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Evet
Quick	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	Hayır
Heap	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	Hayır
Counting	$\mathcal{O}(n + k)$	$\mathcal{O}(n + k)$	$\mathcal{O}(k)$	Evet

23.1 Basit Sıralamalar: Insertion Sort

Ekleme sıralaması, kart oyununda elini sıralaman gibi çalışır: her elemanı alıp solundaki sıralı kısımda doğru yerine yerleştirir. Küçük veya neredeyse sıralı diziler için pratikte hızlıdır.

```

1 void insertionSort(vector<int>& a) {
2     for (int i = 1; i < (int)a.size(); ++i) {
3         int key = a[i], j = i - 1;
4         while (j >= 0 && a[j] > key) {      // büyükleri saga kaydir
5             a[j + 1] = a[j];
6             --j;
7         }
8         a[j + 1] = key;                      // dogru yere koy
9     }
10 }
```

23.2 Merge Sort: Böl ve Birleştir

Birleştirme sıralaması bir *böl-yönet* algoritmasıdır: diziyi ikiye böler, her yarıyı özyinelemeli sıralar, sonra iki sıralı yarıyı birleştirir. $\mathcal{O}(n \log n)$ garantisi verir ve kararlıdır.

```

1 void merge(vector<int>& a, int left, int mid, int right) {
2     vector<int> temp;
3     int i = left, j = mid + 1;
4     while (i <= mid && j <= right) // iki yariyi sirali birlestir
5         temp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
6     while (i <= mid) temp.push_back(a[i++]);
7     while (j <= right) temp.push_back(a[j++]);
8     for (int k = 0; k < (int)temp.size(); ++k)
9         a[left + k] = temp[k];
10 }
11
12 void mergeSort(vector<int>& a, int left, int right) {
13     if (left >= right) return;
14     int mid = left + (right - left) / 2;
15     mergeSort(a, left, mid); // sol yariyi sirala
16     mergeSort(a, mid + 1, right); // sag yariyi sirala
17     merge(a, left, mid, right); // birlestir
18 }

```

23.3 Quick Sort: Bölümleme (Partition)

Hızlı sıralama bir *pivot* seçer ve diziyi “pivottan küçükler” ve “büyükler” diye ikiye böler; sonra her parçayı özyinelemeli sıralar. Ortalama $\mathcal{O}(n \log n)$ ’dir ama kötü pivot seçiminde $\mathcal{O}(n^2)$ ’ye düşebilir.

```

1 int partition(vector<int>& a, int left, int right) {
2     int pivot = a[right]; // son elemani pivot sec
3     int i = left - 1;
4     for (int j = left; j < right; ++j)
5         if (a[j] < pivot) swap(a[++i], a[j]);
6     swap(a[i + 1], a[right]); // pivotu ortasına koy
7     return i + 1;
8 }
9
10 void quickSort(vector<int>& a, int left, int right) {
11     if (left >= right) return;
12     int p = partition(a, left, right);
13     quickSort(a, left, p - 1); // sol parca
14     quickSort(a, p + 1, right); // sag parca
15 }

```

23.4 Counting Sort: Karşılaştırmaz Sıralama

Elemanlar küçük bir tam sayı aralığındaysa, karşılaştırma yapmadan sayarak $\mathcal{O}(n+k)$ ’de sıralarız. Bu, karşılaştırma sıralamalarının $\mathcal{O}(n \log n)$ alt sınırını aşar çünkü karşılaştırma kullanmaz.

```

1 void countingSort(vector<int>& a, int maxValue) {
2     vector<int> count(maxValue + 1, 0);
3     for (int x : a) ++count[x]; // her degeri say
4     int idx = 0;
5     for (int v = 0; v <= maxValue; ++v) // sirayla geri yaz
6         while (count[v]-- > 0) a[idx++] = v;
7 }

```

İpucu

Gerçek kodda kendi sıralamanı yazma: `std::sort` (introsort — quicksort+heapsort+insertion karışımı) $\mathcal{O}(n \log n)$ garantisi verir ve son derece optimize edilmiştir. Kararlılık gerekiyorsa `std::stable_sort` kullan. Kendi karşılaştırıcını lambda ile verebilirsin: `sort(v.begin(), v.end(), [](auto& a, auto& b){ return a > b; });`

24. Arama ve İkili Arama

24.1 Doğrusal ve İkili Arama

Sırasız veride tek yol doğrusal aramadır ($\mathcal{O}(n)$). Ama veri *sıralıysa*, ikili arama her adımda arama uzayını yarıya indirerek $\mathcal{O}(\log n)$ 'de bulur.

```

1 int binarySearch(const vector<int>& a, int target) {
2     int left = 0, right = a.size() - 1;
3     while (left <= right) {
4         int mid = left + (right - left) / 2; // tasmayı onleyen orta
5         if (a[mid] == target) return mid;
6         else if (a[mid] < target) left = mid + 1;
7         else right = mid - 1;
8     }
9     return -1; // bulunamadi
10 }
```

24.2 Cevap Üzerinde İkili Arama (Binary Search on Answer)

İkili aramanın güçlü bir genellemesi: cevabın *monoton* bir yes/no koşulunu sağladığı problemlerde, cevabın kendisini arayabiliriz. Örnek: “ k gün içinde bitirmek için gereken minimum günlük kapasite nedir?”

```

1 // feasible(x): x kapasite ile is k gunde bitiyor mu? (monoton: buyudukce true)
2 int smallestValid(int lo, int hi, function<bool(int)> feasible) {
3     while (lo < hi) {
4         int mid = lo + (hi - lo) / 2;
5         if (feasible(mid)) hi = mid; // orta yeterli -> daha kucuk dene
6         else lo = mid + 1; // yetersiz -> daha buyuk gerek
7     }
8     return lo; // ilk 'true' olan deger
9 }
```

25. Böl ve Yönet (Divide and Conquer)

Böl-yönet stratejisi bir problemi (1) daha küçük alt problemlere *böler*, (2) her birini özyinelemeli *çözer*, (3) sonuçları *birleştirir*. Merge sort ve quick sort bunun örnekleridir. Çalışma süresi genelde *Master teoremi* ile analiz edilir.

Bilgi

Master Teoremi: $T(n) = aT(n/b) + f(n)$ biçimindeki yinelemeler için: alt problem sayısı a , küçülme oranı b , birleştirme maliyeti $f(n)$. Merge sort'ta $a = 2, b = 2, f(n) = \mathcal{O}(n)$ olduğundan $T(n) = \mathcal{O}(n \log n)$ çıkar.

25.1 Uygulama: Hızlı Üs Alma (Binary Exponentiation)

a^n 'i $\mathcal{O}(n)$ yerine $\mathcal{O}(\log n)$ 'de hesaplarız: $a^n = (a^{n/2})^2$ özdeşliğini kullanarak üssü her adımda yarıya indiririz.

```

1 long long power(long long base, long long exp, long long mod) {
2     long long result = 1;
3     base %= mod;
4     while (exp > 0) {
5         if (exp & 1) result = result * base % mod; // us tekse carp
6         base = base * base % mod; // tabani karesini al
7         exp >>= 1; // ussu yariya indir
8     }
9     return result; // O(log us)
10 }
```

26. Açgözlü Algoritmalar (Greedy)

Açgözlü algoritma, her adımda o an *en iyi görünen* yerel seçimi yapar ve geri dönmez. Her problem için işe yaramaz — ama işe yaradığında (bir “açgözlü seçim özelliği” varsa) hem basit hem hızlıdır. Doğruluğu genelde kanıt gerektirir.

26.1 Uygulama: Etkinlik Seçimi (Activity Selection)

Bitiş zamanlarına göre sıralı etkinliklerden, çakışmadan en fazla kaçını seçebiliriz? Açgözlü kural: *her zaman en erken biteni seç.*

```

1 int maxActivities(vector<pair<int,int>> activities) { // {bitis, baslangic}
2     sort(activities.begin(), activities.end()); // bitis zamanina gore
3     int count = 0, lastEnd = -1;
4     for (auto [end, start] : activities)
5         if (start >= lastEnd) { // cakismiyorsa sec
6             ++count;
7             lastEnd = end;
8         }
9     return count; // O(n log n)
10 }
```

26.2 Açgözlü Ne Zaman Yanılır?

Açgözlü yaklaşım her zaman en iyiyi (optimum) bulmaz. Klasik örnek: bozuk para problemi. {1, 3, 4} birimleriyle 6 tutarını açgözlü (en büyük parayı önce al) $4 + 1 + 1 = 3$ parayla verir, ama optimum $3 + 3 = 2$ paradır! Böyle durumlarda *dinamik programlama* gerekir — ki bir sonraki bölümün konusu budur.

Dikkat

Bir açgözlü algoritma kurmadan önce, açgözlü seçimin gerçekten optimuma götürdüğünü *kanıtla* (değiş-tokuş argümanı / exchange argument) veya küçük karşı-örneklerle test et. “Mantıklı görünüyor” yeterli değildir; yukarıdaki bozuk para örneği tam da bu tuzağı gösterir.

27. Geri İzleme (Backtracking)

Geri izleme, bir çözümü adım adım inşa eden ve bir adımın çözüme götürmeyeceği anlaşıldığında *geri alıp* (backtrack) başka bir yol deneyen bir yöntemdir. Tüm olası kombinasyonları sistemli biçimde — ama umutsuz dalları budayarak — tarar. Permütasyonlar, alt kümeler, bulmacalar (sudoku, N-vezir) bununla çözülür.

27.1 Uygulama: Tüm Permütasyonlar

```

1 void permutations(vector<int>& a, int k, vector<vector<int>>& result) {
2     if (k == (int)a.size()) { result.push_back(a); return; }
3     for (int i = k; i < (int)a.size(); ++i) {
4         swap(a[k], a[i]);           // sec
5         permutations(a, k + 1, result); // derinlere in
6         swap(a[k], a[i]);           // geri al (backtrack)
7     }
8 }
```

27.2 Uygulama: N-Vezir Problemi

$N \times N$ tahtaya, birbirini tehdit etmeyecek şekilde N vezir yerleştir. Her satıra bir vezir koyar, güvenli sütun ararız; çıkmaza girersek geri döneriz.

```

1 bool isSafe(vector<int>& cols, int row, int s) {
2     for (int i = 0; i < row; ++i)
3         if (cols[i] == s || // ayni sutun
4             abs(cols[i] - s) == abs(i - row)) // ayni capraz
5             return false;
6     return true;
7 }
8
9 int nQueens(int n, int row, vector<int>& cols) {
10    if (row == n) return 1; // bir cozum tamamlandi
11    int count = 0;
12    for (int s = 0; s < n; ++s)
13        if (isSafe(cols, row, s)) {
14            cols[row] = s; // vezir koy
15            count += nQueens(n, row + 1, cols);
16            // ustune yazacagimiz icin ayrica geri almaya gerek yok
17        }
18    return count; // toplam cozum sayisi
19 }
```

Bilgi

Geri izlemenin gücü *budamadır* (pruning): olanaksız dalları erken elemek arama uzayını dramatik biçimde küçültür. N-vezirde isSafe kontrolü, milyonlarca imkânsız yerleşimi hiç denemeden eler. İyi budama, üstel bir aramayı pratikte çalışabilir hale getirir.

28. Dinamik Programlama (Ayrıntılı)

Dinamik programlama (DP), bir problemi *örtüşen alt problemlere* bölüp, her alt problemi **bir kez** çözerek ve sonucunu saklayarak (tekrar hesaplamadan) çözen bir tekniktir. Üstel çözümleri polinom zamana indirmesiyle algoritmanın en güçlü araçlarından biridir. Bu bölüm, DP'yi sıfırdan, bol örneklerle ve iki temel stiliyle (yukarıdan-aşağı ve aşağıdan-yukarı) ele alır.

28.1 DP Ne Zaman Uygulanır? İki Koşul

Bir problem DP ile çözülebilir, ancak ve ancak şu iki özelliğe sahipse:

Özellik	Anlamı
Örtüşen alt problemler	Aynı alt problemler tekrar tekrar çözülür (özyineleme ağacında tekrar).
Optimal alt yapı	Problemin optimum çözümü, alt problemlerin optimum çözümlerinden kurulur.

Örneğin Fibonacci: $F(n) = F(n-1) + F(n-2)$. Saf özyineleme $F(5)$ 'i hesaplarken $F(3)$ 'ü *iki kez*, $F(2)$ 'yi *üç kez* hesaplar — alt problemler örtüşür. Toplam çağrı sayısı üsteldir ($\mathcal{O}(2^n)$).

```

1 // SAF özyineleme:  $\mathcal{O}(2^n)$  -- KOTU! fib(50) evrenin yasından uzun sürer.
2 long long fibSlow(int n) {
3     if (n <= 1) return n;
4     return fibSlow(n - 1) + fibSlow(n - 2); // aynı degerler tekrar tekrar
5 }

```

28.2 İki DP Stili: Memoization vs Tabulation

Aynı DP'yi iki şekilde yazabiliriz. İkisi de her alt problemi bir kez çözer; fark, *yönde* ve uygulama-
madadır.

	Memoization (yukarıdan-aşağı)	Tabulation (aşağıdan-yukarı)
Yaklaşım	Özyineleme + önbellek	Döngüyle tablo doldurma
Yön	Büyük problemten küçüğe	Küçük alt problemten büyüğe
Sadece gerekli alt problemler	Evet (tembel)	Hayır (hepsi)
Yığın taşması riski	Var (derin özyineleme)	Yok
Genelde tercih	Yazması kolay	Biraz daha hızlı, bellek optimize edilebilir

```

1 // (1) MEMOIZATION -- özyineleme + önbellek.  $\mathcal{O}(n)$ 
2 vector<long long> memo;
3 long long fibMemo(int n) {
4     if (n <= 1) return n;
5     if (memo[n] != -1) return memo[n]; // daha önce hesaplandı mı?
6     return memo[n] = fibMemo(n-1) + fibMemo(n-2);
7 }
8 // Kullanım: memo.assign(n+1, -1); fibMemo(n);
9
10 // (2) TABULATION -- tabloyu küçükten büyüğe doldur.  $\mathcal{O}(n)$ 
11 long long fibTab(int n) {
12     if (n <= 1) return n;
13     vector<long long> dp(n + 1);
14     dp[0] = 0; dp[1] = 1;
15     for (int i = 2; i <= n; ++i)
16         dp[i] = dp[i-1] + dp[i-2];
17     return dp[n];

```

```

18 }
19
20 // (3) BELLEK OPTİMİZASYONU -- sadece son iki degeri tut. O(1) bellek
21 long long fibFast(int n) {
22     if (n <= 1) return n;
23     long long prev = 0, curr = 1;
24     for (int i = 2; i <= n; ++i) {
25         long long next = prev + curr;
26         prev = curr;
27         curr = next;
28     }
29     return curr;
30 }

```

Bilgi

DP'ye yaklaşımın 5 adımı: (1) *Durumu* tanımla — bir alt problemi hangi parametreler benzersiz belirler? (2) *Geçiş* (recurrence) yaz — bir durum, hangi daha küçük durumlardan hesaplanır? (3) *Taban durumları* belirle. (4) *Sırayı* belirle (memo ya da tablo). (5) Gerekiyorsa *belleği optimize et*. Bu bölümdeki her örneği bu adımlarla okuyacağız.

28.3 1B DP: Merdiven Çıkma ve Bozuk Para

Merdiven Çıkma

n basamaklı bir merdiveni bir seferde 1 veya 2 basamak atlayarak kaç farklı şekilde çıkabilirsin?

Durum: $dp[i] = i$. basamağa kaç yolla gelinir. **Geçiş:** $dp[i] = dp[i-1] + dp[i-2]$ (son adım 1 veya 2).

```

1 int climbStairs(int n) {
2     vector<int> dp(n + 1, 0);
3     dp[0] = 1; // yerinde durmanın 1 yolu
4     dp[1] = 1;
5     for (int i = 2; i <= n; ++i)
6         dp[i] = dp[i-1] + dp[i-2];
7     return dp[n];
8 }

```

Bozuk Para (Coin Change) — Minimum Para

Verilen coins birimleriyle amount'ı oluşturmak için gereken *en az* para sayısı. (Açgözlünün yanlışlığı, DP'nin doğru çözdüğü problem.) **Durum:** $dp[t] = t$ tutarı için minimum para. **Geçiş:** her para p için $dp[t] = \min(dp[t], dp[t-p] + 1)$.

```

1 int coinChange(const vector<int>& coins, int amount) {
2     const int INF = 1e9;
3     vector<int> dp(amount + 1, INF);
4     dp[0] = 0; // 0 tutar için 0 para
5     for (int t = 1; t <= amount; ++t)
6         for (int p : coins)
7             if (p <= t && dp[t - p] + 1 < dp[t])
8                 dp[t] = dp[t - p] + 1;
9     return dp[amount] == INF ? -1 : dp[amount]; // -1: mümkün değil
10 }

```

Bozuk Para — Kaç Farklı Yol?

Aynı paralarla amount'ı kaç *farklı* şekilde oluşturabiliriz? Burada dış döngünün *paralar* olması önemlidir — yoksa aynı kombinasyon farklı sırayla tekrar sayılır.

```

1 long long coinCombinations(const vector<int>& coins, int amount) {
2     vector<long long> dp(amount + 1, 0);
3     dp[0] = 1;
4     for (int p : coins) // DIS dongu: paralar
5         for (int t = p; t <= amount; ++t) // IC dongu: tutar
6             dp[t] += dp[t - p];
7     return dp[amount];
8 }

```

28.4 Sırt Çantası (0/1 Knapsack)

Klasik DP problemi: her biri bir weight ve value'e sahip nesneler var; kapasitesi W olan çantaya, toplam ağırlık W'yi aşmadan koyabileceğin *maksimum değer* nedir? “0/1” her nesnenin ya alınıp ya alınmadığını (parçalanamaz) belirtir.

Durum: $dp[i][w]$ = ilk i nesneyle ve w kapasiteyle elde edilen maks değer. **Geçiş:** nesne i 'yi *alma* ($dp[i-1][w]$) veya *al* ($dp[i-1][w-weight] + value$) — büyük olanı seç.

```

1 int knapsack(const vector<int>& weight, const vector<int>& value, int W) {
2     int n = weight.size();
3     vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
4     for (int i = 1; i <= n; ++i)
5         for (int w = 0; w <= W; ++w) {
6             dp[i][w] = dp[i-1][w]; // nesne i'yi ALMA
7             if (weight[i-1] <= w) // sigiyorsa AL secenegini de dene
8                 dp[i][w] = max(dp[i][w],
9                               dp[i-1][w - weight[i-1]] + value[i-1]);
10        }
11    return dp[n][W]; // O(n * W)
12 }

```

Bellek optimizasyonu

$dp[i]$ yalnızca $dp[i-1]$ 'e bağlı olduğundan 2B tabloyu 1B'ye indirebiliriz. Kritik nokta: iç döngüyü *tersten* ($w = W \rightarrow weight$) çevirmek, her nesnenin bir kez kullanılmasını garantiler.

```

1 int knapsack1B(const vector<int>& weight, const vector<int>& value, int W) {
2     vector<int> dp(W + 1, 0);
3     for (int i = 0; i < (int)weight.size(); ++i)
4         for (int w = W; w >= weight[i]; --w) // TERSTEN!
5             dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
6     return dp[W]; // O(n*W) zaman, O(W) bellek
7 }

```

28.5 En Uzun Ortak Alt Dizi (LCS)

İki string'in *ortak* (ardışık olması gerekmeyen ama sıra korunan) en uzun alt dizisinin uzunluğu. Diff araçları, DNA karşılaştırma ve sürüm kontrolü bunu kullanır. **Durum:** $dp[i][j]$ = a'nın ilk i karakteri ile b'nin ilk j karakteri için LCS uzunluğu.

```

1 int lcs(const string& a, const string& b) {
2     int n = a.size(), m = b.size();
3     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
4     for (int i = 1; i <= n; ++i)

```

```

5     for (int j = 1; j <= m; ++j) {
6         if (a[i-1] == b[j-1])
7             dp[i][j] = dp[i-1][j-1] + 1;      // karakter eslesti
8         else
9             dp[i][j] = max(dp[i-1][j], dp[i][j-1]); // birini atla
10    }
11    return dp[n][m];                          // O(n * m)
12 }

```

28.6 En Uzun Artan Alt Dizi (LIS)

Bir dizideki, elemanları kesin artan en uzun alt dizinin uzunluğu. İki çözüm sunuyoruz: sezgisel $O(n^2)$ DP ve zekice $O(n \log n)$.

```

1  // (1) DP -- O(n^2). dp[i] = i'de biten LIS uzunlugu
2  int lisSquared(const vector<int>& a) {
3      int n = a.size();
4      vector<int> dp(n, 1);
5      for (int i = 0; i < n; ++i)
6          for (int j = 0; j < i; ++j)
7              if (a[j] < a[i])
8                  dp[i] = max(dp[i], dp[j] + 1);
9      return *max_element(dp.begin(), dp.end());
10 }
11
12 // (2) Sabir dizisi + ikili arama -- O(n log n)
13 int lisLog(const vector<int>& a) {
14     vector<int> tails;                // tails[k]: uzunlugu k+1 olan
15     for (int x : a) {                 // artan altdizinin en kucuk sonu
16         auto it = lower_bound(tails.begin(), tails.end(), x);
17         if (it == tails.end()) tails.push_back(x); // uzat
18         else *it = x;                 // daha kucukle degistir
19     }
20     return tails.size();
21 }

```

28.7 Düzenleme Uzaklığı (Edit Distance / Levenshtein)

Bir string'i diğerine dönüştürmek için gereken en az işlem (ekle, sil, değiştir). Yazım denetimi ve DNA hizalama kullanır. **Durum:** $dp[i][j] = a[0..i]$ 'yi $b[0..j]$ 'ye çevirmenin min maliyeti.

```

1  int editDistance(const string& a, const string& b) {
2      int n = a.size(), m = b.size();
3      vector<vector<int>> dp(n + 1, vector<int>(m + 1));
4      for (int i = 0; i <= n; ++i) dp[i][0] = i; // hepsini sil
5      for (int j = 0; j <= m; ++j) dp[0][j] = j; // hepsini ekle
6      for (int i = 1; i <= n; ++i)
7          for (int j = 1; j <= m; ++j) {
8              if (a[i-1] == b[j-1])
9                  dp[i][j] = dp[i-1][j-1];      // ayni -> islem yok
10             else
11                 dp[i][j] = 1 + min({ dp[i-1][j],      // sil
12                                     dp[i][j-1],        // ekle
13                                     dp[i-1][j-1] });    // degistir
14         }
15     return dp[n][m];                          // O(n * m)
16 }

```

28.8 Grid Üzerinde DP: Minimum Yol Toplamı

Sol üstten sağ alta, yalnızca sağa/aşağı giderek en düşük maliyetli yol. **Durum:** $dp[i][j] = (i, j)$ hücresine ulaşmanın min maliyeti; sadece üstten veya soldan gelir.

```

1 int minPathSum(const vector<vector<int>>& grid) {
2     int n = grid.size(), m = grid[0].size();
3     vector<vector<int>> dp(n, vector<int>(m));
4     dp[0][0] = grid[0][0];
5     for (int j = 1; j < m; ++j) dp[0][j] = dp[0][j-1] + grid[0][j];
6     for (int i = 1; i < n; ++i) dp[i][0] = dp[i-1][0] + grid[i][0];
7     for (int i = 1; i < n; ++i)
8         for (int j = 1; j < m; ++j)
9             dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]);
10    return dp[n-1][m-1]; // O(n * m)
11 }

```

28.9 Bitmask DP: Gezgin Satıcı (TSP)

Az sayıda düğüm ($n \leq 20$) olduğunda, ziyaret edilen düğüm kümesini bir *bitmask* (bit maskesi) olarak kodlayıp DP yapabiliriz. TSP: tüm şehirleri tam bir kez gezip başa dönen en kısa tur. **Durum:** $dp[mask][i]$ = mask kümesindeki şehirler ziyaret edilmiş ve şu an i 'deyken min maliyet.

```

1 int tsp(const vector<vector<int>>& d) { // d: mesafe matrisi
2     int n = d.size();
3     const int INF = 1e9;
4     vector<vector<int>> dp(1 << n, vector<int>(n, INF));
5     dp[1][0] = 0; // sadece 0. şehir ziyaret, oradayız
6     for (int mask = 1; mask < (1 << n); ++mask)
7         for (int u = 0; u < n; ++u) {
8             if (dp[mask][u] == INF) continue;
9             if (!(mask & (1 << u))) continue;
10            for (int v = 0; v < n; ++v) // sonraki şehir
11                if (!(mask & (1 << v)))
12                    dp[mask | (1 << v)][v] =
13                        min(dp[mask | (1 << v)][v], dp[mask][u] + d[u][v]);
14        }
15    int result = INF;
16    for (int u = 1; u < n; ++u) // başa (0) donus
17        result = min(result, dp[(1 << n) - 1][u] + d[u][0]);
18    return result; // O(2^n * n^2)
19 }

```

28.10 Ağaç Üzerinde DP

DP yalnızca dizilerde değil ağaçlarda da çalışır: her düğümün cevabı, çocuklarının cevaplarından hesaplanır (post-order). Örnek: bir ağaçta komşu olmayan düğümlerden en büyük bağımsız küme (maksimum ağırlık).

```

1 // dp[u][0] = u alınmadan alt ağacın en iyisi
2 // dp[u][1] = u alınarak alt ağacın en iyisi
3 void treeDP(int u, int parent, const vector<vector<int>>& adj,
4             const vector<int>& weight, vector<array<int,2>>& dp) {
5     dp[u][0] = 0;
6     dp[u][1] = weight[u];
7     for (int v : adj[u]) {
8         if (v == parent) continue;
9         treeDP(v, u, adj, weight, dp);

```

```

10     dp[u][0] += max(dp[v][0], dp[v][1]); // u yoksa çocuk serbest
11     dp[u][1] += dp[v][0];               // u varsa çocuk alınamaz
12 }
13 }
14 // Cevap: max(dp[root][0], dp[root][1])

```

28.11 Özetle: DP Problemlerini Tanımak

Problem tipi	Tipik durum tanımı
Dizide seçim (knapsack)	dp[i][capacity]
İki string (LCS, edit)	dp[i][j] (iki örnek)
Alt dizi (LIS)	dp[i] = i'de biten en iyi
Grid	dp[i][j] = hücreye ulaşma
Aralık (matrix chain)	dp[i][j] = [i, j] aralığı
Alt küme (TSP)	dp[mask][i] (bitmask)
Ağaç	dp[node][state] (post-order)

İpucu

DP öğrenmenin en iyi yolu bol problem çözmektir. “Bu problemi daha küçük bir versiyonundan çözebilir miyim?” diye sor. Cevap evetse ve alt problemler örtüşüyorsa, DP büyük ihtimalle uygulanır. Önce yavaş özyinelemeyi yaz, sonra memo dizisi ekleyerek hızlandır — bu yol neredeyse her zaman işe yarar.

29. String Algoritmaları

Metin işleme, örüntü arama ve biyoinformatik string algoritmalarına dayanır. En temel problem *örüntü eşleme*: bir metin içinde bir deseni bulmak.

29.1 KMP: Doğrusal Zamanda Örüntü Arama

Naif arama $\mathcal{O}(nm)$ 'dir. Knuth-Morris-Pratt (KMP), desenin kendi tekrarlarından yararlanarak geri gitmeden $\mathcal{O}(n+m)$ 'de arar. Anahtar, *başarısızlık fonksiyonu* (LPS — en uzun uygun örnek=sonek) tablosudur.

```

1  vector<int> buildLps(const string& pattern) {
2      int m = pattern.size();
3      vector<int> lps(m, 0);
4      int len = 0;
5      for (int i = 1; i < m; ) {
6          if (pattern[i] == pattern[len]) lps[i++] = ++len;
7          else if (len) len = lps[len - 1]; // geri sic
8          else lps[i++] = 0;
9      }
10     return lps;
11 }
12
13 vector<int> kmpSearch(const string& text, const string& pattern) {

```

```

14     vector<int> lps = buildLps(pattern), result;
15     int i = 0, j = 0; // i: metin, j: desen
16     while (i < (int)text.size()) {
17         if (text[i] == pattern[j]) { ++i; ++j; }
18         if (j == (int)pattern.size()) { // tam eslesme bulundu
19             result.push_back(i - j);
20             j = lps[j - 1];
21         } else if (i < (int)text.size() && text[i] != pattern[j]) {
22             if (j) j = lps[j - 1]; // deseni kaydır
23             else ++i;
24         }
25     }
26     return result; // eslesmelerin baslangic indeksleri
27 }

```

29.2 Uygulama: Palindrom Kontrolü ve Sayma

```

1 bool isPalindrome(const string& s) {
2     int left = 0, right = s.size() - 1;
3     while (left < right)
4         if (s[left++] != s[right--]) return false;
5     return true; // O(n)
6 }

```

30. Bit Manipülasyonu

Sayıları ikili tabanda, bit düzeyinde işlemek bazı problemleri son derece hızlı ve zarif çözer. Alt küme üretme, bayrak (flag) tutma ve düşük seviye optimizasyonlar bit işlemlerine dayanır.

İşlem	Kod	Anlamı
Bit oku	$(x \gg i) \& 1$	<i>i.</i> bit 1 mi?
Bit kur	$x \mid (1 \ll i)$	<i>i.</i> biti 1 yap
Bit temizle	$x \& \sim(1 \ll i)$	<i>i.</i> biti 0 yap
Bit çevir	$x \wedge (1 \ll i)$	<i>i.</i> biti tersle
Son biti al	$x \& -x$	En düşük kurulu bit
2'nin kuvveti mi?	$x \&\& !(x \& (x-1))$	Tek bit kuruluysa

```

1 // Kurulu bit sayisi (population count)
2 int countBits(unsigned x) {
3     int count = 0;
4     while (x) { x &= (x - 1); ++count; } // son kurulu biti sil
5     return count;
6 }
7 // Derleyici built-in'i (çok daha hızlı):
8 int count = __builtin_popcount(x);
9
10 // Bir kumenin TUM alt kumelerini gez (bitmask ile)
11 void subsets(const vector<int>& a) {

```

```

12  int n = a.size();
13  for (int mask = 0; mask < (1 << n); ++mask) {
14      for (int i = 0; i < n; ++i)
15          if (mask & (1 << i)) cout << a[i] << " ";    // i. eleman kumede
16      cout << "\n";
17  }                                     // toplam 2^n alt kume
18  }

```

Bilgi

XOR'un sihri: $a \oplus a = 0$ ve $a \oplus 0 = a$ olduğundan, bir dizide her eleman iki kez, biri tek kez geçiyorsa, hepsini XOR'layarak tek olanı $\mathcal{O}(n)$ zaman ve $\mathcal{O}(1)$ bellekte buluruz: `int tek = 0;` `for (int x : a) tek ^= x;` Bu, hash tablosuna gerek kalmadan çalışan klasik bir numaradır.

31. Algoritma Teknikleri — Alıştırmalar ve Çözümler

31.1 Sıralama

Soru 71

Bir tam sayı dizisindeki *inversiyon* sayısını bulun. İversiyon, $i < j$ olmasına rağmen $A[i] > A[j]$ olan (i, j) ikilisidir. Kaba kuvvet $\mathcal{O}(n^2)$ yerine merge sort tabanlı $\mathcal{O}(n \log n)$ bir çözüm yazın.

Çözüm. İversiyon sayısı, bir dizinin ne kadar “karışık” olduğunun ölçüsüdür; sıralı bir dizide 0, tersten sıralı bir dizide ise $\binom{n}{2}$ inversiyon vardır. Merge sort’un birleştirme (merge) adımını değiştirerek bu sayıyı bedavaya sayabiliriz: sol yarı ile sağ yarıyı birleştirirken, sol taraftan bir eleman yerine sağ taraftan bir eleman seçtiğimiz her seferinde, sağdaki bu eleman soldaki *kalan tüm elemanlardan* küçüktür. Yani soldaki kalan eleman sayısı kadar inversiyon bir anda sayılır.

Neden çalışır? Merge sort böl-yönet mantığıyla inversiyonları üç gruba ayırır: (1) tamamen sol yarıda olanlar, (2) tamamen sağ yarıda olanlar, (3) biri solda biri sağda olan “çapraz” inversiyonlar. İlk iki grup özyinelemeli çağrılarda sayılır; çapraz olanlar ise birleştirme sırasında sayılır. Toplam maliyet merge sort ile aynıdır: $\mathcal{O}(n \log n)$ zaman, $\mathcal{O}(n)$ ek bellek.

```

1  #include <vector>
2  using namespace std;
3
4  // [left, right) yarı açık aralığını sırala ve inversiyon sayısını döndür
5  long long mergeCount(vector<int>& a, vector<int>& buffer, int left, int right) {
6      if (right - left <= 1) return 0;    // 0 veya 1 eleman: inversiyon yok
7      int mid = left + (right - left) / 2;
8      long long count = 0;
9      count += mergeCount(a, buffer, left, mid);    // sol yarıdaki inversiyonlar
10     count += mergeCount(a, buffer, mid, right);    // sağ yarıdaki inversiyonlar
11
12     // Birleştirme (merge) adımı: çapraz inversiyonları say
13     int i = left, j = mid, k = left;
14     while (i < mid && j < right) {
15         if (a[i] <= a[j]) {
16             buffer[k++] = a[i++];    // sol elemanı sec, inversiyon yok
17         } else {
18             buffer[k++] = a[j++];    // sağ eleman küçük
19             count += (mid - i);    // soldaki kalan tüm elemanlarla inversiyon
20         }
21     }

```

```

22     while (i < mid)    buffer[k++] = a[i++];        // sol kalanlar
23     while (j < right) buffer[k++] = a[j++];        // sag kalanlar
24     for (int t = left; t < right; ++t) a[t] = buffer[t]; // geri kopyala
25     return count;
26 }
27
28 long long countInversions(vector<int> a) {
29     vector<int> buffer(a.size());
30     return mergeCount(a, buffer, 0, (int)a.size());
31 }

```

Dikkat

İnversiyon sayısı n orta büyüklükte bile $\binom{n}{2}$ mertebesine ulaşabildiği için sonucu mutlaka long long tipinde tutun; int taşması sık yapılan bir hatadır.

Soru 72

Sıralanmamış bir dizide k . en büyük elemanı bulun. Diziyi tamamen sıralamadan ($\mathcal{O}(n \log n)$), ortalama $\mathcal{O}(n)$ zamanda çalışan QuickSelect algoritmasını yazın.

Çözüm. QuickSelect, quicksort’un “sadece bir tarafa in” versiyonudur. Bir pivot seçip diziyi bölümlere ayırdıktan (partition) sonra, pivotun nihai konumu (index) bize onun *kaçıncı sıradaki* eleman olduğunu söyler. Aradığımız konumla karşılaştırıp sadece pivotun bir tarafına özyineleniriz; diğer tarafı tamamen atarız. Quicksort iki tarafa da indiği için $\mathcal{O}(n \log n)$, QuickSelect tek tarafa indiği için ortalama $n + n/2 + n/4 + \dots \approx 2n = \mathcal{O}(n)$ olur.

“ k . en büyük” istendiği için, k . en büyük eleman artan sıralamada $(n - k)$. indekstir. Pivotu rastgele seçmek en kötü durum olan $\mathcal{O}(n^2)$ olasılığını pratikte yok eder (kötü niyetli girdiye karşı koruma).

```

1  #include <vector>
2  #include <cstdlib>
3  using namespace std;
4
5  // [left, right] aralığında partition; pivotun son indeksini dondur (Lomuto)
6  int partition(vector<int>& a, int left, int right) {
7      int pivotIndex = left + rand() % (right - left + 1);
8      swap(a[pivotIndex], a[right]);        // pivotu sona tasi
9      int pivot = a[right];
10     int store = left;
11     for (int i = left; i < right; ++i)
12         if (a[i] < pivot) swap(a[i], a[store++]);
13     swap(a[store], a[right]);              // pivotu dogru yerine koy
14     return store;
15 }
16
17 int kthLargest(vector<int> a, int k) {
18     int n = (int)a.size();
19     int target = n - k;                    // k. en buyuk = (n-k). en kucuk (0-tabanlı)
20     int left = 0, right = n - 1;
21     while (left <= right) {
22         int p = partition(a, left, right);
23         if (p == target) return a[p];      // tam yerinde bulduk
24         else if (p < target) left = p + 1; // sag tarafa in
25         else right = p - 1;               // sol tarafa in
26     }
27     return -1;                            // buraya gelinmez (gecerli k icin)

```

28 }

Bilgi

Deterministik en kötü durum $\mathcal{O}(n)$ garantisi isteniyorsa “median of medians” pivot seçimi kullanılır, ancak sabit çarpanı büyük olduğu için pratikte rastgele pivotlu QuickSelect tercih edilir. Alternatif olarak boyut k ’lık bir min-heap ile $\mathcal{O}(n \log k)$ elde edilebilir.

Soru 73

Yalnızca 0, 1 ve 2 değerlerinden oluşan (kırmızı, beyaz, mavi) bir diziyi tek geçişte ve sabit ek bellekle sıralayın (Dutch National Flag / Hollanda bayrağı problemi).

Çözüm. Sayma sıralaması (counting sort) da işi görür ama iki geçiş gerektirir. Dijkstra’nın “Hollanda bayrağı” algoritması tek geçişte, üç işaretçi (three pointers) ile çözer. Fikir, diziyi dört bölgeye ayırmaktır: kesin 0’lar, kesin 1’ler, işlenmemiş bölge, kesin 2’ler. low işaretçisi 0’ların sınırını, high 2’lerin sınırını, mid ise tarama başını temsil eder. Değişmez (invariant): $[0, \text{low})$ hep 0, $[\text{low}, \text{mid})$ hep 1, (high, n) hep 2.

mid eleman 0 ise low ile takas edip ikisini de ilerletiriz. 2 ise high ile takas edip sadece high’ı geri çekeriz (takas edilen eleman henüz incelenmediği için mid ilerlemez!). 1 ise sadece mid ilerler. Karmaşıklık $\mathcal{O}(n)$ zaman, $\mathcal{O}(1)$ bellek.

```

1  #include <vector>
2  using namespace std;
3
4  void sortColors(vector<int>& nums) {
5      int low = 0, mid = 0, high = (int)nums.size() - 1;
6      while (mid <= high) {
7          if (nums[mid] == 0) {
8              swap(nums[low++], nums[mid++]); // 0'i one at
9          } else if (nums[mid] == 2) {
10             swap(nums[mid], nums[high--]); // 2'yi sona at, mid ILERLEMEZ
11          } else {
12              mid++; // 1 zaten yerinde
13          }
14      }
15  }
```

İpucu

Neden 2 ile takas ederken mid++ yapmıyoruz? Çünkü high’dan gelen elemanın ne olduğunu bilmiyoruz; onu bir sonraki turda yeniden incelemeliyiz. Oysa 0 ile takas ettiğimiz low konumu zaten taranmış (orada ya 1 ya da 0 vardı) olduğu için güvenle ilerleyebiliriz.

31.2 Arama**Soru 74**

Artan sıralı, ancak bilinmeyen bir noktadan döndürülmüş (rotated) ve tekrar içermeyen bir dizide verilen hedefi $\mathcal{O}(\log n)$ zamanda arayın. Örn. $[4, 5, 6, 7, 0, 1, 2]$ içinde 0’ı bulun.

Çözüm. Döndürülmüş sıralı dizi, iki sıralı parçadan oluşur. İkili aramanın (binary search) her adımında, mid’e baktığımızda yarılardan *en az biri* kesinlikle sıralıdır. Hangi yarının sıralı olduğunu $\text{nums}[\text{left}] \leq \text{nums}[\text{mid}]$ kontrolüyle anlarız. Sıralı yarının uç değerleri arasında hedef var mı diye bakarız: varsa o yarıya, yoksa diğer yarıya ineriz. Böylece klasik ikili aramanın $\mathcal{O}(\log n)$ garantisini korumuş oluruz.

Kritik nokta, karşılaştırmalarda $\leq$ mı $<$ mu kullanılacağıdır; `nums[left] <= nums[mid]` eşitliği tek elemanlı yarıyı (`left == mid`) doğru sınıflandırır.

```

1  #include <vector>
2  using namespace std;
3
4  int searchRotated(vector<int>& nums, int target) {
5      int left = 0, right = (int)nums.size() - 1;
6      while (left <= right) {
7          int mid = left + (right - left) / 2;
8          if (nums[mid] == target) return mid;
9          if (nums[left] <= nums[mid]) {           // sol yari sirali
10             if (nums[left] <= target && target < nums[mid])
11                 right = mid - 1;                // hedef sol yarida
12             else
13                 left = mid + 1;
14         } else {                                // sag yari sirali
15             if (nums[mid] < target && target <= nums[right])
16                 left = mid + 1;                 // hedef sag yarida
17             else
18                 right = mid - 1;
19         }
20     }
21     return -1;
22 }
```

Dikkat

Dizi tekrar eden elemanlar içeriyorsa (örn. `[1, 1, 1, 0, 1]`) `nums[left] == nums[mid]` durumunda hangi yarının sıralı olduğunu anlayamayız; bu durumda `left++` ile bir adım daralmak gerekir ve en kötü durum $\mathcal{O}(n)$ 'e çıkar.

Soru 75

Cevap üzerinde ikili arama: Bir bant (konveyör) üzerinde belli sırayla taşınacak paketlerin ağırlıkları `weights` dizisinde verilmiş. Paketleri sırayı bozmadan en fazla `days` günde taşımak istiyoruz. Günlük minimum gemi kapasitesini bulun.

Çözüm. Bu problemi doğrudan çözmek zor, ama “ X kapasiteyle bu işi `days` günde bitirebilir miyiz?” sorusunu $\mathcal{O}(n)$ 'de yanıtlayabiliriz (ağgözlüce her güne sığıdığı kadar paket koy). Kritik gözlem: kapasite arttıkça gerekli gün sayısı *monoton azalır*. Yani bir eşik kapasitenin altında “hayır”, üstünde “evet” cevabı vardır. Bu monotonluk, cevabın kendisi üzerinde ikili arama yapmamıza olanak tanır.

Arama aralığının alt sınırı `max(weights)`'tir (en ağır paket tek başına bir güne sığmalı), üst sınırı `sum(weights)`'tir (her şeyi tek günde taşı). İkili aramayla en küçük geçerli kapasiteyi buluruz. Karmaşıklık: $\mathcal{O}(n \log(\text{sum}))$.

```

1  #include <vector>
2  #include <numeric>
3  #include <algorithm>
4  using namespace std;
5
6  // capacity kapasiteyle kac gun gerekir? days'ten kucuk/esitse yeterli
7  bool canShip(vector<int>& weights, int capacity, int days) {
8      int neededDays = 1, load = 0;
9      for (int w : weights) {
10         if (load + w > capacity) {           // bu paket bu güne sığmaz
11             neededDays++;                    // yeni güne geç
```

```

12         load = 0;
13     }
14     load += w;
15 }
16 return neededDays <= days;
17 }
18
19 int shipWithinDays(vector<int>& weights, int days) {
20     int low = *max_element(weights.begin(), weights.end());
21     int high = accumulate(weights.begin(), weights.end(), 0);
22     while (low < high) { // en küçük geçerli kapasiteyi bul
23         int mid = low + (high - low) / 2;
24         if (canShip(weights, mid, days))
25             high = mid; // mid yeterli, daha küçükünü dene
26         else
27             low = mid + 1; // mid yetersiz, büyüt
28     }
29     return low;
30 }

```

Genel Kalıp

“Cevap üzerinde ikili arama” deseninin anahtarı bir *karar fonksiyonu* $feasible(x)$ bulmaktır: bu fonksiyon bir eşikten sonra hep true (veya hep false) olmalı. Bu monotonluk sağlandığında, en küçük/en büyük geçerli değeri logaritmik sürede buluruz. “Minimum en büyük değer” ya da “maksimum en küçük değer” isteyen problemler genelde bu kalıba oturur.

Soru 76

Unimodal (tek tepeli) bir dizide – önce kesin artıp sonra kesin azalan – tepe (peak) elemanı ternary search (üçlü arama) ile bulun.

Çözüm. İkili arama iki parçaya böler; ternary search aralığı iki iç noktayla üç parçaya böler ve her adımda dördte birini eler. Unimodal bir fonksiyonda $m1 < m2$ ise tepe kesinlikle $m1$ ’in solunda değildir (çünkü fonksiyon hâlâ artıyor), dolayısıyla $left = m1 + 1$. Aksi halde tepe $m2$ ’nin sağında değildir, $right = m2 - 1$. Her turda aralık $2/3$ oranında daraldığı için karmaşıklık $\mathcal{O}(\log_{3/2} n) = \mathcal{O}(\log n)$ ’dir.

Bu teknik özellikle sürekli (real) alanlarda konveks/konkav fonksiyonların ekstremumunu bulmakta çok kullanışlıdır; orada döngüyü sabit sayıda (örn. 200) iterasyonla ya da bir eps eşliğiyle sınırlarız.

```

1  #include <vector>
2  using namespace std;
3
4  int findPeak(vector<int>& a) {
5      int left = 0, right = (int)a.size() - 1;
6      while (left < right) {
7          int m1 = left + (right - left) / 3;
8          int m2 = right - (right - left) / 3;
9          if (a[m1] < a[m2])
10             left = m1 + 1; // tepe m1'in sagi tarafinda
11         else
12             right = m2 - 1; // tepe m2'nin sol tarafinda
13     }
14     return left; // left == right: tepe
15 }

```

Bilgi

Ayrık (integer) dizilerde tepe bulmak için ikili arama ($a[mid]$ ile $a[mid + 1]$ karşılaştırması) daha yaygın ve sabit çarpanı düşüktür. Ternary search'ün asıl gücü, konveks/unimodal *sürekli* fonksiyonların minimum/maksimumunu bulurken ortaya çıkar.

31.3 Böl ve Yönet**Soru 77**

Maksimum alt dizi toplamını (maximum subarray sum) böl-yönet ile $\mathcal{O}(n \log n)$ çözün, ardından Kadane algoritmasının $\mathcal{O}(n)$ çözümüyle karşılaştırın.

Çözüm. Böl-yönet kurgusunda diziyi ortadan ikiye böleriz. Maksimum alt dizi üç yerden birinde olur: (1) tamamen sol yarıda, (2) tamamen sağ yarıda, (3) orta çizgiyi *geçen* bir alt dizide. İlk ikisini özyinelemeyle buluruz. Üçüncüsü için, ortadan başlayıp sola doğru en iyi eki ve sağa doğru en iyi eki ayrı ayrı $\mathcal{O}(n)$ 'de hesaplayıp toplarız. Yineleme bağıntısı $T(n) = 2T(n/2) + \mathcal{O}(n)$ olduğundan (aynı merge sort gibi) sonuç $\mathcal{O}(n \log n)$ 'dir.

Kadane algoritması ise aynı problemi tek geçişte, dinamik programlama fikriyle çözer: “şu ana kadar biten en iyi alt dizi toplamı”nı taşır ve negatifleşince sıfırlar. Karşılaştırmada Kadane hem daha basit hem de $\mathcal{O}(n)$ ile daha hızlıdır; böl-yönet çözümü ise böl-yönet düşünme biçimini öğretmek için değerlidir.

```

1  #include <vector>
2  #include <algorithm>
3  #include <climits>
4  using namespace std;
5
6  // --- Böl-yönet:  $\mathcal{O}(n \log n)$  ---
7  long long crossSum(vector<int>& a, int left, int mid, int right) {
8      long long leftBest = LLONG_MIN, sum = 0;
9      for (int i = mid; i >= left; --i) { sum += a[i]; leftBest = max(leftBest, sum); }
10     long long rightBest = LLONG_MIN; sum = 0;
11     for (int i = mid + 1; i <= right; ++i) { sum += a[i]; rightBest = max(rightBest, sum); }
12     return leftBest + rightBest;
13 }
14 long long maxSubDC(vector<int>& a, int left, int right) {
15     if (left == right) return a[left];
16     int mid = left + (right - left) / 2;
17     return max({ maxSubDC(a, left, mid),
18                 maxSubDC(a, mid + 1, right),
19                 crossSum(a, left, mid, right) });
20 }
21
22 // --- Kadane:  $\mathcal{O}(n)$  ---
23 long long maxSubKadane(vector<int>& a) {
24     long long best = a[0], cur = a[0];
25     for (size_t i = 1; i < a.size(); ++i) {
26         cur = max((long long)a[i], cur + a[i]); // yeni başla ya da uzat
27         best = max(best, cur);
28     }
29     return best;
30 }

```

Dikkat

Tüm elemanlar negatif olabileceği için başlangıç değerini 0 değil, `a[0]` (ya da `LLONG_MIN`) yapın. `best = 0` ile başlamak, tümü negatif dizilerde yanlışlıkla boş alt diziyi (toplam 0) döndürür.

Soru 78

Hızlı üs alma (fast exponentiation) ile matris kuvveti olarak n . Fibonacci sayısını $\mathcal{O}(\log n)$ zamanda hesaplayın. (Yeni konu, kısa)

Çözüm. Fibonacci için $\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$ özdeşliği geçerlidir. Yani 2×2 “dönüşüm matrisinin” n . kuvvetini alırsak F_n ’i buluruz. Bir sayının n . kuvvetini hesaplariken $x^n = (x^{n/2})^2$ (çift n) özelliğini kullanan “binary exponentiation”, çarpma sayısını n ’den $\log n$ ’e düşürür. Aynı fikir matrislerde de çalışır: skaler çarpma yerine matris çarpımı kullanırız. Böylece Fibonacci $\mathcal{O}(\log n)$ matris çarpımıyla (her biri sabit 2^3 işlem) hesaplanır.

```

1 #include <array>
2 using namespace std;
3 typedef array<array<long long,2>,2> Mat;
4
5 Mat mul(const Mat& A, const Mat& B) {
6     Mat C = {{0,0},{0,0}};
7     for (int i = 0; i < 2; ++i)
8         for (int j = 0; j < 2; ++j)
9             for (int k = 0; k < 2; ++k)
10                 C[i][j] += A[i][k] * B[k][j];    // (mod almak gerekirse burada)
11     return C;
12 }
13
14 long long fib(long long n) {
15     if (n == 0) return 0;
16     Mat result = {{1,0},{0,1}};                // birim matris
17     Mat base = {{1,1},{1,0}};
18     while (n > 0) {                             // binary exponentiation
19         if (n & 1) result = mul(result, base);    // bit set ise carp
20         base = mul(base, base);                  // tabani karele
21         n >>= 1;
22     }
23     return result[0][1];                        // F_n bu hucrede
24 }
```

Bilgi

Aynı “binary exponentiation” iskeleti, modüler üs alma ($a^b \bmod m$) için kriptografide, doğrusal yineleme bağıntılarını hızlandırmak için yarışma programlamada yaygın kullanılır. Matris çarpımını mod ile yaparsanız çok büyük Fibonacci sayılarını taşmadan hesaplırsınız.

31.4 Ağgözlü (Greedy)**Soru 79**

Aktivite seçimi (activity selection): Başlangıç ve bitiş zamanları verilen aktivitelerden, hiçbirini çakışmayacak şekilde en fazla kaç tanesini seçebileceğinizi bulun.

Çözüm. Ağgözlü sezgi: her adımda *en erken biten* aktiviteyi seçmek, geriye kalan zamanı maksimize eder ve en fazla aktiviteye yer bırakır. Bunu kanıtlamak için “değiştirme argümanı” (exchange argument) kullanılır: herhangi bir optimal çözümdeki ilk aktiviteyi, en erken biten aktiviteyle değiştirsek

çözüm hâlâ geçerli ve en az o kadar büyük kalır. Dolayısıyla aktiviteleri bitiş zamanına göre sıralayıp, son seçilenle çakışmayan ilk aktiviteyi almak optimaldir. Sıralama $\mathcal{O}(n \log n)$, tarama $\mathcal{O}(n)$.

```

1 #include <vector>
2 #include <algorithm>
3 using namespace std;
4
5 int maxActivities(vector<pair<int,int>> acts) { // {start, finish}
6     // Bitiş zamanına göre sırala
7     sort(acts.begin(), acts.end(),
8         [](auto& x, auto& y){ return x.second < y.second; });
9     int count = 0, lastFinish = -1;
10    for (auto& [start, finish] : acts) {
11        if (start >= lastFinish) { // son secilenle cakismiyor
12            count++;
13            lastFinish = finish;
14        }
15    }
16    return count;
17 }

```

Greedy ne zaman doğru?

Açgözlü yaklaşım ancak problem “greedy choice property” ve “optimal substructure” özelliklerini taşıyorsa doğru sonuç verir. Bunu her zaman ya bir değiştirme argümanıyla kanıtlamalı ya da küçük karşı örneklerle test etmelisiniz; “sezgisel olarak doğru görünmesi” yeterli değildir.

Soru 80

Jump Game: $\text{nums}[i]$, i indeksinden ileri atlayabileceğiniz maksimum adımı gösterir. 0. indeksten son indekse ulaşılabilir mi?

Çözüm. Açgözlü ölçü, o ana kadar ulaşılabilen *en uzak indekstir* (reach). Diziyi soldan sağa tararken, eğer bulunduğumuz indeks reach’i aşmışsa oraya hiç ulaşamamışız demektir, false döneriz. Aksi halde reach’i $i + \text{nums}[i]$ ile güncelleriz. En baştan reach, son indekse eşit veya büyük olursa ulaşılabilir. Tek geçiş, $\mathcal{O}(n)$ zaman, $\mathcal{O}(1)$ bellek. Bu, dinamik programlama çözümünden (her indekse ulaşılabilir mi tablosu) hem daha hızlı hem daha basittir.

```

1 #include <vector>
2 using namespace std;
3
4 bool canJump(vector<int>& nums) {
5     int reach = 0; // ulaşabildiğimiz en uzak indeks
6     for (int i = 0; i < (int)nums.size(); ++i) {
7         if (i > reach) return false; // buraya hic ulasilamadi
8         reach = max(reach, i + nums[i]); // menzili genislet
9     }
10    return true; // dongu bitti demek son indekse ulasildi
11 }

```

Soru 81

Para üstü problemi: Verilen madeni para türleriyle bir tutarı en az sayıda parayla ödemek istiyoruz. Açgözlü (her seferinde en büyük parayı seç) yaklaşımının neden *genelde* yanlış olduğunu bir örnekle gösterin.

Çözüm. Açgözlü yaklaşım, standart para sistemlerinde (örn. 1, 5, 10, 25) doğru çalışır ama her para kümesinde değil. Klasik karşı örnek: paralar [1, 3, 4] ve tutar 6 olsun. Açgözlü önce 4’ü seçer,

kalan 2 için iki tane 1 kullanır: toplam 3 para (4+1+1). Oysa optimal çözüm 3+3, yani sadece 2 paradır. Ağgözlü “şimdiki en iyi” seçimi yaptı ama bu, ileride daha kötü bir duruma sürükledi – greedy choice property burada geçerli değil.

Doğru genel çözüm dinamik programlamadır (bkz. *Dinamik Programlama* bölümündeki “madeni para – min” sorusu): $dp[x] = x$ tutarını ödemek için gereken minimum para. Her tutar için tüm para türlerini deneyip en iyisini alırsınız; karmaşıklık $\mathcal{O}(\text{amount} \times \text{coins})$. Bu, ağgözlünün göremediği “geriye dönük” en iyi bileşimleri de değerlendirir.

```

1 #include <vector>
2 #include <algorithm>
3 using namespace std;
4
5 // Yanlis: acgozlu (karsi ornek uretmek icin)
6 int coinGreedy(vector<int> coins, int amount) {
7     sort(coins.rbegin(), coins.rend()); // buyukten kucuge
8     int count = 0;
9     for (int c : coins)
10         while (amount >= c) { amount -= c; count++; }
11     return amount == 0 ? count : -1; // coins={1,3,4}, amount=6 -> 3 (yanlis!)
12 }
```

Dikkat

Ağgözlü para üstü yalnızca “canonical” (kurallı) para sistemlerinde optimaldir. Rastgele bir para kümesi verildiğinde her zaman DP kullanın; aksi halde yukarıdaki gibi optimalin altında sonuçlar alırsınız.

Soru 82

Aralık birleştirme (merge intervals): Örtüşen aralıklardan oluşan bir listeyi, örtüşenleri birleştirerek sadeleştirin. Örn. $[[1, 3], [2, 6], [8, 10]] \rightarrow [[1, 6], [8, 10]]$.

Çözüm. Aralıkları başlangıç noktasına göre sıralarsak, örtüşen aralıklar mutlaka listede yan yana gelir. Sonucu tutan listeye ilk aralığı ekleriz; her yeni aralık için, son eklenen aralığın bitişi yeni aralığın başlangıcından büyük veya eşitse örtüşme var demektir – son aralığın bitişi $\max$ ile genişletiriz. Aksi halde araya boşluk vardır, yeni aralığı olduğu gibi ekleriz. Sıralama $\mathcal{O}(n \log n)$ baskın maliyettir; tarama $\mathcal{O}(n)$.

```

1 #include <vector>
2 #include <algorithm>
3 using namespace std;
4
5 vector<vector<int>> mergeIntervals(vector<vector<int>> intervals) {
6     if (intervals.empty()) return {};
7     sort(intervals.begin(), intervals.end()); // baslangica gore sirala
8     vector<vector<int>> merged;
9     merged.push_back(intervals[0]);
10    for (size_t i = 1; i < intervals.size(); ++i) {
11        if (intervals[i][0] <= merged.back()[1]) // ortusuyor
12            merged.back()[1] = max(merged.back()[1], intervals[i][1]);
13        else
14            merged.push_back(intervals[i]); // yeni ayri aralik
15    }
16    return merged;
17 }
```

31.5 Geri İzleme (Backtracking)

Soru 83

Tekrarsız bir dizinin tüm alt kümelerini (subsets / power set) üretin. n elemanlı bir kümenin 2^n alt kümesi vardır.

Çözüm. Geri izleme (backtracking) bir karar ağacını sistematik gezmektir. Her eleman için iki karar veririz: alt kümeye *ekle* ya da *ekleme*. Bir *start* indeksi taşıyarak her adımda mevcut kısmi çözümü (partial solution) kaydeder, sonra kalan her elemanı sırayla ekleyip özyineleniriz; dönüşte eklediğimizi geri alırız (pop_back) – işte “geri izleme” budur. Toplam 2^n alt küme ürettiğimiz için karmaşıklık $\mathcal{O}(n \cdot 2^n)$ (her alt kümeyi kopyalamak dahil).

```

1 #include <vector>
2 using namespace std;
3
4 void backtrack(vector<int>& nums, int start,
5               vector<int>& current, vector<vector<int>>& result) {
6     result.push_back(current);           // her dugum gecerli bir alt kume
7     for (int i = start; i < (int)nums.size(); ++i) {
8         current.push_back(nums[i]);      // sec
9         backtrack(nums, i + 1, current, result);
10        current.pop_back();              // geri al (backtrack)
11    }
12 }
13
14 vector<vector<int>> subsets(vector<int>& nums) {
15     vector<vector<int>> result;
16     vector<int> current;
17     backtrack(nums, 0, current, result);
18     return result;
19 }
```

İpucu

Alt kümeleri bit maskeleriyle de üretebilirsiniz: 0'dan $2^n - 1$ 'e her sayının ikili gösterimindeki set bitler, o alt kümeye hangi elemanların girdiğini söyler (bkz. *Bit Manipülasyonu* bölümü). Backtracking versiyonu ise “kısıt ekleme” (örn. tekrarları atlama) gerektiren durumlarda daha esnekler.

Soru 84

Tekrarsız bir dizinin tüm permütasyonlarını üretin. n eleman için $n!$ permütasyon vardır.

Çözüm. Permütasyonda sıra önemlidir, bu yüzden her pozisyona kalan tüm elemanları deneriz. Hangi elemanların kullanıldığını bir used boolean dizisiyle izleriz. Kısmi permütasyon tam uzunluğa ulaştığında sonuca ekleriz. Alternatif olarak yerinde takas (swap) yöntemiyle ekstra bellek olmadan da yapılabilir. Karmaşıklık $\mathcal{O}(n \cdot n!)$.

```

1 #include <vector>
2 using namespace std;
3
4 void permute(vector<int>& nums, vector<int>& current,
5             vector<bool>& used, vector<vector<int>>& result) {
6     if (current.size() == nums.size()) { // tam permutasyon hazır
7         result.push_back(current);
8         return;
9     }
```

```

10     for (int i = 0; i < (int)nums.size(); ++i) {
11         if (used[i]) continue;           // zaten kullanıldı
12         used[i] = true;
13         current.push_back(nums[i]);
14         permute(nums, current, used, result);
15         current.pop_back();              // geri al
16         used[i] = false;
17     }
18 }
19
20 vector<vector<int>> permutations(vector<int>& nums) {
21     vector<vector<int>> result;
22     vector<int> current;
23     vector<bool> used(nums.size(), false);
24     permute(nums, current, used, result);
25     return result;
26 }

```

Soru 85

Combination Sum: Tekrarsız pozitif candidates dizisi ve bir target verildiğinde, toplamı hedefe eşit olan tüm kombinasyonları bulun. Her aday *sınırsız* kez kullanılabilir.

Çözüm. Her adayı sınırsız kullanabildiğimiz için, bir adayı seçtikten sonra özyinelemeli çağrıda start indeksini *ilerletmeyiz* (i’de kalırız); böylece aynı aday tekrar seçilebilir. Aday listesini indeksle sınırlamak, aynı kombinasyonun farklı sıralamalarını (örn. [2,3] ve [3,2]) saymamızı engeller – yalnızca artan sırada üretiriz. Kalan hedef 0 olunca geçerli bir kombinasyon buluruz; negatife düşerse dalı buda (prune). Erken budama için adayları sıralayıp remaining - candidates[i] < 0 olunca döngüyü kırabiliriz.

```

1  #include <vector>
2  #include <algorithm>
3  using namespace std;
4
5  void findCombos(vector<int>& candidates, int remaining, int start,
6                 vector<int>& current, vector<vector<int>>& result) {
7      if (remaining == 0) { result.push_back(current); return; }
8      for (int i = start; i < (int)candidates.size(); ++i) {
9          if (candidates[i] > remaining) break; // sirali oldugu icin kalanlar da büyük
10         current.push_back(candidates[i]);
11         // i ile devam: ayni aday tekrar kullanılabilir
12         findCombos(candidates, remaining - candidates[i], i, current, result);
13         current.pop_back();                  // geri al
14     }
15 }
16
17 vector<vector<int>> combinationSum(vector<int> candidates, int target) {
18     sort(candidates.begin(), candidates.end()); // budama icin sirala
19     vector<vector<int>> result;
20     vector<int> current;
21     findCombos(candidates, target, 0, current, result);
22     return result;
23 }

```

Soru 86

N-Queens: $N \times N$ satranç tahtasına, hiçbiri birbirini tehdit etmeyecek şekilde N vezir yerleştirmenin kaç farklı yolu olduğunu bulun.

Çözüm. Vezirler aynı satır, sütun ve çaprazları paylaşamaz. Satır satır ilerleyerek her satıra tam bir vezir koyarız; böylece satır çakışması otomatik önlenir. Her sütun ve iki çapraz yönü (sol-üst ve sağ-üst) için birer boolean dizi tutarak bir konumun güvenli olup olmadığını $\mathcal{O}(1)$ 'de kontrol ederiz. Çaprazları tanımlarken $\text{row} + \text{col}$ sabiti bir çapraz ailesini, $\text{row} - \text{col}$ sabiti diğerini belirler (ikincisini negatiften kaçınmak için $\text{row} - \text{col} + N$ ile kaydırırız). Bir satıra hiçbir güvenli sütun yoksa geri izleriz. Karmaşıklık üst sınırı $\mathcal{O}(N!)$ 'dir ama budama sayesinde pratikte çok daha hızlıdır.

```

1 #include <vector>
2 using namespace std;
3
4 int solveQueens(int row, int n, vector<bool>& cols,
5               vector<bool>& diag1, vector<bool>& diag2) {
6     if (row == n) return 1; // tüm satırlara yerleştik: 1 çözüm
7     int count = 0;
8     for (int col = 0; col < n; ++col) {
9         int d1 = row + col; // sol-alt / sag-ust caprazi
10        int d2 = row - col + n; // sol-ust / sag-alt caprazi (kaydırmalı)
11        if (cols[col] || diag1[d1] || diag2[d2]) continue; // tehdit var
12        cols[col] = diag1[d1] = diag2[d2] = true; // yerleştir
13        count += solveQueens(row + 1, n, cols, diag1, diag2);
14        cols[col] = diag1[d1] = diag2[d2] = false; // geri al
15    }
16    return count;
17 }
18
19 int totalNQueens(int n) {
20     vector<bool> cols(n, false), diag1(2*n, false), diag2(2*n, false);
21     return solveQueens(0, n, cols, diag1, diag2);
22 }

```

Bilgi

Çapraz indekslemede $\text{row} + \text{col}$ değeri 0 ile $2n - 2$ arasında, $\text{row} - \text{col}$ değeri ise $-(n - 1)$ ile $n - 1$ arasında değişir; ikincisini negatif indeksten kurtarmak için $+ n$ ile kaydırırız. Bu, N-Queens'in klasik ve öğretici bit numarasıdır.

31.6 Dinamik Programlama**Soru 87**

Ev Soyguncusu (House Robber): Her evde belli miktarda para var. Yan yana iki evi soyamazsınız (alarm çalar). Soyabileceğiniz maksimum toplam parayı bulun.

Çözüm. Bu, dinamik programlamanın en temiz örneklerinden biridir. Her ev için iki seçenek var: *soy* ya da *atla*. $\text{dp}[i]$, ilk i evi göz önüne aldığımızdaki maksimum kazanç olsun. i . evi soyarsak $i - 1$ 'i soyamayız, yani $\text{dp}[i - 2] + \text{nums}[i]$ kazanırız; soymazsak $\text{dp}[i - 1]$ kadar kalırız. İkisinin maksimumunu alırız: $\text{dp}[i] = \max(\text{dp}[i - 1], \text{dp}[i - 2] + \text{nums}[i])$. Bu bağıntı “optimal substructure” içerir: büyük problemin çözümü küçük alt problemlerin çözümünden kurulur. Sadece son iki değer gerektiğinden $\mathcal{O}(1)$ bellekle, $\mathcal{O}(n)$ zamanda çözülür.

```

1 #include <vector>
2 #include <algorithm>

```

```

3 using namespace std;
4
5 int rob(vector<int>& nums) {
6     int prev2 = 0;    // dp[i-2]
7     int prev1 = 0;    // dp[i-1]
8     for (int money : nums) {
9         int cur = max(prev1, prev2 + money); // soyuma vs. soy
10        prev2 = prev1;
11        prev1 = cur;
12    }
13    return prev1;
14 }

```

Soru 88

Benzersiz Yollar (Unique Paths): $m \times n$ bir gridde sol üst köşeden sağ alt köşeye, sadece *sağa* ve *aşağı* giderek kaç farklı yol vardır?

Çözüm. Bir hücreye yalnızca yukarıdan ya da soldan gelinebilir. Dolayısıyla o hücreye varan yol sayısı, üstteki ve soldaki hücrelere varan yolların toplamıdır: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. İlk satır ve ilk sütundaki her hücreye tek yol vardır (hep düz git). Tabloyu satır satır doldururuz. 2B tablo $\mathcal{O}(mn)$ bellek kullanır ama sadece bir önceki satıra ihtiyaç olduğundan tek boyutlu diziyle $\mathcal{O}(n)$ belleğe indirilebilir (aşağıdaki gibi). Zaman $\mathcal{O}(mn)$.

```

1 #include <vector>
2 using namespace std;
3
4 int uniquePaths(int m, int n) {
5     vector<int> dp(n, 1);           // ilk satir: her hucreye 1 yol
6     for (int i = 1; i < m; ++i)
7         for (int j = 1; j < n; ++j)
8             dp[j] += dp[j - 1];     // ust(dp[j]) + sol(dp[j-1])
9     return dp[n - 1];
10 }

```

Bilgi

Bu problemin kapalı formu da vardır: sağ alt köşeye ulaşmak için toplam $(m - 1) + (n - 1)$ hamlenin $(m - 1)$ 'ini “aşağı” seçmek gerektiğinden cevap $\binom{m+n-2}{m-1}$ 'dir. DP çözümü ise engelli grid gibi varyasyonlara kolayca genişler.

Soru 89

Kelime Kırma (Word Break): Bir s dizgesi ve bir sözlük verildiğinde, s 'in sözlükteki kelimelerin (tekrarlı) birleşimiyle bölünüp bölünemeyeceğini belirleyin.

Çözüm. $dp[i]$, s 'in ilk i karakterinin sözlük kelimeleriyle bölünebilir olup olmadığını tutsun. $dp[0] = \text{true}$ (boş dizge her zaman bölünür). Her i için, ondan önceki her j noktasına bakarız: eğer $dp[j]$ doğruysa (yani $[0, j]$ bölünüyorsa) ve $s[j..i]$ alt dizgesi sözlükteyse, o zaman $dp[i]$ de doğrudur. Sözlüğü `unordered_set`'e koyarak kelime kontrolünü $\mathcal{O}(1)$ 'e yaklaştırırız (kelime uzunluğu sabit varsayımıyla). Karmaşıklık $\mathcal{O}(n^2)$ alt dizge çifti $\times$ kontrol maliyeti.

```

1 #include <string>
2 #include <vector>
3 #include <unordered_set>
4 using namespace std;
5

```

```

6 bool wordBreak(string s, vector<string>& wordDict) {
7     unordered_set<string> dict(wordDict.begin(), wordDict.end());
8     int n = (int)s.size();
9     vector<bool> dp(n + 1, false);
10    dp[0] = true; // bos dizge bolunebilir
11    for (int i = 1; i <= n; ++i) {
12        for (int j = 0; j < i; ++j) {
13            if (dp[j] && dict.count(s.substr(j, i - j))) {
14                dp[i] = true; // [0,j) bolunuyor ve [j,i) sozlukte
15                break;
16            }
17        }
18    }
19    return dp[n];
20 }

```

Soru 90

Madeni Para – Minimum (Coin Change): Sınırsız sayıda madeni para türü ve bir amount verildiğinde, tutarı oluşturmak için gereken minimum para sayısını bulun; mümkün değilse -1 döndürün.

Çözüm. Açgözlü bölümünde gördüğümüz gibi, greedy bu problemde genelde yanlır; doğru çözüm DP'dir. $dp[x]$, x tutarını ödemek için gereken minimum para olsun. $dp[0] = 0$. Her x tutarı için tüm para türlerini deneriz: $dp[x] = \min(dp[x], dp[x - coin] + 1)$. Yani “ $x - coin$ ’i optimal ödedikten sonra bir para daha ekle”. Ulaşılamayan tutarları “sonsuz” (örn. $amount + 1$) ile işaretleriz; sonuç hâlâ sonsuzsa -1 döneriz. Bu “bottom-up” (aşağıdan yukarı) yaklaşım tüm alt tutarları küçükten büyüğe çözer. Karmaşıklık $\mathcal{O}(\text{amount} \times \text{coins})$.

```

1 #include <vector>
2 #include <algorithm>
3 using namespace std;
4
5 int coinChange(vector<int>& coins, int amount) {
6     vector<int> dp(amount + 1, amount + 1); // "sonsuz" isaretcisi
7     dp[0] = 0; // 0 tutar için 0 para
8     for (int x = 1; x <= amount; ++x)
9         for (int coin : coins)
10            if (coin <= x)
11                dp[x] = min(dp[x], dp[x - coin] + 1);
12    return dp[amount] > amount ? -1 : dp[amount];
13 }

```

Soru 91

En Uzun Artan Alt Dizi (Longest Increasing Subsequence, LIS): Bir dizideki en uzun kesin artan alt dizinin uzunluğunu $\mathcal{O}(n \log n)$ zamanda bulun.

Çözüm. Basit DP çözümü $\mathcal{O}(n^2)$ 'dir ($dp[i] = i$ 'de biten LIS). Daha hızlı yöntem, “patience sorting” fikrine dayanır: $tails[k]$, uzunluğu $k + 1$ olan artan alt dizilerin bitebileceği en küçük son elemanı tutar. Bu dizi her zaman sıralıdır. Her yeni eleman için, $tails$ içinde onu yerleştirebileceğimiz ilk konumu ikili aramayla (lower_bound) buluruz: o konumdaki değeri günceller (daha küçük bir sonla aynı uzunluğu koru) ya da diziyi bir uzatırız. $tails$ ’in nihai boyutu LIS uzunluğudur. İkili arama sayesinde $\mathcal{O}(n \log n)$.

Dikkat: $tails$ dizisi gerçek LIS’i içermez, yalnızca uzunluğunu verir; gerçek diziyi geri kurmak için ek işaretçiler gerekir.

```

1 #include <vector>
2 #include <algorithm>
3 using namespace std;
4
5 int lengthOfLIS(vector<int>& nums) {
6     vector<int> tails; // tails[k]: uzunluk k+1 için en küçük son
7     for (int x : nums) {
8         // x'i yerleştirebileceğimiz ilk konum (kesin artan için lower_bound)
9         auto it = lower_bound(tails.begin(), tails.end(), x);
10        if (it == tails.end())
11            tails.push_back(x); // LIS'i bir uzat
12        else
13            *it = x; // daha küçük sonla aynı uzunluğu koru
14    }
15    return (int)tails.size();
16 }

```

Dikkat

Kesin artan (strictly increasing) için `lower_bound`, artmayan/azalmayan (non-decreasing) için `upper_bound` kullanılır. Yanlış olanı seçmek eşit elemanlarda bir birim hataya yol açar.

31.7 String

Soru 92

KMP (Knuth-Morris-Pratt) algoritmasıyla bir metinde bir örüntünün (pattern) tüm eşleşme başlangıçlarını $\mathcal{O}(n + m)$ zamanda bulun.

Çözüm. Kaba kuvvet arama, uyumsuzlukta metinde bir adım geri kayar ve tekrar dener; bu $\mathcal{O}(nm)$ 'dir. KMP, örüntünün kendi iç yapısından yararlanır: uyumsuzluk olduğunda, örüntünün başına dönmek yerine, o ana kadar eşleşen ön ekin *en uzun uygun ön-son eki* (longest proper prefix which is also suffix, LPS) kadar geri atlar. LPS dizisini önceden $\mathcal{O}(m)$ 'de hesaplarız; sonra metni tek geçişte tararız, metin işaretçisi hiç geri gitmez. Toplam $\mathcal{O}(n + m)$.

LPS[i], örüntünün $[0..i]$ ön ekinin, hem baş hem son olan en uzun (kendisi hariç) parçasının uzunluğudur. Bu, “uyumsuzlukta örüntünün neresinden devam edebilirim” sorusunun cevabıdır.

```

1 #include <string>
2 #include <vector>
3 using namespace std;
4
5 vector<int> buildLPS(const string& p) {
6     vector<int> lps(p.size(), 0);
7     int len = 0; // önceki en uzun ön-son eki
8     for (int i = 1; i < (int)p.size(); ) {
9         if (p[i] == p[len]) lps[i++] = ++len; // uzat
10        else if (len > 0) len = lps[len - 1]; // geri atla (yeniden dene)
11        else lps[i++] = 0; // eşleşme yok
12    }
13    return lps;
14 }
15
16 vector<int> kmpSearch(const string& text, const string& pattern) {
17     vector<int> lps = buildLPS(pattern), matches;
18     int i = 0, j = 0; // i: metin, j: oruntu
19     while (i < (int)text.size()) {
20         if (text[i] == pattern[j]) { i++; j++; }

```

```

21     if (j == (int)pattern.size()) {           // tam eslesme
22         matches.push_back(i - j);
23         j = lps[j - 1];                       // sonraki eslesmeyi ara
24     } else if (i < (int)text.size() && text[i] != pattern[j]) {
25         if (j > 0) j = lps[j - 1];           // oruntude geri atla
26         else i++;                             // metinde ilerle
27     }
28 }
29 return matches;
30 }

```

Soru 93

Rabin-Karp / rolling hash: Bir metinde bir örüntüyü, kayan (rolling) hash kullanarak ortalama $\mathcal{O}(n + m)$ zamanda arayın.

Çözüm. Fikir, örüntünün hash değerini bir kez hesaplayıp, metin üzerinde m uzunluklu her pencerenin hash'ini karşılaştırmaktır. Her pencereyi sıfırdan hash'lemek $\mathcal{O}(m)$ olurdu; “rolling hash” ise pencereyi bir karakter kaydırırken hash'i $\mathcal{O}(1)$ 'de günceller: baştaki karakterin katkısını çıkar, hepsini tabana (base) çarp, yeni karakteri ekle. Bu bir polinom hash'idir: $h = \sum c_i \cdot b^{m-1-i} \bmod M$. Hash'ler eşitse (olası çakışmaya – “spurious hit” – karşı) karakterleri doğrulamak için birebir karşılaştırma yaparız. Büyük bir asal MOD ve iyi bir BASE ile çakışma nadirdir; ortalama $\mathcal{O}(n + m)$, en kötü (kötü hash) $\mathcal{O}(nm)$.

```

1  #include <string>
2  #include <vector>
3  using namespace std;
4
5  vector<int> rabinKarp(const string& text, const string& pattern) {
6      const long long BASE = 256, MOD = 1000000007;
7      int n = (int)text.size(), m = (int)pattern.size();
8      vector<int> matches;
9      if (m > n) return matches;
10
11      long long patHash = 0, winHash = 0, power = 1;
12      for (int i = 0; i < m; ++i) {           // ilk pencere + oruntu hash'i
13          patHash = (patHash * BASE + pattern[i]) % MOD;
14          winHash = (winHash * BASE + text[i]) % MOD;
15          if (i < m - 1) power = (power * BASE) % MOD; // BASE^(m-1)
16      }
17      for (int i = 0; i + m <= n; ++i) {
18          if (patHash == winHash &&
19              text.compare(i, m, pattern) == 0) // dogrula (cakismaya karsi)
20              matches.push_back(i);
21          if (i + m < n) {                     // pencereyi kaydir
22              winHash = (winHash - text[i] * power % MOD + MOD) % MOD; // basi cikar
23              winHash = (winHash * BASE + text[i + m]) % MOD;           // yeni karakteri ekle
24          }
25      }
26      return matches;
27 }

```

Bilgi

Tek modüllü hash'e karşı özel olarak tasarlanmış girdiler çakışma saldırısı yapabilir. Yarışma programlamada güvenlik için genelde iki farklı (BASE, MOD) çifti kullanılıp her iki hash de eşleşince kabul edilir (“double hashing”).

Soru 94

En uzun palindromik alt dizge (longest palindromic substring): Merkezden genişleme (expand around center) yöntemiyle bulun.

Çözüm. Her palindromun bir merkezi vardır. Tek uzunluklu palindromların merkezi tek bir karakter, çift uzunluklu palindromların merkezi iki karakter arasındır. n karakterlik dizgede $2n - 1$ olası merkez vardır. Her merkezden başlayıp sol ve sağ işaretçileri dışa doğru açarak, karakterler eşit olduğu sürece palindromu genişletiriz. En uzun bulunamı saklarız. Her genişleme $\mathcal{O}(n)$, toplam merkez $2n - 1$ olduğu için $\mathcal{O}(n^2)$ zaman, $\mathcal{O}(1)$ bellek. (Manacher algoritması $\mathcal{O}(n)$ 'dir ama expand-around-center hem yazması hem anlaması çok daha kolaydır.)

```

1  #include <string>
2  using namespace std;
3
4  // left, right merkezinden disa dogru genisle; palindrom uzunlugunu dondur
5  int expand(const string& s, int left, int right) {
6      while (left >= 0 && right < (int)s.size() && s[left] == s[right]) {
7          left--; right++;
8      }
9      return right - left - 1;    // son gecerli uzunluk
10 }
11
12 string longestPalindrome(const string& s) {
13     if (s.empty()) return "";
14     int start = 0, maxLen = 1;
15     for (int i = 0; i < (int)s.size(); ++i) {
16         int len1 = expand(s, i, i);        // tek uzunluklu merkez
17         int len2 = expand(s, i, i + 1);    // çift uzunluklu merkez
18         int len = max(len1, len2);
19         if (len > maxLen) {
20             maxLen = len;
21             start = i - (len - 1) / 2;    // baslangic indeksini geri hesapla
22         }
23     }
24     return s.substr(start, maxLen);
25 }
```

İpucu

$start = i - (len - 1) / 2$ formülü hem tek hem çift uzunluklu palindromlarda doğru başlangıcı verir. Küçük bir örnekle (örn. "babad" ve "cbbd") elle doğrulamak, bu indeks aritmetiğini içselleştirmenin en iyi yoludur.

31.8 Bit Manipülasyonu**Soru 95**

Bir tam sayının ikili gösteriminde kaç bitin set (1) olduğunu (popcount / Hamming weight) sayın. Bit sayısı kadar dönen verimli bir yöntem kullanın.

Çözüm. Naif yöntem her bit için 32 tur döner. Brian Kernighan numarası ise yalnızca *set olan bit sayısı* kadar döner: $n \& (n - 1)$ ifadesi, n 'in en sağdaki (en düşük anlamlı) set bitini temizler. Neden? $n - 1$, en sağdaki 1'i 0 yapar ve ondan sağdaki tüm 0'ları 1'e çevirir; n ile AND'lediğimizde o en sağdaki 1 ve sağı sıfırlanır. Bu işlemi n sıfır olana dek tekrarlayıp kaç kez yaptığımızı sayarız. Karmaşıklık: set bit sayısı ile orantılı, en kötü $\mathcal{O}(\text{bit sayısı})$.

```

1 int countSetBits(unsigned int n) {
2     int count = 0;
3     while (n) {
4         n &= (n - 1);    // en sagdaki set biti temizle
5         count++;
6     }
7     return count;
8     // Alternatif: C++20'de std::popcount(n), derleyici built-in'i __builtin_popcount(n)
9 }

```

Soru 96

Tek Eleman (Single Number): Bir dizide her eleman iki kez geçiyor, sadece biri bir kez. Sabit bellekle ve tek geçişte o tek elemanı bulun.

Çözüm. XOR işleminin iki temel özelliğini kullanırız: (1) $x \oplus x = 0$ (bir sayı kendisiyle XOR'lanınca sıfırlanır), (2) $x \oplus 0 = x$. Ayrıca XOR birleşmeli (associative) ve değişmelidir (commutative), yani sıra önemsizdir. Tüm elemanları XOR'ladığımızda, iki kez geçen her eleman birbirini götürüp 0 olur; geriye yalnızca tek geçen eleman kalır. $\mathcal{O}(n)$ zaman, $\mathcal{O}(1)$ bellek – hash tablosuna bile gerek yok.

```

1 #include <vector>
2 using namespace std;
3
4 int singleNumber(vector<int>& nums) {
5     int result = 0;
6     for (int x : nums)
7         result ^= x;    // eslesenler birbirini goturur, tek kalan doner
8     return result;
9 }

```

Bilgi

Aynı fikir varyantlara genişler: iki tek eleman varsa, önce hepsini XOR'layıp sonucun herhangi bir set bitine göre diziyi iki gruba ayırırsınız. “Her eleman üç kez, biri bir kez” varyantı ise bit bazında mod3 sayımıyla çözülür.

Soru 97

Bir kümenin tüm alt kümelerini bitmask (bit maskesi) ile üretin. n elemanlı küme için 2^n alt kümeyi döngüyle listeleyin.

Çözüm. n elemanlı bir kümenin her alt kümesini, n bitlik bir sayının ikili gösterimiyle birebir eşleyebiliriz: i . bit set ise i . eleman alt kümededir. 0'dan $2^n - 1$ 'e kadar tüm sayıları gezersek tüm alt kümeleri üretmiş oluruz. Bir mask için hangi elemanların dahil olduğunu, her bit pozisyonunu mask & $(1 \ll i)$ ile kontrol ederek buluruz. Bu, backtracking'e göre daha az kod ve daha az özyineleme yükü demektir. Karmaşıklık $\mathcal{O}(n \cdot 2^n)$. Bit sayacı olarak unsigned/long long kullanın; $n \leq 63$ ile sınırlıdır.

```

1 #include <vector>
2 using namespace std;
3
4 vector<vector<int>> allSubsets(vector<int>& nums) {
5     int n = (int)nums.size();
6     vector<vector<int>> result;
7     for (int mask = 0; mask < (1 << n); ++mask) {    // 0 .. 2^n - 1
8         vector<int> subset;

```

```

9      for (int i = 0; i < n; ++i)
10         if (mask & (1 << i))                // i. bit set mi?
11             subset.push_back(nums[i]);
12         result.push_back(subset);
13     }
14     return result;
15 }

```

İpucu

Bir maskenin *tüm alt maskelerini* (submask) gezmek için özel bir kalıp vardır: `for (int sub = mask; sub > 0; sub = (sub - 1) & mask)`. Bu, alt küme-toplamı DP'lerinde çok işe yarar ve tüm mask/submask çiftlerini $\mathcal{O}(3^n)$ toplamda gezer.

Soru 98

İki'nin Kuvveti mi? Bir pozitif tam sayının tam olarak 2'nin bir kuvveti olup olmadığını tek bir bit işlemiyle kontrol edin.

Çözüm. 2'nin kuvvetleri (1, 2, 4, 8, ...) ikilik tabanda tam olarak *bir* set bite sahiptir: 100...0. Böyle bir sayıdan 1 çıkarınca, o tek set bit 0 olur ve sağındaki tüm bitler 1'e döner: örn. 8 = 1000, 7 = 0111. Bu ikisini AND'lersek sonuç 0 olur. Birden fazla set biti olan sayılarda ise AND sıfır çıkmaz. Sıfır ve negatifi elemek için $n > 0$ kontrolünü eklemek şarttır (aksi halde $n = 0$ yanlışlıkla "kuvvet" sanılır). Tek karşılaştırma, $\mathcal{O}(1)$.

```

1 bool isPowerOfTwo(int n) {
2     // n > 0 olmalı; tek set bitli sayılar için n & (n-1) == 0
3     return n > 0 && (n & (n - 1)) == 0;
4 }

```

Bit numaraları özeti

$n \& (n-1)$ en sağdaki set biti siler (popcount, 2-kuvvet testi). $n \& (-n)$ en sağdaki set biti *izole eder* (Fenwick/BIT ağaçlarında kullanılır). $n \mid (n+1)$ en sağdaki 0'ı set eder. Bu küçük özdeşlikleri ezberlemek, düşük seviyeli optimizasyonlarda ve yarışma problemlerinde büyük kolaylık sağlar.

32. Kapanış: Doğru Aracı Seçmek

Bu rehber boyunca veri yapılarını ve algoritmaları tek tek gördük. Gerçek problemlerde asıl beceri, hangi aracın ne zaman kullanılacağını sezmektir. Aşağıdaki tablo hızlı bir başvuru sunar.

İhtiyaç	Uygun araç
Hızlı indeksli erişim	Dizi / vector
Sık baş/orta ekleme-silme	Bağlı liste / list
Anahtarla $O(1)$ arama	Hash tablosu / unordered_map
Sıralı anahtar + aralık sorgusu	Dengeli ağaç / map
En büyük/küçüğe hızlı erişim	Heap / priority_queue
Önek arama, otomatik tamamlama	Trie
Aralık toplamı + güncelleme	Segment / Fenwick ağacı
En kısa yol (ağırlıksız)	BFS
En kısa yol (pozitif ağırlık)	Dijkstra
Bağlantı/döngü sorgusu	Union-Find (DSU)
Örtüşen alt problemler	Dinamik programlama
Tüm kombinasyonlar + budama	Geri izleme

İpucu

Bir probleme yaklaşırken sıra: (1) *Girdi boyutuna* bak — hedeflenen karmaşıklığı söyler. (2) Problemi bildiğin bir kalıba benzet. (3) Basit (brute-force) çözümü yaz, sonra veri yapısı veya DP ile hızlandır. (4) Kenar durumlarını (boş girdi, tek eleman, taşma) test et. Bol pratik, bu sezgiyi geliştiren tek yoldur.